

 目录

§0 智能体的原理架构

- §0.0 本章定位
- §0.1 模型层：LLM 怎么工作
- §0.2 接口层：API 协议与字节流
- §0.3 记忆层：完整存储架构
- §0.4 智能体层：思考与执行原理
- §0.5 整体架构：现代主流 Agent 蓝图
- §0.6 全书坐标
- §0.7 本章小结
- §0.8 反模式速记
- §0.9 术语速查
- §0.10 推荐下一章

§1 ReAct：把基础流程节点 ②③ 显式化

- §1.0 本章定位
- §1.1 没有 ReAct 之前：纯提示词时代的窘境
- §1.2 ReAct 论文核心贡献：把"思考"和"行动"解耦
- §1.3 思考-行动-观察循环：三步缺一不可
- §1.4 ReAct 提示词模板逐字段拆解
- §1.5 ReAct 兄弟范式：补什么不足
- §1.6 ReAct 的"原生竞争方案"：Function Calling 与 Tool Use
- §1.7 历史名作：AutoGPT 与 BabyAGI
- §1.8 ReAct 局限性：每个局限对应基础流程的哪个节点
- §1.9 业界用 ReAct 的真实场景
- §1.10 代际演进图：从 ReAct 到 2026（5 代叠加模型）
- §1.11 本章小结
- §1.12 反模式速记
- §1.13 术语速查
- §1.14 推荐下一章

§2 LangChain：第一代框架的心智模型

- §2.0 本章定位
- §2.1 LangChain 解决了什么 — 2022 末的工程痛点

- §2.2 历史脉络：v0.0 → v0.3 的四次重构
- §2.3 核心抽象六件套对应到流程节点
- §2.4 LCEL 表达式语言：管道符的设计哲学
- §2.5 包结构三层：core / langchain / community / partners
- §2.6 Runnable 接口：LCEL 背后的统一抽象
- §2.7 记忆模块四种深入对比
- §2.8 文档加载器与文本分割器
- §2.9 输出解析器三类与现代 with_structured_output
- §2.10 缓存：LLM cache + 嵌入 cache
- §2.11 流式：astream / stream
- §2.12 重试与回退：with_retry / with_fallbacks
- §2.13 bind_tools 现代工具绑定
- §2.14 LangSmith 观测平台
- §2.15 业界目前怎么用 LangChain
- §2.16 本章小结
- §2.17 反模式速记
- §2.18 术语速查
- §2.19 推荐下一章

§3 用 LangChain 实现 ReAct（流程跑通实战）

- §3.0 本章定位
- §3.1 老接口（已废弃）：AgentExecutor + initialize_agent
- §3.2 现代做法：Tool Calling Agent + LCEL
- §3.3 完整代码：50 行从 prompt 到 agent 到调用
- §3.4 节点 ② 输出可见性：怎么看模型的"思考"过程
- §3.5 LangChain 实现 ReAct 的硬伤逐节点拆解
- §3.6 回调（Callbacks）系统
- §3.7 错误处理：OutputParserException 捕获
- §3.8 业界目前怎么用 LangChain：迁出老接口
- §3.9 从 LangChain 老接口迁移到 LangGraph 的官方迁移指南
- §3.10 本章小结
- §3.11 反模式速记
- §3.12 术语速查
- §3.13 推荐下一章

§4 LangGraph：第二代框架的心智模型

§4.0 本章定位

§4.1 LangGraph 起源：2024 年 LangChain 团队的反思

§4.2 核心模型转变：从"链"到"图"

§4.3 三要素深入：状态 / 节点 / 边

§4.4 超步并行执行模型（Pregel）

§4.5 五行最小图（Hello World）

§4.6 编译与可视化：StateGraph vs CompiledGraph

§4.7 业界目前怎么用 LangGraph：典型案例

§4.8 本章小结

§4.9 反模式速记

§4.10 术语速查

§4.11 推荐下一章

§5 用 LangGraph 实现 ReAct（同任务对比 §3）

§5.0 本章定位

§5.1 手写 ReAct 循环：State + 条件边

§5.2 prebuilt create_react_agent

§5.3 检查点：三种保存器 + thread_id 续跑

§5.4 人在回路：interrupt() 与 NodeInterrupt

§5.5 与 §3 LangChain 实现的逐行对比表

§5.6 GraphRecursionError 与递归保护

§5.7 时间旅行：分支重放

§5.8 业界目前怎么用 LangGraph

§5.9 本章小结

§5.10 反模式速记

§5.11 术语速查

§5.12 推荐下一章

§6 LangGraph 进阶（为生产环境完善每个节点）

§6.0 本章定位

§6.1 节点 ⑥ 状态深入：State / Reducer / add_messages / Command

§6.2 节点 ⑦ 高级循环：子图与 Send API

§6.3 多智能体模式：Supervisor / Plan-Execute / Swarm

§6.4 函数式接口：@entrypoint / @task（0.2 引入）

§6.5 长期记忆：Store API + BaseStore

§6.6 节点 ⑧ 流式输出五模式 + LangSmith 集成

§6.7 节点 ④ 工具集成：@tool / ToolNode / MCP 适配器

§6.8 通道（Channels）深入

§6.9 Send vs add_conditional_edges 选择决策

§6.10 LangGraph Studio：可视化调试

§6.11 LangGraph Server / Platform / Cloud 三种部署形态

§6.12 LangGraph SDK 客户端库

§6.13 后台任务 + 定时调度（Cron）

§6.14 检查点的索引策略 + 容量管理

§6.15 混合架构：LangChain 组件 + LangGraph 编排（业界默认答案）

§6.16 本章小结

§6.17 反模式速记

§6.18 术语速查

§6.19 推荐下一章

§7 三者深度对比 + 业界主流玩法

§7.0 本章定位

§7.1 同任务三种实现并列：纯 ReAct prompt / LangChain / LangGraph

§7.2 8 节点逐节点对比表（核心对照）

§7.3 选型决策树（Mermaid #19）

§7.4 多维对比表

§7.5 成本对比：token / 调用费用

§7.6 延迟对比：串行 / 并行 / 流式

§7.7 测试策略对比

§7.8 错误处理与重试策略对比

§7.9 版本与迁移：从 LangChain v0.x → LangGraph 0.x

§7.10 2026 年初业界真实选型快照

§7.11 何时不该用三件套

§7.12 本章小结

§7.13 反模式速记

§7.14 术语速查

§7.15 推荐下一章

§8 用三件套落地 custom_agent（实现视角）

§8.0 本章定位

§8.1 现有架构速览

§8.2 8 节点流程到用户工程模块的映射表

§8.3 接入点决策矩阵

§8.4 与现有 RAG / Skills / Sessions / KB 的关系

§8.5 渐进式迁移路径：3 步走

§8.6 部署形态四选一

§8.7 Sessions 与 thread_id 映射设计

§8.8 Skills 与 LangGraph 节点 / 子图的关系

§8.9 API 兼容层设计：既有 RESTful 接口怎么挂 LangGraph

§8.10 多租户隔离：thread_id 命名空间 + checkpoint 表分区

§8.11 当前不必上的判断条件

§8.12 本章小结

§8.13 反模式速记

§8.14 术语速查

§8.15 推荐下一章

§9 全景速览

§9.0 本章定位

§9.1 16 大类对应到 8 节点（全景图）

§9.2 看图视角：按"功能定位"而非"流量大小"

§9.3 各类关系：互补 / 替代 / 横切

§9.4 本手册使用法

§9.5 本章小结

§9.6 反模式速记

§9.7 术语速查

§9.8 推荐下一章

§10 通用框架家族

§10.0 本章定位

§10.1 第一代框架：LangChain / Haystack / Semantic Kernel

§10.2 第二代状态图：LangGraph / Burr / Inngest Agent Kit

§10.3 多智能体专门：CrewAI / AutoGen / OpenAI Agents SDK / Camel-AI / MetaGPT

§10.4 类型安全派：PydanticAI / Instructor / Marvin

§10.5 编译式：DSPy / TextGrad / Trace

§10.6 各派别在 8 节点上的着力点对比表 (Mermaid #29)

§10.7 Anthropic Building Effective Agents 五大模式

§10.8 本章小结

§10.9 反模式速记 + 术语速查 + 推荐下一章

§11 领域专门派

§11.0 本章定位

§11.1 检索增强生成 (RAG) 框架

§11.2 编程类智能体

§11.3 业界为什么这两个领域要"自立门户"

§11.4 计算机使用 / 浏览器使用: Anthropic Computer Use / Browser Use

§11.5 语音智能体

§11.6 多模态智能体

§11.7 本章小结 + 反模式 + 术语 + 推荐下一章

§12 JavaScript / TypeScript 平行宇宙

§12.0 本章定位

§12.1 Vercel AI SDK

§12.2 Mastra

§12.3 Inngest Agent Kit

§12.4 LangChain.js / LangGraph.js (采用率真相)

§12.5 Cloudflare Workers AI + Agents

§12.6 双宇宙的技术与组织原因

§12.7 本章小结 + 反模式 + 术语 + 推荐下一章

§13 低代码与可视化

§13.0 本章定位

§13.1 国际: Flowise / Langflow / n8n

§13.2 中国生态

§13.3 适用边界

§13.4 本章小结 + 反模式 + 术语 + 推荐下一章

§14 云厂商平台

§14.0 本章定位

§14.1 AWS Bedrock Agents

§14.2 Google Vertex AI Agent Builder

§14.3 Azure AI Foundry (前 AI Studio)

§14.4 Databricks Agent Framework

§14.5 厂商锁定 vs 开箱即用

§14.6 本章小结 + 反模式 + 术语 + 推荐下一章

§15 基础设施层 (横切节点 ⑥ + ⑧)

§15.0 本章定位

§15.1 沙箱与运行时: E2B / Daytona / Modal / Replicate

§15.2 评估 (Evaluation)

§15.3 观测 (Observability)

§15.4 提示词管理工具

§15.5 成本观测: FinOps for Agents

§15.6 业界为什么把这层独立出来 (Mermaid #30)

§15.7 本章小结 + 反模式 + 术语 + 推荐下一章

§16 协议与标准 (重做节点 ④ 工具调用)

§16.0 本章定位

§16.1 模型上下文协议 (MCP) —— Anthropic 推出的事实标准

§16.2 智能体到智能体协议 (A2A) / AGNTCY

§16.3 Anthropic Skills 模式

§16.4 MCP 服务器开发实战

§16.5 本章小结 + 反模式 + 术语 + 推荐下一章

§17 2025–2026 新兴趋势

§17.0 本章定位

§17.1 智能体即代码 (Agent as Code)

§17.2 常驻智能体 (Ambient Agent)

§17.3 MCP 服务器生态爆发

§17.4 编译式提示词 (DSPy 范式扩展到主流)

§17.5 提示词缓存 (Prompt Caching)

§17.6 延展思考模型 (Reasoning Models) 对智能体设计的冲击

§17.7 本章小结 + 反模式 + 术语 + 推荐下一章

§18 选型决策矩阵

§18.0 本章定位

§18.1 按团队语言栈

§18.2 按场景

§18.3 按部署形态

§18.4 按团队规模

§18.5 总决策树

§18.6 以观测为先（observability-first）选型

§18.7 本章小结 + 反模式 + 术语 + 推荐下一章

§19 全栈对照 custom_agent（选型视角）

§19.0 本章定位

§19.1 项目当前栈映射到生态全景图 + 8 节点流程

§19.2 缺什么：评估 / 观测 / MCP 接入是优先级最高的三个空白

§19.3 建议优先级：不动 / 评估 / 立即引入

§19.4 一年路线图建议

§19.5 MCP 接入路径：未来对接 Claude Code / Cursor 客户端

§19.6 评估与回归测试基础设施缺口：Promptfoo 最小可行实现

§19.7 本章小结 + 反模式 + 术语 + 全书完结语

附录 A — 接口速查卡

LangChain LCEL 速查

LangGraph StateGraph 速查

附录 B — 参考链接

论文

官方文档

业界博客

附录 C — 各章反模式汇总速查

附录 D — 版本时间线

LangChain

LangGraph

生态里程碑

附录 E — 完整生态版本快照表（2026.04）

附录 F — 中外生态对比表

附录 G — Mermaid 索引（31 张）

附录 H — 全书英文术语总表

A

B

C

D / E

F / G / H

I / J / L

M / N / O

P / Q / R

S

T / V / W / Z / 其他

附录 I — Anthropic Building Effective Agents 五大模式映射 8 节点

模式 1: 提示词链 (Prompt Chaining)

模式 2: 路由 (Routing)

模式 3: 并行化 (Parallelization)

模式 4: 协调-工作者 (Orchestrator-Workers)

模式 5: 评估-优化 (Evaluator-Optimizer)

五模式组合

附录 J — LangChain → LangGraph 迁移速查表

API 对照

Memory → Checkpointer 迁移

完整迁移示例

附录 K — OpenAI / Anthropic / Google 工具调用三家协议对照

OpenAI Function Calling (2023.06) / Tools (2023.11)

Anthropic Tool Use (2024.05)

Google Function Declarations (2024)

字段对照表

跨厂商兼容工具 (推荐)

Agent 框架系统教程 + 全景手册

「单文件教程 · 含 Part A 详细教程 + Part B 生态全景手册 · 共 §0-§19 + 附录 A-K

版本 2026.04 · 适配 LangChain 0.3.x / LangGraph 0.2.x · 适配 Claude Sonnet 4.6 / Opus 4.7

本书贯穿"智能体执行闭环 8 节点"作为主线 —— 所有原理、框架、特性、生态分类都挂到这条主线上。读者读完后形成"一条流程，多种实现"的统一心智模型。

Part A — 详细教程

「本部分系统讲透三件套：推理范式 ReAct (§1) + 第一代框架 LangChain (§2-§3) + 第二代框架 LangGraph (§4-§6)，并以 §7 横向对比、§8 落地用户工程收尾。

§0 智能体的原理架构

「本章是全书地基 —— 从最底层的 LLM 工作原理，一路讲到生产级 Agent 架构的完整 6 层蓝图，重点覆盖记忆与存储体系。

§0.0 本章定位

项	内容
章节定位	全书地基章，把智能体从"模型层 → 接口层 → 记忆层 → 智能体层 → 整体架构"5层完整原理讲透
与上下章的因果链	起始章。读完后能从"LLM 内部 4 步"一路讲到"6 层 Graph-based Agent 怎么搭"，含三层记忆体系（短期 / 中期 / 长期）
学完能做什么	(1) 看懂 LLM 推理 4 步与"概率分布展开"洞察；(2) 看懂 API 字节流 + 工具协议 + SSE；(3) 掌握三层记忆体系完整工程实现 （Context+KV Cache / Memory+Checkpoint / Vector DB+Store）；(4) 解释 CoT / ReAct / 推理模型的实现机理；(5) 看懂主流 6 层 Graph-based Agent 架构与 8 节点 / 5 代叠加模型的映射
本章地图	§0.1 模型层 → §0.2 接口层 → §0.3 记忆层（重点） → §0.4 智能体层 → §0.5 整体架构 → §0.6 全书坐标

§0.1 模型层：LLM 怎么工作

「基础流程对应：节点 ② 推理的内部机制。」

LLM 推理 4 步流水线

CODE

Tokenizer → Token IDs → ① Embedding → ② Self-Attention → ③ TransformerN 0(n²)

每生成一个 token 重复一次这 4 步，直到达到 stop 条件或 max_tokens。

Token IDs 到底是什么？

「关键澄清：Token IDs 既不是原本文字符，也不是向量 —— 它是一串整数。

具体来说，模型看不到字符（如 "你"），也看不到向量 —— 它只能读一串整数 ID（如 [100, 101, 102, 103, 104, 105]）。

阶段	数据形态	例子
用户输入	字符串（原文本）	你好，世界！
Tokenizer 切分后	整数 ID 序列（Token IDs）	[100, 101, 102, 103, 104, 105]
Embedding 后	d 维浮点向量序列	[[0.1, 0.2, ..., 0.9], [0.3, 0.4, ..., 0.8], ..., [0.7, 0.8, ..., 0.9]]
Sample 输出	一个整数 ID	106
Detokenize 后	字符串	你

完整数据流：

CODE

```
tokenizer.encode()
[123, 456, 789, 1011] ← Token IDs (tokenizer.encode(token 1))
↓ ① Embedding
[0.5, 0.2, -0.1, ...], (d = 4096-12288, embedding_dim)
↓ ② Self-Attention
Transformer
↓ ④ Sample
tokenizer.decode()
```

为什么必须先变成整数？

角度	解释
神经网络只懂数字	字符是符号、模型本质是数字运算
整数比字符串紧凑	100 个英文字 ≈ 25 个 token = 25 个 32 位整数
词表是整数索引	所有 50k-200k 个 token 都有唯一 ID，模型权重按 ID 查

Token ID 与 Embedding 向量的关系

概念	数据形式	大小	用途
Token ID	整数（如 12345）	4 字节	tokenizer 切分得到，是模型的输入格式
Embedding 向量	d 维浮点数（如 [0.5, 0.2, -0.1, ...]）	16-50 KB	模型内部第 1 层把 ID 查表得到，模型实际计算的对象

模型权重里有一张大表（embedding matrix，形状 `[[[?][?][?][?][?][?][?][?][?][?]]`），输入 token ID 后从这张表"查行"得到对应向量。

Tokenizer 如何切分：BPE 算法

模型不直接看字符。BPE（Byte Pair Encoding）算法把文字切成 token：

输入	Token 数	中英对比
Hello world	2	1.0× 基线
????	4-6	2-3× 贵
function_calling	3-4	—
?????	5-7	2-3× 贵

中文比英文贵 1.5-3 倍 —— 因为词表训练以英文优先，中文字符往往独占 1-2 个 token。

CODE

```
from anthropic import Anthropic
client = Anthropic()
resp = client.messages.count_tokens(
    model="claude-sonnet-4-6",
    messages=[{"role": "user", "content": "?????"}]
)
print(resp.input_tokens)
```

Embedding：把 token 变成"模型可计算的向量"

模型看到 token ID（一个整数）没法直接思考，要先把它变成**向量**——一串浮点数（如 4096 个数字）。这串数字就是这个 token 的"含义指纹"。

概念	通俗类比
Token ID	"苹果"在词表里的编号（如 12345）
Embedding 向量	把 12345 翻译成 4096 个数字组成的"含义指纹"
维度 d	指纹有多少个数字（GPT-3 是 12288，Llama 3 70B 是 8192）
含义近的 token	向量在高维空间中距离近 —— "猫"与"狗"近，"猫"与"汽车"远

Self-Attention（自注意力）：每个词"问问其他词"

直觉理解

人读句子时，理解某个词会回头看上下文里的相关词。**Self-Attention（自注意力）就是模型版的"回头看"机制。**

CODE

```

[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]

```

每个 token "回头看"的过程是用 Q / K / V 三个向量做的：

角色	简称	含义	通俗类比
Query	Q	"我想知道什么"	提问者带着问题
Key	K	"我能提供什么"	别人胸口的标签
Value	V	"实际信息内容"	别人肚子里的内容

▸ 工作流程（一次 attention）

1. 当前 token 用自己的 **Q** 跟所有其他 token 的 **K** 比对，得到一组"相关性分数"
2. 把这组分数过 softmax 归一化成概率分布（所有数加起来 = 1）
3. 用这个概率分布去加权融合所有 token 的 **V**，得到"在这个语境里，我（当前 token）的新含义"

「结果：每个 token 不再是孤立的"词典义"，而是带上了上下文信息的"语境化含义"。

▸ "Self" 的意思

"Self" 指 Q、K、V 都来自同一个序列（自己看自己）—— 区别于"cross attention"（Q 来自一个序列、K/V 来自另一个）。LLM 用的是 self-attention。

▸ $O(n^2)$ 复杂度怎么来的

序列里 n 个 token，每个 token 都要看其他 n 个 token 一遍 —— 共 $n \times n = n^2$ 次"看"。

序列长 n	总"看"次数	影响
1k token	1M（百万）	快
10k token	100M	慢 10-100×
100k token	10B	极慢
1M token（Opus 4.7）	1T（万亿）	必须用稀疏 attention 优化

这是上下文窗口有上限的根本原因 —— 不是技术做不到，是 $O(n^2)$ 太贵。

▸ 公式骨架（懂线性代数可看）

```
attention(Q, K, V) = softmax(Q · KT / √d) · V
```

[CODE](#)

不懂线性代数没关系，记住三件事：

部分	干啥
$Q \cdot K^T$	算每对 token 的相关性分数（分数矩阵）
$\text{softmax}(\dots)$	把分数归一化成概率分布
$\dots \cdot V$	用概率分布加权融合 V
$/ \sqrt{d}$	数值稳定性技巧（防止分数过大）

▸ 为什么 KV Cache 能优化

关键洞察：**K 和 V 是 token 自己的属性，只取决于这个 token 自己，不取决于后面有谁。**

所以一旦算过某个 token 的 K 和 V，**永远不变** —— 后续新 token 来时，只需查询历史 token 的 K/V（不必重算），只算新 token 自己的 K/V。

详细机制见 §0.3.2。

多头注意力（Multi-Head Attention）：一个 token 多个角度同时看

▸ 单头不够

只用一组 Q/K/V 算 attention 太单调 —— 一个 token 只能从"一个角度"理解上下文。比如读"苹果"时，可能既需要关注它是水果（语义关系），也需要关注它在句子里是宾语（语法关系）。

▸ 多头机制

把 Q/K/V 切成 N 个"头"（典型 $N = 16-128$ ），每组独立做一次 attention，然后把 N 个结果拼起来：

```

[?] [?] [?] [?] [?] Q × K^T → [?] [?] [?] attention [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] Q1 × K1^T → attention_1 → [?] [?] [?]
           Q2 × K2^T → attention_2 → [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
           Q8 × K8^T → attention_8 → [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]

```

CODE

头编号	实际可能学到
Head 1	"动作-宾语"语法关系
Head 2	"代词指向哪个名词"
Head 3	"时间词修饰哪个动作"
Head 4	"数量词修饰哪个名词"
...	...

每个头的"分工"是训练时自动涌现的，不是手工设计。

▸ 工程参数

模型	总维度 d	头数	每头维度
GPT-3 175B	12288	96	128
Llama 3 70B	8192	64	128
Llama 3 8B	4096	32	128

▸ KV Cache 的现代优化（MQA / GQA）

每头都有独立 K/V → KV Cache 也按头数倍增。多头注意力让 KV Cache 占显存暴增。

优化	含义	节省
多查询注意力（Multi-Query Attention, MQA）	多个 Query 头共享 1 组 K/V	KV Cache ÷ 头数
分组查询注意力（Grouped-Query Attention, GQA）	多个 Query 头分组共享 K/V	KV Cache ÷ 组数（折中方案）

模型	用什么
Llama 2（旧）	普通多头
Llama 3 / Mistral / 多数现代模型	GQA （典型 8 组 → KV Cache 减少 8 倍）
一些极端优化模型	MQA（KV Cache 最少但质量略降）

GQA 是 2024 后几乎所有大模型的"省显存"标配。

位置编码（Positional Encoding）：让模型知道 token 顺序

▸ 不加位置编码模型分不清顺序

Self-Attention 本身对顺序不敏感 —— 不管你把 "我打他" 还是 "他打我"，每个 token 看其他 token 的方式一样。这显然不行。

▸ 解决：把"位置信息"也注入进来

每个 token 的 embedding 不只是"含义指纹"，还混入"位置指纹"：

阶段	输入	输出
原始 embedding	token 的语义向量	"这个 token 的含义"
+ 位置编码	+ 位置 0/1/2/... 的位置向量	"这个 token 在第 N 位的含义"

主流 3 种位置编码

方法	思路	代表模型
绝对位置 (Sinusoidal / Learned)	给位置 0/1/2/... 各一个固定向量	原始 Transformer / GPT-2
相对位置 (RPE)	不直接编码位置，而是编码"两个 token 距离"	T5
旋转位置编码 (RoPE, Rotary Position Embedding)	把 Q/K 在每个位置上"旋转"一个特定角度	Llama 3 / Qwen / DeepSeek 等几乎全部 2023+ 模型

RoPE 为什么主流

优势	说明
自然支持长上下文	旋转角度可以外推到训练时未见过的位置
数学优雅	直接作用在 attention 计算里，不增加 token 数
工程友好	不增加显存 / 计算量

长上下文（1M token Opus 4.7）的实现关键之一就是 RoPE 的位置外推优化（如 PI / NTK-aware / YaRN 等技巧）。

Transformer 一层结构：N 层堆叠

一层 = "Multi-Head Attention + FFN" + 残差 + 归一化

每一层 Transformer 的计算流：

▸ 总参数量怎么来的

模型	层数	隐藏维度 d	头数	总参数
GPT-2 small	12	768	12	124M
Llama 3 8B	32	4096	32	8B
Llama 3 70B	80	8192	64	70B
GPT-3 175B	96	12288	96	175B

层数 × 每层参数 (attention + FFN) ≈ 总参数量。

▸ 推理时各层之间是流水线

每个 token 走完 N 层后才输出概率分布。长 prompt + 多层 + 大维度 = 推理慢。

输出层 + 采样：概率分布展开

▸ 输出层 (LM Head)

最后一层把每个 token 的隐藏向量乘以词表矩阵，得到词表大小的"分数向量" (logits)：

```
hidden_vector (d [?]) × W_lm_head (d × vocab_size) → logits (vocab_size [?])
```

[CODE](#)

▸ softmax：分数 → 概率分布

```
logits = [3.2, 1.1, -0.5, 4.7, ...] ← [?][?][?][?][?]token [?][?][?]  
softmax(logits) = [0.18, 0.02, 0.005, 0.79, ...] ← [?][?][?][?][?][?][?][?][?][?][?][?][?][?]
```

[CODE](#)

▸ 采样策略详解

控制下一个 token 怎么从概率分布里挑出来：

参数	数学含义	直觉效果
<code>temperature</code>	logits 除以 T 后再 softmax	T 越小越"确定"、T 越大越"随机"
<code>top_p</code> (核采样)	累积概率到 P 为止的 token 集合内采样	动态截断长尾
<code>top_k</code>	只看概率最高的 K 个 token	静态截断长尾
<code>repetition_penalty</code>	重复 token 的 logits 减分	避免复读
<code>frequency_penalty</code>	出现次数越多惩罚越大	鼓励词汇多样性
<code>presence_penalty</code>	出现过的 token 一律减分	鼓励引入新概念

▸ temperature 的真实数学含义

CODE

```
logits: [3.2, 1.1, -0.5, 4.7]
```

T = 0.5 (核采样) :

```
logits / 0.5 = [6.4, 2.2, -1.0, 9.4] ←
```

```
softmax = [0.05, 0.001, 0.000, 0.949] ← token 4 (token 4)
```

T = 1.0 (核采样) :

```
softmax([3.2, 1.1, -0.5, 4.7]) = [0.18, 0.02, 0.005, 0.79]
```

T = 2.0 (核采样) :

```
logits / 2.0 = [1.6, 0.55, -0.25, 2.35] ←
```

```
softmax = [0.27, 0.10, 0.045, 0.59] ←
```

T → 0: $\text{argmax}(\text{logits})$

T → ∞: 所有 token 概率相等

▸ top_p（核采样）的算法

CODE

```
def top_p_sample(logits, p=0.9):
    probs = softmax(logits)
    sorted_probs, sorted_idx = sort_descending(probs)

    cum_probs = cumsum(sorted_probs)
    cutoff = first_idx_where(cum_probs >= p)
    return sample_from(sorted_idx[:cutoff], sorted_probs[:cutoff])
```

▸ 实战推荐配置

任务	temperature	top_p	备注
工具调用 / 结构化输出	0	1.0	完全确定性
客服 / 问答	0.3-0.7	0.9	适度自然
创意写作	0.8-1.2	0.95	多样性高
代码生成	0-0.3	0.95	倾向确定

关键洞察：模型本质是"统计下一个 token 的概率分布展开" —— 不是"逻辑推理"。

现象	实现原因
幻觉	高概率 ≠ 事实正确
temperature=0 仍可能错	概率分布本身可能错
思维链有效	多生成几个 token 让分布"展开"
推理模型有效	内部生成几千个"思考 token"再给答案

训练 vs 推理：两个完全不同的过程

▸ 训练 (Training, 发生在厂商内部)

阶段	含义	时长 / 成本
预训练 (Pre-training)	用万亿级 token (互联网 + 书 + 代码) 训练模型预测下一 token	数月 / 千万-亿美元 (Llama 3 70B 约 \$30M)
监督微调 (SFT)	用人工标注的"指令-回复"对, 让模型学会按指令做事	几天 / 数十万美元
人类反馈强化学习 (RLHF / DPO)	用人工对回复排序训练奖励模型, 再优化策略	几周 / 数百万美元
能力扩展 (推理 / 工具)	o1 / Claude Extended Thinking 这种"推理模型"在 SFT/RL 阶段加入"思考 token"	数月 / 数百万美元

结果: 得到"模型权重文件" (Llama 3 70B 约 140 GB)。

▸ 推理 (Inference, 发生在你调 API 时)

阶段	含义	时长
加载权重到 GPU	模型权重 + KV Cache 占用显存	启动一次
处理 prompt (prefill)	算整个 prompt 的 forward + KV cache	100-2000ms
生成 token (decode)	每个 token 重复一次 forward + 采样	5-50ms / token

▶ 关键差异

维度	训练	推理
谁做	厂商 (OpenAI / Anthropic / Google)	你 + 厂商 API
计算图	前向 + 反向 (梯度)	仅前向
数据	万亿 token 语料	你的 prompt
输出	模型权重	一段文本
成本量级	千万-亿美元	单次 \$0.001-\$1
可调参数	学习率 / 批大小 / 优化器	temperature / top_p / max_tokens

▶ 为什么这区分重要

场景	影响
你看到的 token 价格	仅推理成本，不含训练分摊
模型"知识截止"日期	训练数据截止时间
模型"擅长什么"	由训练数据 + RLHF 决定，推理时无法改
想让模型"学习新东西"	不能改权重；只能在 prompt 里给少样本 (in-context learning) 或微调

调 API 时你只用推理。所有"知识"都在权重里，权重在训练阶段就冻结了。

LLM 的能力与边界

能力	工程含义
理解自然语言	节点 ① 不必结构化输入
生成自然语言	节点 ⑧ 直接给用户读
上下文学习	prompt 里给少样本示例就学会
工具调用（function calling）	节点 ④ 输出结构化 JSON
推理（reasoning）	节点 ② 多步逻辑

局限	后果	谁来补
没有长期记忆	重启对话就忘	§0.3 记忆层
不能行动	只会"说"	§0.2 工具调用
不能验证事实	幻觉	RAG / 多步验证
上下文窗口有限	太长就爆	§0.3.2 截断 / 摘要
不擅精确计算	1234×5678 易错	调计算器工具
训练数据有截止	不知最近的事	实时检索工具

记住这张表 —— 所有智能体框架做的事，都是在补 LLM 这些洞。

小结一行：LLM 是 4 步流水线（Token / Embedding / Attention / Sample），本质是概率分布展开 —— 它的 6 大局限定义了智能体框架的所有职责。

一次 API 调用的 7 步流水线

4.

关键洞察：首 token 慢（100-2000ms 走完整 prompt），后续 token 快（5-50ms 命中 KV Cache）。**TTFT (Time To First Token)** 才是用户体验，不是 total latency。

实测吞吐量（2026.04）

模型	输出速度
Claude Sonnet 4.6	60-100 token/s
Claude Opus 4.7	25-50 token/s
GPT-4o	80-120 token/s
GPT-4o-mini	150-250 token/s
Llama 3.3 70B 自部署	30-80 token/s

工具调用协议：结构化解码

模型能输出结构化 JSON 调工具，不是魔法，是 **grammar-constrained decoding**：

步	操作
1	API 收到 <code>tools</code> 参数后注入工具 schema 到 prompt
2	模型生成时，特殊 token（如 <code><tool_use></code> ）触发"工具模式"
3	后续 token 受 schema 约束 —— 只能输出符合 schema 的 token
4	API 把这段结构化内容包装在 <code>tool_calls</code> / <code>tool_use</code> 字段返回

三家协议字节级对比

维度	OpenAI Function Calling	Anthropic Tool Use	Google Function Declarations
工具描述字段	<code>tools[].function.parameters</code>	<code>tools[].input_schema</code>	<code>function_declarations[].parameters</code>
输出形式	<code>tool_calls[]</code> 数组	<code>content</code> 内 <code>tool_use</code> 块	<code>parts[].functionCall</code>
参数格式	<code>arguments</code> JSON 字符串	<code>input</code> 字典	<code>args</code> 字典
ID 字段	<code>id</code> (<code>call_xxx</code>)	<code>id</code> (<code>toolu_xxx</code>)	无显式 ID
并行调用	是	是	部分模型

Anthropic 字节级响应：

CODE

```

[?]
  "stop_reason": "tool_use",
  "content": [
    {"type": "text", "text": "[?][?][?][?][?][?]},
    {"type": "tool_use", "id": "toolu_01abc",
      "name": "get_weather", "input": {"city": "[?][?][?]"}
  ]
[?][?][?]
[?]

```

回填给模型：

CODE

```

{"role": "user", "content": [
  {"type": "tool_result",
    "tool_use_id": "toolu_01abc",
    "content": "[?][?][?][?][?][?][?][?]"
  ]
}]

```

详细字节对照见 [附录 K](#)。

结构化解码内部机制（grammar-constrained decoding 怎么工作）

▸ 朴素做法 vs 约束解码

朴素做法（GPT-3 时代）：在 prompt 里写“请输出 JSON 格式”，靠模型听话。**失败率 5-30%**（模型经常输出错的 JSON、加多余文字、漏字段）。

结构化解码（GPT-4 / Claude 现代做法）：在采样阶段**强约束**模型只能输出符合 schema 的 token。**失败率 < 0.1%**。

▸ 约束解码的字节级工作原理

把 schema 编译成“状态机 + 允许 token 集”，每生成一个 token 时：

```
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] city":  
[?] [?] [?] [?]  
[?] [?] schema [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]  
[?] [?] [?] [?]  
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] token, [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] token?  
[?] [?] [?] [?]  
[?] [?] [?] [?] logits (50,000 [?])  
[?] [?] [?] [?]  
[?] [?] [?] [?] token [?] [?] logits [?] [?] [?] [?]  
[?] [?] [?] [?]  
softmax → [?] [?] [?] [?] token [?] [?] [?]  
[?] [?] [?] [?]  
[?] [?] [?] [?] token ([?] [?] [?] [?] [?] [?] [?] [?] [?] [?])
```

实现思路

阶段	操作
1. Schema → 状态机	把 JSON Schema 编译成有限状态自动机 (FSA)
2. 当前状态 → 允许字符集	状态机告诉你当前能接受哪些字符
3. 字符集 → 允许 token 集	用 tokenizer 反推哪些 token 可以"prefix-match"上
4. logits 屏蔽	不允许的 token logits = $-\infty$
5. 重新归一化 + 采样	softmax + 采样
6. 状态机推进	根据生成的 token 更新状态

▶ 主流约束解码引擎

引擎	出品方	强项
Outlines（开源）	Normal Computing	Pythonic, 与 vLLM 集成
JSON Mode（OpenAI 2024.06）	OpenAI 内置	API 透明
Structured Outputs（OpenAI 2024.08）	OpenAI 内置	完整 JSON Schema
lm-format-enforcer	开源	字段级约束
xgrammar	开源	高性能（支持 LLVM 风格语法）
vLLM guided decoding	vLLM	自部署首选

▶ 性能开销

操作	额外成本
状态机查询	< 1ms / token（缓存后）
token 集合预计算	启动一次性（首次 ~100ms）
logits 屏蔽	几乎为零（GPU 上 mask 操作）
总开销	生成速度下降 < 5%

▸ 工具调用 = 特殊 schema 的约束解码

CODE

```

[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] schema
schema = {
  "type": "object",
  "properties": {
    "name": {"type": "string", "enum": ["get_weather", "search"]},
    "arguments": {"type": "object", "properties": {...}}
  }
}

[?] [?] [?] [?]
  "required": ["name", "arguments"]
}

[?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
constrained_decode(model, prompt, schema)

```

这就是 OpenAI Function Calling / Anthropic Tool Use 不会输出格式错乱的根本原因 —— 不是"prompt 哄好了模型", 是采样层强制约束。

▸ Anthropic Tool Use 的额外妙处

Claude 的 Tool Use 在训练阶段还把"工具调用 vs 文本回复"作为训练目标, 让模型主动学会何时调用工具, 约束解码只是兜底。这就是为什么 Claude 的工具调用准确率 (选对工具 + 填对参数) 业界最高。

流式输出: SSE 字节流

Server-Sent Events 是 HTTP 长连接 + 行级事件:

CODE

```

HTTP/1.1 200 OK
Content-Type: text/event-stream

event: content_block_delta
data: {"type": "content_block_delta", "delta": {"type": "text_delta", "text": "[?][?]}}

event: content_block_delta
data: {"type": "content_block_delta", "delta": {"type": "text_delta", "text": "[?][?]}}

event: message_stop
data: {"type": "message_stop"}

```

工具调用 + 流式的边界冲突：文本可 token 级吐字符，但工具调用块**必须收齐整个 JSON 才能用**：

CODE

小结一行：API 调用是 7 步流水线，工具调用靠结构化解码，三家协议字节级有差异 —— 跨厂商代码必须用 LangChain `bind tools` 这类适配层。

§0.3 记忆层：完整存储架构

基础流程对应：节点 ⑥ 状态管理的全套实现。本节是全章重点。

§0.3.1 三层记忆模型总览

智能体的记忆按"时间尺度 + 持久化层级"分三层:

三层之间的协作关系

第 36 页

§0.3.2 短期记忆：Context Window + KV Cache + Prompt Caching

▸ Context Window 的工程现实

每次 API 调用塞进去的所有内容总和。容量看起来大但跑几轮就满：

模型	上下文窗口	约等于
Claude Opus 4.7 (1M)	1,000,000 token	75 万英文字 / 50 万中文字
Claude Sonnet 4.6	200,000 token	15 万英文字
GPT-4o	128,000 token	9.6 万英文字
Gemini 2.5 Pro	2,000,000 token	150 万英文字

▸ KV Cache：续写为什么便宜

» 核心洞察

回到 §0.1：每个 token 都有 K ("我能提供什么") 和 V ("实际信息") —— **这两者只取决于 token 自己，不取决于后面有谁。**一旦算过，永远不变。

所以可以缓存：把每个 token 的 K、V 算出来后存到 GPU 显存里。下次生成下一个 token 时，**只算新 token 的 K/V + 让它的 Q 去查询所有历史 K/V**，不必重算历史。

» 不缓存 vs 缓存的对比

维度	不带 KV Cache	带 KV Cache
第 1 个 token (首次)	算整个 prompt 的 attention	同 (首次必须算)
第 2 个 token	重算整个序列 (重复劳动)	只算新 token, 历史读 cache
第 N 个 token	重算 N-1 次序列	仅算 1 次
单 token 复杂度	$O(n^2)$	$O(n)$
累计 N 个 token	$O(n^3)$	$O(n^2)$

» 时序示意

CODE

```

???prompt = "?????"
Token 0 "?????K0, V0 → ??
Token 1 "?????K1, V1 → ???attend K0, V0?
Token 2 "?????K2, V2 → ???attend K0..K1?
Token 3 "?????K3, V3 → ???attend K0..K2?
Token 4 "?????K4, V4 → ???attend K0..K3?

?????
Token 5 "?????K5, V5 → ??
          attend K0..K4 ???cache (???)

?????
Token 6 "!" → ??K6, V6 → ??
          attend K0..K5 ???cache

```

每个 token 只算一次 K/V, 永远缓存。这是为什么 LLM 能"流式打字"般快速生成 —— 后续 token 几乎是免费的。

>> 工程后果

现象	解释
首次调用慢（100-2000ms TTFT）	必须把整个 prompt 的 K/V 全算并缓存
后续 token 快（5-50ms/token）	只算新 token + 读历史 cache
续写（多轮对话）便宜	历史 token 的 K/V 已缓存，可复用
GPU 显存吃紧	KV Cache 占大量显存，长序列尤其

「直觉总结：KV Cache = "每个 token 的属性只算一次、永远缓存"。这就是后续 token 能秒出的根本原因。

► Prompt Caching：把 KV Cache 持久化

Anthropic 2024.08 推出 —— 把"同 prefix 的 KV Cache"跨调用持久化：

CODE

```
client.messages.create(  
    model="claude-sonnet-4-6",  
    system=[  
        {"type": "text",  
         "text": "<10k token [?][?][?][?][?]>",  
         "cache_control": {"type": "ephemeral"}    # ★ [?][?][?][?]  
    ],  
    messages=[{"role": "user", "content": "[?][?][?]}]  
[?]
```

字节级证据（usage 字段）：

CODE

```
[?  
    "input_tokens": 50,  
    "cache_creation_input_tokens": 0,  
    "cache_read_input_tokens": 10000,  
    "output_tokens": 200  
]
```

▸ 价格折扣对比

厂商	缓存命中部分价格
Anthropic	10% 普通价 （省 90%）
OpenAI	50% 普通价
Google	50% 普通价

▸ 工程后果

现象	解释
系统提示 + 工具描述固定时极受益	prefix 不变，全部命中
改一个字 → 后续全失效	cache 是 prefix-based
单次调用不省	首次还要付全价创建缓存
多轮长对话省最多	历史不变只追加新消息

小结一行：短期记忆 = Context（数据）+ KV Cache（计算优化）+ Prompt Caching（持久化优化），稳定 prefix 前置 + cache_control 是降本的关键。

§0.3.3 中期记忆：会话历史的工程实现

▸ **LangChain Memory 4 类（会话级，详见 §2.7）**

类型	工程含义	适用
ConversationBufferMemory	完整存所有历史	< 10 轮短对话
ConversationSummaryMemory	用 LLM 总结历史	10-50 轮不需细节
ConversationKGMemory	抽取实体+关系成知识图	实体关系密集
VectorStoreRetrieverMemory	历史片段存向量库	长对话需细节

四类的共同缺陷：**会话级 + 进程内** —— 跨进程或跨会话失忆。

▸ **LangGraph Checkpoint：跨进程持久化**

LangGraph 把 State 写到外部存储，让智能体跨进程、跨会话续跑。

CODE

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.postgres import PostgresSaver

# 创建 Saver 对象
checkpointer = MemorySaver() # 内存
checkpointer = SqliteSaver.from_conn_string(...) # SQLite
checkpointer = PostgresSaver.from_conn_string(...) # PostgreSQL

agent = create_react_agent(llm, tools, checkpointer=checkpointer)

# 线程 ID 配置
config = {"configurable": {"thread_id": "user-123"}}
agent.invoke({"messages": [...]} , config) # 开始运行
```

▸ **Postgres 表结构**

PostgresSaver.setup() 创建 4 张表：

表	内容
checkpoints	State 快照
checkpoint_writes	节点写入记录
checkpoint_blobs	大字段二进制
checkpoint_migrations	版本管理

▶ 写入时序

CODE

```
T+0      [?][?][?][?]invoke(input, thread_id=X)
T+1ms    Checkpointer.load(thread_id=X) → [?][?][?]State
[?][?][?][?][?][?][?][?][?][?][?][?]Node A [?]
T+50ms   Node A [?][?][?][?]
T+51ms   Checkpointer.save(checkpoint_1) ← [?][?][?][?]
[?][?][?][?][?][?][?][?][?][?][?][?]Node B [?]
[?][?][?]
```

每个超步屏障写一次 checkpoint。

▶ 中期记忆的范围

thread_id	范围
同 thread_id 的多次 invoke	自动续上历史
不同 thread_id	完全隔离
命名建议	user-{user_id} / session-{uuid}

小结一行：中期记忆的演化路径是 LangChain Memory（会话级）→ LangGraph Checkpoint（跨进程持久化）—— 后者是生产级标配。

§0.3.4 长期记忆：Vector DB + Store API + Skills

长期记忆的三种工程实现

实现	适合	起源
Vector Database	语义召回 / RAG	2022+
LangGraph Store API	跨 thread 结构化记忆	2024.10
Anthropic Skills + Memory	能力打包 + 自我成长	2025

Vector DB 工作原理

CODE

1. 输入文本 (query) 通过 embedding 模型生成 embedding 向量
 2. 在 Vector DB 中搜索与 query 向量最相似的 embedding 向量 (cosine similarity)
 3. 返回 top-k 个最相似的 embedding 向量
 4. 使用这些 embedding 向量生成 prompt (context)

Embedding 是什么 (回顾 §0.1)

把一段文本（一句话 / 一段话 / 一篇文档）通过专门的"embedding 模型"（如 OpenAI `text-embedding-3-small` / Anthropic `voyage-3` / 开源 `bge-large`）变成一串浮点数向量（如 1536 个数字）。

概念	通俗类比
文本 → embedding	给整句话画一个"含义指纹"（与 §0.1 单 token embedding 不同，是整段的）
维度 d	一般 768 / 1024 / 1536 / 3072
含义近的文本	向量在高维空间里距离近

► 余弦相似度（Cosine Similarity）：怎么衡量"两个向量不相关"

直觉：把每个向量看成 d 维空间里的一支箭头。两支箭头方向越接近、相关性越高。余弦相似度算的就是这两支箭头之间的"夹角余弦值"：

夹角	cos 值	含义
0°（完全同向）	1.0	完全相同
60°	0.5	中等相关
90°（垂直）	0.0	不相关
180°（反向）	-1.0	反义

公式（看不懂没关系）：

CODE

cosine(A, B) = (A · B) / (|A| × |B|)

???

A · B = a1×b1 + a2×b2 + ... + ad×bd (????????????)

|A| = sqrt(a1² + a2² + ... + ad²) (A ???)

|B| = sqrt(b1² + b2² + ... + bd²) (B ???)

只看方向不看长度 —— 这就是为什么用余弦而非欧氏距离：embedding 的"长度"可能反映文档篇幅而非语义。

► 为什么不用线性扫描

朴素做法：query 向量跟库里**每一个**向量都算 cosine。10 万条向量就要 10 万次乘加。100 万条就要 100 万次。**线性 O(n) 太慢。**

- 索引算法把 $O(n)$ 降到 $O(\log n)$

算法	原理（一句话）	召回率	速度
暴力 (Flat)	每个都比	100%	慢
HNSW	建多层"图"网络，查询时从顶层粗找 → 底层细找（类似跳表）	95-99%	快
IVF	先聚成 k 个簇，查询时只在最近的几个簇里找	90-95%	中
DiskANN	HNSW 思路 + 磁盘存储，适合超大规模	95-99%	中（磁盘 IO）

▶ 实战代码

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embedding = OpenAIEmbeddings()

embeddings = OpenAIEmbeddings(model="text-embedding-3-small") # 1536

docs = [
    "The quick brown fox jumps over the lazy dog. This is a sentence with 28 characters, including spaces and punctuation. It is a pangram, meaning it contains every letter of the alphabet at least once. The sentence is often used as a test for fonts and typewriters. It is a classic example of a pangram in English. The sentence is often used as a test for fonts and typewriters. It is a classic example of a pangram in English."
]

db = FAISS.from_texts(docs, embeddings)

query = "The quick brown fox jumps over the lazy dog."
results = db.similarity_search(query, k=3)

cosine_similarity(results[0].page_content, query) # 0.9999999999999999
```

▸ 主流向量库

工具	类型	适合
Pinecone	SaaS (Serverless / Standard)	中小规模
Qdrant	开源 + Cloud	中等规模
Weaviate	开源 + Cloud	复杂查询
Chroma	嵌入式 / 单机	开发
pgvector	Postgres 扩展	已有 Postgres 时首选
FAISS	库 (Meta 开源)	单进程嵌入式

▸ Embedding 模型选型（决定召回质量的根因）

向量检索的召回率上限完全由 **embedding 模型** 决定。模型越强、向量越能"理解"语义。

» 主流模型对比 (2026.04)

模型	维度	MTEB 分数	价格	强项
OpenAI text-embedding-3-large	3072 (可降至 256-1024)	64.6	\$0.13/M token	通用、英文强
OpenAI text-embedding-3-small	1536	62.3	\$0.02/M	性价比高
Cohere embed-v3	1024	64.5	\$0.10/M	多语言强
Voyage-3 (Anthropic 推荐)	1024	66.1	\$0.18/M	长文档强
bge-large-en-v1.5 (开源)	1024	64.2	0 (自部署)	英文
bge-m3 (开源, 多语言)	1024	60.2	0 (自部署)	中英混合首选
multilingual-e5-large (开源)	1024	60.0	0	多语言
nomic-embed-text	768	62.3	0	轻量、本地

「 MTEB (Massive Text Embedding Benchmark) = 业界主流评估, 56 个任务平均分数。

» 选型决策树

场景	推荐
英文通用	<code>text-embedding-3-small</code> （性价比）
长文档（金融/法律）	<code>voyage-3</code> 或 <code>voyage-3-large</code>
中英混合	<code>bge-m3</code> 自部署 或 <code>voyage-3</code>
隐私 / 离线	<code>bge-m3</code> 或 <code>nomic-embed</code>
大规模（> 1 亿条）	自部署开源模型省钱

» Matryoshka Embeddings（俄罗斯套娃嵌入）

OpenAI text-embedding-3 系列支持**降维而几乎不损失精度** —— 训练时把不同维度的子向量都优化得有意义：

CODE

```
from openai import OpenAI
client = OpenAI()

# 3072 维完整向量
emb_full = client.embed("The quick brown fox jumps over the lazy dog", model="text-embedding-3-large")

# 256 维紧凑向量
emb_compact = emb_full[:256]
```

工程价值：高精度场景用 3072 维存底库，低延迟场景用 256 维快查 —— 一份模型支持多档需求。

混合检索 (Hybrid Retrieval) ：dense + sparse 结合

» 单纯向量检索的局限

问题	说明
不擅长"精确匹配"	查"OAuth 2.0"可能召回"OAuth 1.0"和"SAML 2.0"（语义近但不准）
不擅长"罕见术语"	公司内部代号 / 专业缩写 模型没见过
不擅长"数字 / 名字"	"GPT-4o" 和 "GPT-3.5" 向量很近

» 关键字检索 (Sparse / BM25) 的强项

BM25 (Best Match 25) 是经典的关键字检索算法，强在精确匹配（如倒排索引）。

维度	Dense 向量 (语义)	Sparse 关键字 (BM25)
强项	同义词、近义概念	精确词汇、专有名词
弱项	罕见术语、缩写	同义词、改写
召回机制	余弦相似	词频 + 逆文档频率
速度	$O(\log n)$ HNSW	$O(\log n)$ 倒排索引

混合检索的标准做法

CODE

» Reciprocal Rank Fusion (RRF) 算法

CODE

» Reranker（重排序）模型

进一步提升精度的“压舱石”。

Reranker	出品方	强项
Cohere Rerank v3	Cohere	跨语言精度高、API 简单
bge-reranker-v2-m3（开源）	BAAI	中文强
Voyage rerank-v3	Anthropic	通用
Jina Reranker v2	Jina	多语言

Reranker 跟 embedding 不同 —— 它直接读 **query + doc** 一对判分，不是预先编码再算相似度。精度高但速度慢，适合 **top 30** → **top 5** 的精排。

» 召回率提升幅度（实测）

方案	召回率（top 5）
纯 dense	65%
纯 BM25	55%
Dense + BM25 + RRF	78%
Dense + BM25 + RRF + Reranker	88%

► LangGraph Store API（详见 §6.5）

CODE

```

from langgraph.store.memory import InMemoryStore
from langgraph.store.postgres import PostgresStore

store = InMemoryStore() # [?]
# store = PostgresStore(conn_string) # [?]

# namespace [?][?][?][?]
namespace = ("user_memories", "user-123")

[?][?][?]
store.put(namespace, key="diet", value={"preference": "vegetarian"})
store.put(namespace, key="lang", value={"primary": "[?][?][?]})

[?][?][?]
mem = store.get(namespace, "diet")
print(mem.value) # {"preference": "vegetarian"}

[?][?][?][?]namespace [?][?]
all_mems = store.search(namespace)

```

► 加 embedding 索引：语义召回

CODE

```
from langchain_openai import OpenAIEmbeddings

store = InMemoryStore(
    index={"embed": OpenAIEmbeddings(), "dims": 1536}
)

store.put(namespace, "m1", {"text": "?????????"})
store.put(namespace, "m2", {"text": "?????????"})

results = store.search(namespace, query="?????")
# m2 (???) ???
```

► 在节点函数里使用 Store

CODE

```
def call_model(state, *, store):
    user_id = state["user_id"]
    namespace = ("user_memories", user_id)

    mems = store.search(namespace, query=state["messages"][-1].content)
    context = "\n".join(m.value["text"] for m in mems)

    prompt = state["messages"][-1].content
    enriched = [
        SystemMessage(f"???context?"),
        *state["messages"]
    ]

    response = llm.invoke(enriched)

    if "???" in state["messages"][-1].content:
        store.put(namespace, str(uuid.uuid4()), {"text": "..."})

    return {"messages": [response]}
```

► Anthropic Skills (2025 新模式)

把"能力"打包成 Skill —— 含 prompt + tools + 示例 + memory :

维度	Tool（工具）	Skill（技能）
粒度	单个函数	一组工具 + prompt + 示例
持久化	无	含 memory 字段
复用	跨智能体	跨智能体 + 跨用户
配置	代码	配置 / 数据库

详见 §16.3。

▸ 三种长期记忆的对比

维度	Vector DB	Store API	Skills
数据形态	文本片段	结构化 key-value	能力包
召回方式	语义相似	namespace 精确 / 语义	按需加载
适合	文档 / 知识	用户偏好 / 状态	可复用能力
成熟度	高（2022+）	中（2024+）	早期（2025+）

小结一行：长期记忆三种实现互补 —— Vector DB 做文档检索、Store API 做结构化记忆、Skills 做能力封装。

§0.3.5 状态持久化与时间旅行

▸ **get_state_history : 看历史快照**

CODE

```
config = {"configurable": {"thread_id": "user-123"}}

for state in agent.get_state_history(config):
    print(f"state.metadata['step'] state.next")
    print(f" state: {state.values}")
    print(f" checkpoint_id: {state.config['configurable']['checkpoint_id']}")
```

▸ **分支重放 (Branching)**

从历史某 checkpoint 改 state 后继续跑 —— 创出"平行宇宙"分支:

CODE

```
checkpoint
target = list(agent.get_state_history(config))[3]

config
new_config = target.config

state
agent.update_state(new_config, {"messages": [HumanMessage("")]})
result = agent.invoke(None, config=new_config)
```

▸ **工程价值**

场景	价值
调试：回到错的那步看 state	调试效率指数级提升
A/B：从同一中间状态分两支	比较不同后续策略
用户"撤回"：从历史某点重新选	客服必备
审计：完整还原任务执行链	合规 / 事后分析

小结一行：时间旅行让 §0.3 的"中期记忆"从"现在"扩展到"历史 + 分支"，是 LangGraph 区别于 LangChain 的杀手级能力。

§0.3.6 多租户隔离的命名空间

生产环境必须按 workspace / 租户隔离记忆，避免数据泄漏。

▸ 三层隔离

隔离层	字段
LangGraph thread	<code>thread_id</code> 唯一
LangGraph 命名空间	<code>checkpoint_ns</code> （多任务时区分）
Store namespace	<code>("ws_id", "user_id", ...)</code> 元组

CODE

```
config = {"configurable": {
    "thread_id": session_id,
    "checkpoint_ns": workspace_id,    # ★ [?][?][?][?]
    "user_id": user_id,
}}

# Store [?][?][?][?]workspace
namespace = ("user_memories", workspace_id, user_id)
```

► Postgres 分区

CODE

```
CREATE TABLE checkpoints (
  thread_id TEXT,
  checkpoint_ns TEXT,
  checkpoint_id TEXT,
) PARTITION BY HASH (checkpoint_ns);

CREATE TABLE checkpoints_p0 PARTITION OF checkpoints
FOR VALUES WITH (modulus 16, remainder 0);
```

► TTL 与清理

CODE

```
DELETE FROM checkpoints WHERE created_at < NOW() - INTERVAL '30 days';
```

► 大字段外挂

错	对
PDF 全文塞 state	存 S3, state 存 URL
图片 base64 塞 messages	用图片 URL (厂商支持)
整个数据库结果塞 state	存 ID 列表, 按需查

小结一行：多租户用 `checkpoint_ns` + Postgres 分区 + Store namespace 三层隔离，大字段必须外挂 S3。

§0.3.7 上下文工程：组装与截断

▸ 完整上下文构成（Anthropic API 实例）

CODE

```

client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=4096,

    ???
    system=[{"type": "text", "text": "...",
              "cache_control": {"type": "ephemeral"}}],

    ???
    tools=[{"name": "...", "description": "...", "input_schema": {...}}],

    ???
    messages=[
        ???
        {"role": "user", "content": "..."},
        {"role": "assistant", "content": [...]},
        ???
        {"role": "user", "content": [
            {"type": "text", "text": "<RAG ???>"},
            {"type": "text", "text": "<RAG ???>"},
            {"type": "text", "text": "???"}
        ]}
    ???
    ???
    ?

```

▶ Token 估算

部分	典型 token 数
系统提示	500-2000
工具描述 (10 个)	2000-5000
历史 (10 轮 + 工具调用)	5000-15000
RAG 片段 (4 chunk)	2000-8000
当前问题	50-200
总计	10-30k token / 单次

▶ 4 种截断策略

策略	实现	适合
FIFO 滑动	保留最近 N 条	短对话
摘要替代	LLM 总结老历史	中等对话
RAG 替代	历史存向量库按相关性召回	长对话需细节
分层 (recent + summary + RAG)	三种组合	严肃生产

▶ Prompt Caching 命中条件

「命中条件：从 prompt 起始位置开始的"连续相同 prefix"」

改动	后果
系统提示完全相同 + 工具完全相同	✓ 命中前面所有
系统提示中改一个字	✗ 全部失效
工具顺序变了	✗ 后续失效

工程建议：稳定不变的内容前置（系统 + 工具），变动的放最后（当前问题 + RAG）。

小结一行：上下文工程的核心是"稳定前置 + cache_control + 智能截断"，token 预算和 cache 命中是降本的两大杠杆。

§0.3.8 存储架构与成本（生产级深化）

▸ 三层记忆的存储成本估算

按 10 万 DAU × 平均 10 会话/天 × 平均 10 轮/会话 估算（中型 SaaS 智能体）：

层	数据量	月度增量	月度存储成本
短期 (KV Cache)	仅在 GPU 内存	0 (请求结束销毁)	0
中期 (Postgres Checkpoint)	单 checkpoint ~25 KB × 10 轮 = 250 KB/会话 × 100 万会话/月	~250 GB/月	~\$15-50 (AWS RDS)
长期 (Vector DB + Store)	单条记忆 ~2-5 KB × 100 万条 / 月	~3-5 GB/月	~\$50-200 (Pinecone / 自托管)
大字段外挂 S3	PDF / 截图 / 大文档	取决业务	~\$0.023/GB/月

关键洞察：Checkpoint 比 Vector DB 体积大 **50-100×**（因为存的是结构化 state，不是稀疏 embedding）。没有 **TTL 清理**就是定时炸弹。

► Postgres Checkpoint 索引策略

LangGraph `PostgresSaver.setup()` 默认建：

```
CREATE INDEX checkpoints_thread_id_idx ON checkpoints(thread_id);
CREATE INDEX checkpoints_thread_id_step_idx ON checkpoints(thread_id, step);
```

CODE

生产环境推荐补加

CODE

```
CREATE INDEX checkpoints_thread_id_created_at_idx
ON checkpoints(thread_id, created_at DESC);

ALTER TABLE checkpoints
PARTITION BY HASH (checkpoint_ns);

CREATE TABLE checkpoints_p0 PARTITION OF checkpoints
FOR VALUES WITH (modulus 16, remainder 0);

-- TTL cron job
DELETE FROM checkpoints
WHERE created_at < NOW() - INTERVAL '30 days';
```

Vector DB 价格对比（2026.04 / 1M 条 1536-d embedding）

厂商	月费	适合
Pinecone Serverless	~\$70（按量计费）	中小规模
Pinecone Standard	~\$140（保留实例）	稳定流量
Qdrant Cloud	~\$50-90	中等规模
Weaviate Cloud	~\$80-130	复杂查询
pgvector（自托管 Postgres）	RDS 实例费 ~\$30-100	已有 Postgres
Chroma（自托管）	服务器成本	开发 / 单机
FAISS（嵌入式）	0	单进程

自部署 vs 云服务的盈亏平衡点：

数据量	推荐
< 10M 条 embedding	pgvector 自托管 （最便宜）
10M - 100M	Qdrant Cloud / Pinecone Serverless
> 100M	Pinecone Standard / 自部署 Qdrant 集群

▶ 向量索引算法对比

算法	召回率	速度	内存	适合
暴力 (Flat)	100%	慢	低	< 10k 条
HNSW	95-99%	快	高	生产首选 (< 100M 条)
IVF	90-95%	中	中	大规模 (> 100M)
DiskANN	95-99%	中	极低 (磁盘)	超大规模 (10B+ 条)

Hot / Warm / Cold 三层存储

CODE

► 实现：自动归档 cron

CODE

▸ 大字段外挂 S3 的容量优化

不外挂（错）	外挂 S3（对）
state 含 PDF 文本（500 KB/checkpoint）	state 仅含 S3 URL（200 字节）
100 万 checkpoint = 500 GB	100 万 checkpoint = 200 MB
Postgres 性能下降	Postgres 保持快
Postgres 月费 \$200+	Postgres \$20 + S3 \$12

CODE

```
def my_node(state):
    if len(pdf_text) > 100_000:
        s3_key = f"pdf/{uuid.uuid4()}"
        s3.put(s3_key, pdf_text)
        return {"docs": [{"s3_key": s3_key, "type": "pdf"}]}
    return {"docs": [{"text": pdf_text, "type": "pdf"}]}

def use_doc(state):
    for doc in state["docs"]:
        text = s3.get(doc["s3_key"]) if "s3_key" in doc else doc["text"]
```

小结一行：存储成本三大杠杆 —— Postgres 索引 + 分区 + TTL 控制中期、向量库选型 + 三层冷热分层控制长期、大字段外挂 S3 控制 Postgres 体积。

§0.3.9 备份与灾备

▸ Checkpoint 灾备方案

方案	RPO（数据丢失窗口）	RTO（恢复时间）	成本
PITR（Point-in-Time Recovery）	< 5 分钟	30-60 分钟	RDS PITR 自带
跨地域只读副本	< 1 秒	切换 1 分钟	+50-100% 实例费
每日全量备份到 S3	< 24 小时	1-2 小时	~\$5/月
CDC（Change Data Capture）→ 二级库	< 1 秒	切换 1-5 分钟	Debezium / pgoutput

▸ 数据库迁移：把 Checkpoint 从 SQLite 升级到 Postgres

CODE

```
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.checkpoint.postgres import PostgresSaver

###
old = SqliteSaver.from_conn_string("checkpoints.db")
###
new = PostgresSaver.from_conn_string("postgres://...")
new.setup()

####
for cp in old.list(config=None):
    new.put(cp.config, cp.checkpoint, cp.metadata, new_versions={})
```


agent v2 [?][?][?][?][?][?]

思维链（CoT）的实现机理

关键洞察：不是"模型在推理"，而是"token 序列展开 + KV cache 跨步复用"。

角度	含义
推理时计算	多生成几十 token = 多算几次 attention = 思考"更深"
KV cache 复用	中间思考 token 进入 cache，后续生成 attend 它们
不需训练	任何 LLM 都能用，零学习成本

实证（GSM8K 数学题）：

提示方式	准确率（GPT-3）
不让 CoT，直接给答案	17%
"think step by step"，再给答案	79%

CoT 让模型把中间计算 token 化、KV cache 化，最终答案 attend 这些中间步骤。

ReAct = CoT + 工具调用

范式	思考缓冲	外部脑外存
纯 prompt	无	无
CoT	Thought tokens	无
ReAct	Thought tokens	Tool returns

ReAct 关键创新：**Observation 文本进入 KV cache**，相当于"把工具结果注入模型的思考"。

详细 ReAct 范式见 §1。

推理模型的内化（4 代）

维度	普通 LLM + CoT	推理模型（o1 / Extended Thinking / R1）
思考 token 位置	用户可见的 prompt 内	用户看不见的 hidden thinking
思考长度	几十~几百 token	几千~几万
用户体验	看到"想"完再说	等很久（5-30s）才出最终答案
价格	普通	5-10× 普通价（推理 token 也算钱）

对 ReAct 循环的影响

任务	普通 + ReAct	推理模型
"查 A 加 B 的天气差"	3 步	可能 1 步（内部思考很深）
"规划一周旅行"	10+ 步	1-2 步给完整计划

推理模型是怎么训练出来的（RL + Verifier）

普通 LLM 的训练

回顾 §0.1 训练 vs 推理：普通 LLM 走 预训练 → SFT → RLHF。RLHF 的奖励信号是人类对回复的偏好排序（"答案 A 比答案 B 好"）。

推理模型的训练增加了什么

OpenAI o1 / DeepSeek-R1 / Anthropic Extended Thinking 在 RLHF 之外，加了一个新阶段：

CODE

普通 LLM：预训练 → SFT → RLHF (人类偏好排序)
推理模型：预训练 → SFT → RLHF → ★ RL with Verifier (自动验证)

核心创新：奖励不再来自人类偏好，而是来自自动可验证的信号：

任务类型	自动验证信号
数学题	答案对不对（精确数字匹配）
代码题	单元测试是否通过
形式逻辑	推理步骤合不合规则
结构化任务	输出是否符合 schema

» 训练流程 (DeepSeek-R1 公开论文为例)

CODE

? ? ? ? ? ? ? ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? ? ? ? ? ? x + 5 = 11"
 ? ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? ? ? ? ? ? ? N ? ? ? ? ? ? ? ? ? ? ? ?
 Sample 1: <think>x = (11-5)/2 = 3</think> ? ? ? ? ? ? ?
 Sample 2: <think>x = 11/2 - 5 = 0.5</think> ? ? ? ? ? ? ? ?
 Sample 3: <think>x = 11 - 5 = 6, x = 6/2 = 3</think> ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? Verifier ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
 Sample 1: +1 (? ?)
 Sample 2: -1 (? ?)
 Sample 3: +1 (? ? ? ? ? ? ? ?)
 ? ? ? ? ? ? ? ? ? ?
 ? ? ? ? ? ? ? ? ? ? ? ? ? PPO / GRPO ? ? ?
 ?
 ? ? ? ? ? ? ? ? ? ?
 ?

» 涌现的能力

DeepSeek-R1 训练时没有显式教模型“该想多久”，但训练后模型自发学会：

涌现行为	含义
"Wait, let me reconsider..."	自我反思（重新审视前面的步骤）
"Let me try a different approach..."	重新规划（换思路）
"Let me verify this answer..."	自我验证（算完再核对）
思考长度自适应	简单题想几百 token，难题想几万 token

这是一个**模型涌现现象** —— 不是工程师写规则，而是 RL 训练逼出来的。

» "Hidden Thinking" 是什么

CODE

```
[[{"text": "12345678", "type": "input"}, {"text": "12345678", "type": "input"}, {"text": "123456789101112", "type": "input"}, {"text": "<think>", "type": "reasoning"}, {"text": "123456789x + 5 = 11, 123x。", "type": "reasoning"}, {"text": "1234567891011121314151617181920x = 11 - 5 = 6", "type": "reasoning"}, {"text": "1234567891011x = 6 / 2 = 3", "type": "reasoning"}, {"text": "123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100", "type": "reasoning"}, {"text": "</think>", "type": "reasoning"}, {"text": "12345", "type": "output"}]]
```

`<think>...</think>` 部分是 hidden thinking，**用户看不到但要付费**（因为占 GPU 计算）。OpenAI o1 把这部分完全隐藏，DeepSeek-R1 / Claude Extended Thinking 给开发者选项可以查看。

» 推理模型 vs 多步 ReAct 的本质区别

维度	多步 ReAct（普通 LLM）	推理模型
"思考"的位置	在多次 API 调用之间（外显）	在单次 API 调用内部（隐式）
思考能用什么	工具结果（外部信息）	仅 prompt + 内部参数
谁控制思考节奏	框架（LangGraph 节点）	模型自己（自适应）
单次调用成本	普通 token	推理 token（贵 5-10x）
适合	需要外部数据的任务	纯推理任务（数学 / 规划 / 代码）

» 什么场景用推理模型才划算

任务	用推理模型	用普通 LLM + ReAct
数学竞赛题	推理模型（步骤多但不用工具）	不行
复杂规划（旅行 / 项目计划）	推理模型（一次出完整计划）	也行但要多步迭代
简单查询（天气 / FAQ）	用推理模型浪费钱	普通模型
需要外部数据（订单 / 物流）	推理模型也得调工具，没特别优势	ReAct 可控性更好
代码审查（无需运行）	推理模型	普通模型不够深
代码生成（要跑测试）	都行，但要配工具	ReAct + 沙箱

► 推理模型的"思考预算"控制（reasoning_effort）

OpenAI o3 等模型支持 `reasoning_effort` 参数：

CODE

```
response = client.chat.completions.create(  
    model="o3",  
    messages=[...],  
    reasoning_effort="medium" # low / medium / high  
)
```

等级	思考 token 量	价格倍数	适合
low	几百-几千	2-3× 普通	中等复杂
medium	几千-万	5×	复杂规划
high	几万-几十万	10×	数学竞赛级

Claude Extended Thinking 类似: `thinking={"type": "enabled", "budget_tokens": 100000}`。

智能体调用 = N 次 LLM API

CODE

```
T+0    API call 1  Input: 5k → Output: tool_use("search_order")  (50 token)  
T+1.5s  ???search_order → ??  
  
T+1.7s API call 2  Input: 6k → Output: tool_use("search_logistics")  (50 token)  
T+3.2s  ???search_logistics → ??  
  
T+3.5s API call 3  Input: 8k → Output: ??????????token?  
T+5s    ?????  
  
????s (LLM 3s + ?????s + ?????s)
```

▶ Token 累积爆炸

调用	输入 token	累积
1	5k	5k
2	5k + 工具结果 1 (1k)	6k
3	5k + 工具 1 + 工具 2	8k
10 (长任务)	...	30-50k

每次调用都重发完整历史 —— 这是“上下文窗口爆炸”的实际机制。

▶ **成本估算公式 (Claude Sonnet 4.6 / 2026.04)**

CODE

```
Input [?]token
Output [?]token
Cache [?]token (90% [?])
```

$$\approx 5 \times (10k \times \$3/M + 500 \times \$15/M) \approx \$0.19$$

[?][?][?][?][?][?][?][?] Prompt Caching, 5k [?][?][?][?]
 $\approx 5 \times (5k \times \$0.30/M + 5k \times \$3/M + 500 \times \$15/M) \approx \$0.12$

Haiku :

失败模式与节点级保护

失败	原因	防御
死循环	模型一直 tool_use 不给 final	<code>recursion_limit</code>
上下文爆	历史超 context window	截断 / 摘要
工具 timeout	函数挂住	工具超时 + retry
模型限流	429	退避 + fallback
幻觉调不存在工具	structured decoding 失败	工具白名单校验

每个都需要节点级保护（详见 §3.5 / §5.6）。

成本架构与 FinOps 深化

▸ 三家厂商完整定价表（2026.04，每百万 token）

模型	Input	Output	Cache 写入	Cache 命中	Batch（50% 折扣）
Claude Opus 4.7（1M context）	\$15	\$75	\$18.75	\$1.50	\$7.50 / \$37.50
Claude Sonnet 4.6	\$3	\$15	\$3.75	\$0.30	\$1.50 / \$7.50
Claude Haiku 4.5	\$0.80	\$4	\$1	\$0.08	\$0.40 / \$2
GPT-4o	\$2.50	\$10	—	\$1.25（50%）	\$1.25 / \$5
GPT-4o-mini	\$0.15	\$0.60	—	\$0.075	\$0.075 / \$0.30
OpenAI o3	\$10	\$40	—	\$2.50	\$5 / \$20
OpenAI o3-mini	\$1.10	\$4.40	—	\$0.55	\$0.55 / \$2.20
Gemini 2.5 Pro（< 200k）	\$1.25	\$10	—	\$0.31（25%）	—
Gemini 2.5 Pro（> 200k）	\$2.50	\$15	—	\$0.625	—
DeepSeek-V3	\$0.27	\$1.10	—	—	—
DeepSeek-R1（推理）	\$0.55	\$2.19	—	—	—

关键价格规律

规律	解释
Output 比 Input 贵 4-5x	模型生成 token 的边际成本是 forward 的几倍
Cache 命中省 75-90%	Anthropic 90% / OpenAI 50% / Google 75%
Batch API 省 50%	牺牲实时性换价格
推理模型贵 5-10x	Hidden thinking token 也算钱
国产模型便宜 5-10x	DeepSeek / 千问 / 智谱 等

真实月度账单案例

案例 A：中型 SaaS 客服智能体

CODE

```

100k x 5 x 5 = 250 LLM
Sonnet 4.6

Input: 8k token
Output: 300 token
Cache

Input (7500k) : 7500k x 0.3 x $3/M = $54,000
Input (7500k) : 7500k x 0.7 x $0.30/M = $12,600
Output : 7500k x 0.7 x $0.30/M = $33,750

Caching: $234,000 / 
Opus 4.7: $501,750 / 
```

案例 B：编程智能体（高 token 长循环）

[?]opus 4.7
 10k × 50 × 30 = 1500 [?][?][?][?][?][?]

[?] [?] [?] [?] [?] [?] [?]
 Input: 30k token ([?] [?] [?] [?] [?] [?] [?])
 Output: 800 token
 Cache [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]

Input (1500k) : $1500 \times 0.8 \times \$1.50/M = \$16,200$
 Input (1500k) : $1500 \times 0.2 \times \$15/M = \$40,500$
 Output : $1500 \times M = \$27,000$

[illegible]

▶ 6 大成本优化策略

策略	节省	实现
Prompt Caching	60-90%	<code>cache_control</code> 标记稳定 prefix
Batch API（非实时任务）	50%	提交批量任务、结果延迟 24h 拿
模型路由（小模型优先）	60-80%	简单查询走 Haiku，复杂走 Sonnet/Opus
截断/摘要历史	30-50%	长对话不要全塞
结构化输出（替代字符串解析）	5-15%	省解析的 token
国产模型路由（数据合规允许时）	80%+	简单任务用 DeepSeek / 千问

▶ 模型路由实例 (Sonnet + Haiku 混合)

CODE

```
def route_model(query: str) -> str:
    """Route model based on query length and token count"""
    if len(query) < 100 and not any(k in query for k in ["sonnet", "haiku"]):
        return "claude-haiku-4-5" # 80% chance
    return "claude-sonnet-4-6" # 20% chance
```

▶ 自部署 vs API 的盈亏平衡点

按 Llama 3.3 70B 自部署 vs Claude Sonnet API :

维度	自部署 (4× H100)	Sonnet API
固定成本	~\$25k/月 (GPU 租赁)	0
可变成本	0 (本地推理)	按 token 计
盈亏平衡 token 数	~80 亿 input token / 月	—
适合	大规模稳态流量	中小规模 + 弹性需求

「结论：日均 < 2.5 亿 input token 的项目，API 比自部署便宜。」

▶ 关键 FinOps 指标（必上仪表盘）

指标	含义	告警阈值
cost_per_request	单次调用成本	上涨 > 30% 告警
cost_per_user / DAU	单用户成本	> \$0.50/天 调查
cost_per_session	单会话成本	> \$0.10 调查
cache_hit_rate	Prompt Caching 命中率	< 50% 调查
token_efficiency	output_tokens / input_tokens	< 0.05 说明 prompt 太啰嗦
model_mix	各模型流量占比	Opus > 20% 调查
error_cost	失败重试的成本	> 5% 总成本调查

▶ 异常监控（Cost Burst Detection）

CODE

```
def detect_burst(recent_costs: list[float]) -> bool:
    mean, std = statistics.mean(recent_costs[-100:]), statistics.stdev(recent_costs[-100:])
    current = recent_costs[-1]
    return current > mean + 3 * std    # 3σ 异常检测

# 使用示例
recent_costs = [...]
if detect_burst(recent_costs):
    print("检测到成本异常!")
```

▸ 延迟成本权衡（Pareto 前沿）

模型	价格（Input）	速度（token/s）	适合
GPT-4o-mini	\$0.15/M	250	高频简单查询
Haiku 4.5	\$0.80/M	200	高频中等查询
Sonnet 4.6	\$3/M	80	一般生产
Gemini 2.5 Pro	\$1.25/M	100	长上下文
GPT-4o	\$2.50/M	100	一般生产
Opus 4.7	\$15/M	35	复杂规划
o3	\$10/M	40（含 thinking）	深度推理

「 Pareto 前沿：Haiku → Gemini 2.5 Pro → Sonnet → Opus 是"价格/能力"最优组合。GPT-4o 在 Pareto 前沿上略劣（同价位 Sonnet 更强）。

小结一行：成本三大杠杆 —— Caching（省 60-90%）、模型路由（省 60-80%）、Batch API（省 50%）；FinOps 必看 7 项指标，不开仪表盘就是黑盒烧钱。

小结一行：智能体层的本质是 N 次 LLM API 调用 + 工具调用 + 累积状态 —— CoT/ReAct 是范式、推理模型是内化、成本和延迟是字节级账本。

§0.5 整体架构：现代主流 Agent 蓝图

「 基础流程对应：把所有节点组装成生产级架构。

6 层 Graph-based Architecture (Mermaid #31)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    User([User]) --> Planner([Planner])
    Planner --> GC([Graph Controller])
    GC --> Executor([Executor])
    GC --> Critic([Critic])
    Executor --> Tools([Tools])
    Tools --> Memory([Memory & State])
    Critic --> Executor
    Memory --> GC
    Memory --> Planner

    style User fill:#fef3c7
    style Planner fill:#dbeafe,stroke:#3b82f6,stroke-width:3px
    style GC fill:#fce7f3,stroke:#ec4899,stroke-width:3px
    style Executor fill:#d1fae5,stroke:#10b981,stroke-width:3px
    style Critic fill:#e9d5ff,stroke:#a855f7,stroke-width:3px
    style Tools fill:#fed7aa
    style Memory fill:#ddd6fe
  
```


6 层职责详解

层	职责	8 节点对应	5 代叠加对应	代表实现	后续章节
Planner	拆任务 / 生成路径 / 控制复杂度	②③	范式层 (2 代 Plan-Execute)	LangGraph Plan-Execute	§6.3
Graph Controller	流程控制 / 分支循环 / 状态机调度	③⑦	编排层 (3 代)	LangGraph StateGraph	§4
Executor	具体执行 / 调工具 / 完成子任务	②③④⑤	范式层 (1 代 ReAct)	<code>create_react_agent</code>	§5
Critic	校验结果 / 触发 retry/replan	⑥⑦	范式层 (2 代 Reflexion)	LLM-as-judge / Promptfoo	§15.2
Tools	RAG / 代码 / API / DB	④	协议层 + 运行层	<code>@tool</code> / ToolNode / MCP / E2B	§6.7 / §16
Memory	三层记忆体系	⑥	编排层 + 协议层	Checkpoint + Store + Vector DB	§0.3 / §6.5

6 层 ↔ 8 节点 ↔ 5 代叠加 三向映射

视角	强项	关注
6 层架构 (§0.5)	工程蓝图	怎么搭
8 节点流程 (§0.6)	执行流程	一次调用怎么走
5 代叠加 (§1.10)	历史演进	每代加了什么能力

三个视角互补，不冲突。

动态运行流程

CODE

```
1. User [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?]
2. Planner [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?]
3. Graph Controller [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?]
4. Executor [?] [?] ReAct (Thought → Action → Observation [?] [?])
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
5. Tools ([?] [?] [?] [?] [?] [?] [?] [?] [?] [?] API / DB)
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
6. Memory ([?] [?] [?] KV cache / [?] [?] [?] Checkpoint / [?] [?] [?] Store)
[?] [?] [?] [?] [?] [?] [?] [?]
7. Critic [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] Executor / Planner replan
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] User
```

解决 ReAct 单体的 3 大短板

维度	单体 ReAct	Graph-based Agent
可控性	模型自决，乱跳	Graph Controller 显式控流程
可扩展性	单线程	可插多 Executor / Agent / 工具
可恢复性	一错全错	Critic + retry/replan, 失败可恢复

单 Agent → Multi-Agent 升级路径

CODE

```
Planner Agent
[?][?][?][?]
Executor Agent A / Executor Agent B / ...    ([?][?][?][?][?][?])
[?][?][?][?]
Critic Agent
[?][?][?][?]
Tool Agent ([?][?][?][?][?])
```

详见 §6.3 多智能体模式（Supervisor / Plan-Execute / Swarm）。

一句话浓缩

「现代 Agent 架构 = 🧠 Planner（想做什么）+ [?][?] Graph Controller（什么时候做、做几次）+ ⚙️ Executor/ReAct（具体怎么做）+ 📖 Memory（记住）+ ✓ Critic（纠错）」

把这张图记在心里，后续 §2-§6 都是在不同层填具体实现。

小结一行：业界 2026 落地蓝图就是 6 层 —— ReAct 是 Executor 内核、LangGraph 是 Graph Controller、MCP 是 Tools 协议、Checkpoint+Store+Vector DB 是 Memory 三层。

§0.6 全书坐标

「本节是全书的"导航地图"——把 8 节点流程图、三件套对照、演进时间线、学习地图四张全局图集中放在这里，作为后续章节的统一参考。

8 节点基础流程图（Mermaid #1）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Start([Start]) --> N1["① 节点1"]
    N1 --> N2["② 节点2"]
    N2 --> N3["③ 节点3"]
    N3 --> N8["⑧ 节点8"]
    N3 --> N4["④ 节点4"]
    N4 --> N5["⑤ 节点5"]
    N5 --> N6["⑥ 节点6"]
    N6 --> N7["⑦ 节点7"]
    N7 --> N2
    N7 --> N8
    N8 --> End([End])

    style N1 fill:#fef3c7,stroke:#f59e0b
    style N2 fill:#dbeafe,stroke:#3b82f6
    style N3 fill:#fce7f3,stroke:#ec4899
    style N4 fill:#d1fae5,stroke:#10b981
    style N5 fill:#d1fae5,stroke:#10b981
    style N6 fill:#fef3c7,stroke:#f59e0b
    style N7 fill:#fce7f3,stroke:#ec4899
    style N8 fill:#dbeafe,stroke:#3b82f6
```

8 节点工程问题

节点	名称	工程问题	后续章节
①	上下文组装	怎么拼？怎么截？怎么注入 RAG？	§1.4 / §2.3 / §6.5
②	大模型推理	怎么思考？要不要显式 Thought？	§1.3 / §17.6
③	决策路由	字符串解析还是结构化输出？	§1.4 / §3.2 / §5.1
④	工具调用	怎么定义？多工具并发？跨厂商？	§3.3 / §6.7 / §16.1 MCP
⑤	工具返回	怎么回填？大返回值怎么截？	§3.5 / §5.1
⑥	状态管理	中间产物存哪？跨调用？跨会话？	§0.3 / §2.7 / §5.3 / §6.5
⑦	循环控制	什么时候停？卡死怎么救？ HITL？	§3.5 / §5.4 / §6.2
⑧	输出格式化	state → 用户回复？流式？JSON？	§2.9 / §6.6

三件套在 8 节点中各管什么

节点	ReAct（推理范式）	LangChain（第一代框架）	LangGraph（第二代框架）
① 上下文	ReAct prompt 模板	<code>ChatPromptTemplate</code>	节点函数 + State 注入
② 推理	显式 Thought	<code>bind_tools()</code>	LLM 节点
③ 决策	字符串 <code>Action: <name></code>	<code>AgentExecutor</code> 黑盒	条件边
④ 工具	prompt 描述	<code>@tool</code> + <code>Tool</code>	<code>@tool</code> + <code>ToolNode</code> + MCP
⑤ 返回	拼回 prompt	AgentExecutor 内部	写回 State
⑥ 状态	prompt 历史	4 类 Memory（会话级）	State + Checkpoint + Store
⑦ 循环	模型自决	<code>max_iterations</code>	<code>recursion_limit</code> + interrupt
⑧ 输出	最后一次输出	OutputParser	5 种 stream 模式

结论：ReAct = 范式（精神层），LangChain = 组件库（货架层），LangGraph = 编排引擎（蓝图层），三者**分层堆叠**，业界主流玩法是组合用。

演进时间线

时间	事件	8 节点影响
1948	Wiener 《控制论》	闭环范式起源
2017	Transformer 论文	模型基石
2022.01	Wei et al. CoT	节点 ②
2022.10	Yao et al. ReAct	节点 ②③⑤
2022.10	LangChain 创建	节点 ① 到 ⑧ 第一代
2023.03	AutoGPT	ReAct 破圈
2023.06	OpenAI Function Calling	节点 ③④
2024.01	LangGraph 0.0	节点 ⑥⑦ 升级
2024.05	Anthropic Tool Use	节点 ④
2024.08	Anthropic Prompt Caching	节点 ① 降本
2024.09	LangChain v0.3	AgentExecutor 退役
2024.10	Anthropic Computer Use	节点 ④ 视觉
2024.11	Anthropic MCP	节点 ④ 协议层
2024.12	OpenAI o1 / Anthropic Extended Thinking	节点 ② 推理模型
2025	LangGraph 0.2 / Skills	节点 ⑥⑦ 进阶
2026.Q1	LangGraph Platform GA	部署形态多元

学习地图 (Mermaid #2)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph LR
    Z["§0 学习地图"]
    A["§1 ReAct"]
    B["§2 LangChain"]
    C["§3 LangChain"]
    D["§4 LangGraph"]
    E["§5 LangGraph"]
    F["§6 LangGraph"]
    G["§7 LangGraph"]
    H["§8 custom_agent"]
    subgraph PartB [Part B]
        direction TB
        B
        C
        D
        E
        F
    end
```

```
Z --> A --> B --> C --> D --> E --> F --> G --> H
H -. Part B .-> PartB
```

```
style Z fill:#fef3c7
style A fill:#dbeafe
style B fill:#d1fae5
style C fill:#d1fae5
style D fill:#fce7f3
style E fill:#fce7f3
style F fill:#fce7f3
style G fill:#e9d5ff
style H fill:#e9d5ff
style PartB fill:#fed7aa
```

§0.7 本章小结

#	核心结论
1	模型层 ：LLM 是 4 步流水线（Token / Embedding / Attention / Sample），本质是"概率分布展开"——这解释幻觉、CoT 有效性、 $O(n^2)$ 上下文限制
2	接口层 ：API 调用 7 步流水线，工具调用靠结构化解码，三家协议（OpenAI / Anthropic / Google）字节级有差异
3	记忆层（重点） ：三层模型 —— 短期（Context+KV Cache+Prompt Caching） / 中期（Memory / Checkpoint） / 长期（Vector DB / Store / Skills）
4	KV Cache + Prompt Caching 是 $O(n^2)$ 优化和省钱关键，命中部分省 60-90%
5	LangGraph Checkpoint 是中期记忆生产级标配（thread_id 续跑、跨进程持久化、时间旅行）
6	Vector DB / Store API / Anthropic Skills 是长期记忆三种互补实现
7	智能体层 ：CoT / ReAct 不是逻辑推理，是 token 序列展开 + KV cache 跨步复用；推理模型把这内化了
8	智能体调用 = N 次 LLM API + 工具调用，token 累积爆 / 成本公式 / 延迟构成都是字节级账本
9	整体架构 ：6 层 Graph-based Agent 蓝图（Planner / Graph Controller / Executor / Critic / Tools / Memory）是 2026 业界主流
10	6 层 ↔ 8 节点 ↔ 5 代叠加 三向视角互补

§0.8 反模式速记

反模式	错在哪	正确做法
把 workflow 叫"智能体"	流程预定义就不是智能体	简单场景用 workflow
期望 LLM 自己记得过去对话	模型每次调用都是无记忆	用 Memory (§2.7) / Checkpoint (§5.3) / Store (§6.5)
长对话不开 Prompt Caching	多花 60-90% 钱	<code>cache_control</code> 标记稳定 prefix
Skills / Memory 不分租户	数据泄漏	<code>checkpoint_ns</code> + namespace 隔离
PDF / 大字段直接塞 state	Checkpoint 爆	存 S3, state 存 URL
推理模型还嵌套大量 ReAct 循环	推理模型已"少而深"	减少外层 ReAct 步数
跨 thread 期望续上	不同 thread_id 完全隔离	用 Store API 跨 thread 共享
同 thread_id 跑不同任务	历史污染	每个任务独立 thread_id

§0.9 术语速查

术语	中文	含义
LLM (Large Language Model)	大语言模型	基于 Transformer 的生成式语言模型
Token	词元	模型处理的基本单位（约 0.75 英文字 / 0.5 中文字）
BPE (Byte Pair Encoding)	字节对编码	Tokenizer 算法
Embedding	嵌入向量	Token ID $\rightarrow$ d 维向量
Self-Attention	自注意力	序列内 token 关系建模
KV Cache	键值缓存	Attention 优化，让生成阶段 $O(n^2)$ 而非 $O(n^3)$
Context Window	上下文窗口	单次调用模型能塞的最大 token 数
Prompt Caching	提示词缓存	厂商把同 prefix 的 KV cache 持久化
Tool Use / Function Calling	工具调用	模型输出结构化 JSON 调外部函数
Structured Decoding	结构化解码	grammar-constrained, 让模型输出符合 schema
SSE (Server-Sent Events)	服务器推送事件	HTTP 长连接 + 行级事件，流式输出协议
TTFT (Time To First Token)	首字延迟	用户从发起到看到第一个字的耗时
Memory	记忆	节点 ⑥ 状态管理（LangChain 4 类 / LangGraph 各种）
Checkpoint	检查点	LangGraph State 的快照（中期记忆）

术语	中文	含义
Store API	存储接口	LangGraph 跨 thread 长期记忆
BaseStore	存储基类	Store API 的抽象
Vector DB	向量数据库	语义召回的后端（Pinecone / Qdrant / pgvector）
Embedding 模型	嵌入模型	文本 → 向量的模型
Cosine Similarity	余弦相似度	向量间相关性度量
<code>thread_id</code>	会话标识	同 thread_id 的 invoke 自动续上
<code>checkpoint_ns</code>	检查点命名空间	多租户隔离
Namespace	命名空间	Store API 中的层级元组
Skills	技能	Anthropic 推的"能力包装"（含 prompt + tools + memory）
CoT (Chain-of-Thought)	思维链	让模型一步一步思考的 prompt 技术
ReAct	推理-行动范式	CoT + 工具调用
Reasoning Model	推理模型	o1 / o3 / Claude Extended Thinking / DeepSeek-R1
Hidden Thinking	隐式思考	推理模型的内部思考 token, 用户不可见
Hallucination	幻觉	模型编造不存在的事实
Planner	规划层	主流架构第 1 层

术语	中文	含义
Graph Controller	调度核心	主流架构第 2 层
Executor	执行层	主流架构第 3 层 (ReAct 内核)
Critic	评估层	主流架构第 4 层
Time Travel	时间旅行	get_state_history + 分支重放
Branching	分支	从历史 checkpoint 创新分支
Multi-Head Attention	多头注意力	把 Q/K/V 切成 N 头并行 attention, 每头学不同关系
MQA (Multi-Query Attention)	多查询注意力	多个 Query 头共享 1 组 K/V, KV Cache 减小 N 倍
GQA (Grouped-Query Attention)	分组查询注意力	多个 Query 头分组共享 K/V, 2024+ 现代大模型标配
RoPE (Rotary Position Embedding)	旋转位置编码	把 Q/K 在每个位置上旋转特定角度; Llama 3+ 主流
Sinusoidal	正弦位置编码	原始 Transformer 用的固定向量位置编码
FFN (Feed-Forward Network)	前馈网络	Transformer 一层里的两个全连接层 + 激活函数
Layer Norm	层归一化	Transformer 每子层前的数值稳定性归一
Residual	残差连接	让深层模型可训练的"跳层"连接
FSA	有限状态自动机	把 JSON Schema 编译成的状态机, 约束解码用

术语	中文	含义
Outlines / xgrammar	约束解码引擎	开源 grammar-constrained decoding 实现
MTEB	嵌入评测	Massive Text Embedding Benchmark 业界标准
Matryoshka Embeddings	俄罗斯套娃嵌入	一份模型可截断到不同维度而不大幅损失精度
BM25	经典关键字检索	Best Match 25, 倒排索引 + 词频 + 逆文档频率
Hybrid Retrieval	混合检索	dense (向量) + sparse (BM25) 并行 + RRF 合并 + Reranker 精排
RRF (Reciprocal Rank Fusion)	倒数排名融合	合并多个检索排序的算法
Reranker	重排序器	直接读 query+doc 对判分的精排模型, 比 embedding 精度高
Verifier	验证器	推理模型 RL 训练时的"答案是否正确"自动判分器
PPO / GRPO	强化学习算法	Proximal Policy Optimization / Group Relative Policy Optimization
<code>reasoning_effort</code>	思考预算参数	OpenAI o3 等推理模型控制 hidden thinking 长度的开关

§0.10 推荐下一章

下一章: [§1 ReAct : 把基础流程节点 ②③ 显式化](#) —— §0 给了完整原理架构, §1 深入讲 ReAct 这个最核心的范式如何形成、它的生态位置、以及 5 代演进图。

§1 ReAct：把基础流程节点 ②③ 显式化

§1.0 本章定位

项	内容
在基础流程中的位置	节点 ② 大模型推理 + ③ 决策路由 + ⑤ 观察
与上下章的因果链	§0 给了流程图，但流程里"模型怎么思考 → 怎么决定下一步"这两步是黑盒；本章把它打开。下一章 §2 讲框架（LangChain）怎么把 ReAct 工程化
学完能做什么	(1) 解释思维链（CoT）与 ReAct 的关系；(2) 写出 ReAct 的标准 prompt 模板；(3) 列举 ReAct 的 5 大局限及其解法（Plan-Execute / Reflexion / ToT）；(4) 区分 ReAct 字符串协议 vs OpenAI Function Calling vs Anthropic Tool Use

§1.1 没有 ReAct 之前：纯提示词时代的窘境

「在基础流程中的位置：节点 ②（推理）单独运转的状态。」

场景引入

2022 年中，给大模型加"工具"的玩法非常原始。开发者想让模型查天气，会这样写 prompt：

CODE

```

[?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?]T00L_CALL: weather(city=[?][?])
[?][?][?][?][?][?][?]

[?][?][?][?][?][?][?][?][?][?][?][?]
[?][?][?]

```

模型可能输出 `T00L_CALL: weather(city=[?][?])` —— 但也可能输出：

模型实际输出	框架解析结果
<code>T00L_CALL: weather(city=[?][?])</code>	成功
<code>[?][?][?][?][?][?][?][?][?][?][?]</code>	失败（编造，没真去查）
<code>[?][?][?][?][?][?][?][?][?][?]</code>	失败（无用废话）
<code>[?][?][?]weather([?][?])[?][?][?][?][?]</code>	失败（格式错乱）
<code>Action: weather\nCity: [?][?]</code>	失败（字段散落）

痛点逐节点拆解

痛点	在基础流程的哪个节点	后果
模型不会"思考再行动"	节点 ②	直接给答案，常错且不可解释
模型不区分"调工具"和"答用户"	节点 ③	输出格式不稳定，框架解析失败率高
工具结果回填后模型容易"忘了任务"	节点 ⑤	多次调用后偏离原始问题
没有显式的"中间状态"	节点 ⑥	全靠 prompt 历史，长就爆

简化流程图（Mermaid #3）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Q["Q[?]"] --> P["P[?] Prompt  
Thought [?]"]
    P --> M["M[?]"]
    M --> A["A[?]"]
    A --> Out["Out[?]"]
```

```
style M fill:#fde68a,stroke:#f59e0b,stroke-width:3px
style A fill:#fecaca,stroke:#ef4444
```

小结一行：纯 Prompt 时代节点 ② 是个黑盒，输出格式不稳定 → 节点 ③ 解析全靠运气。

§1.2 ReAct 论文核心贡献：把"思考"和"行动"解耦

「在基础流程中的位置：把节点 ② 推理拆成"显式思考"+"显式行动"两步。

论文背景

ReAct 论文全称 *ReAct: Synergizing Reasoning and Acting in Language Models*，2022 年 10 月由 Princeton 的 Shunyu Yao 等人提出（ICLR 2023）。

ReAct 这个名字是个双关：- **R**easoning + **A**cting = ReAct（推理 + 行动）- 同时也呼应英文单词 react（反应、应对）

核心贡献：把模型输出强制分三段

ReAct 的精髓 —— 在 prompt 里规定模型必须按以下三段交替输出：

CODE

Thought: [?][?][?][?][?][?][?][?][?][?][?][?][?][?]
Action: weather
Action Input: [?][?]
Observation: [?][?][?][?][?][?][?][?][?][?][?][?]
Thought: [?][?][?][?][?][?][?][?][?][?][?][?][?][?]
Action: Final Answer
Action Input: [?][?][?][?][?][?][?][?][?][?][?][?]

字段	含义	节点
Thought	模型的"思考"——为什么要做下一步	节点 ②
Action	选哪个工具 / 还是给最终答案	节点 ③
Action Input	工具参数	节点 ④
Observation	工具返回的结果（由框架填回）	节点 ⑤

为什么"显式 Thought"如此重要

ReAct 之前已有思维链（CoT）——让模型一步一步思考。但 CoT 只解决了节点 ②（让推理质量上升），没解决节点 ③④⑤（行动）。

范式	节点 ② 思考	节点 ③④ 行动	关系
纯 Prompt	黑盒	黑盒	一锅炖
思维链（CoT, 2022.01）	显式	不涉及	只解决推理
ReAct（2022.10）	显式	显式	思考 + 行动配套

ReAct 论文的核心论点 —— 让模型把"为什么要调这个工具"先说出来再调，会大幅降低错误率。

论文实验结果

任务	纯 CoT 准确率	ReAct 准确率	提升
HotpotQA（多跳问答）	27.4%	35.1%	+7.7
FEVER（事实核查）	56.3%	62.0%	+5.7
ALFWorld（家居仿真）	42%	71%	+29
WebShop（网购代理）	28.7%	40.0%	+11.3

ALFWorld 上 +29 个百分点，证明带"行动"的任务里 ReAct 极强。

小结一行：ReAct = 思维链 + 工具调用，让模型的推理过程从"看不见摸不着"变成"可观察可干预"。

§1.3 思考-行动-观察循环：三步缺一不可

在基础流程中的位置：节点 ②③⑤ 形成的最小闭环。

三步的工程含义

步骤	谁做	内容
Thought（思考）	大语言模型生成	"我需要先查 X 再做 Y"
Action（行动）	大语言模型生成	"调用 search 工具，参数是 X"
Observation（观察）	框架填回	工具的真实返回值

内循环图示（Mermaid #4）

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Start([Start]) --> T1["Thought 1  
search(X)"]
    T1 --> A1["Action 1  
search(X)"]
    A1 --> O1["Observation 1  
search(X)"]
    O1 --> T2["Thought 2  
calculator(...)"]
    T2 --> A2["Action 2  
calculator(...)"]
    A2 --> O2["Observation 2  
calculator(...)"]
    O2 --> T2
    T2 --> FA["Action: Final Answer  
calculator(...)"]
    FA --> End([End])

    style T1 fill:#dbeafe
    style T2 fill:#dbeafe
    style A1 fill:#d1fae5
    style A2 fill:#d1fae5
    style FA fill:#d1fae5
    style O1 fill:#fef3c7
    style O2 fill:#fef3c7
```

为什么三步缺一不可

缺哪步	后果
缺 Thought	模型直接动手，错误率回到纯 Prompt 时代
缺 Action	思考完了不行动，等于纸上谈兵
缺 Observation	行动后看不到结果，无法迭代

ReAct 的本质 — 强制把“想什么”和“做什么”分开写出来，让框架能介入每一步。

小结一行：三步循环让节点 ② → ③ → ④ → ⑤ → ② 形成可观察的反馈环。

§1.4 ReAct 提示词模板逐字段拆解

「在基础流程中的位置：节点 ① 上下文组装的具体模板。

标准 ReAct 模板

CODE[illegible]

```
{tools_description}
```

□ □ □ □ □ □ □

Question:

Thought:

Action: [?][?][?][?][?][?][?][?]

Action Input:

Observation: [][][][][][][]

```
... (Thought / Action / Action Input / Observation [?][?][?][?][?][?])
```

[illegible]

Final Answer:

□ □ □

Question: {input}

```
{agent_scratchpad}
```


关键字段解释

字段	作用	注意
<code>{tools_description}</code>	工具列表（每个工具：名字 + 描述 + 参数 schema）	列得越清楚模型选得越准
<code>Question:</code>	用户原始问题	模型要"记住"
<code>Thought:</code>	强制模型先思考	缺这个就是纯 Prompt 时代
<code>Action:</code>	工具名（必须严格匹配工具列表中的名字）	字符串解析的关键
<code>Action Input:</code>	工具参数	一般要求 JSON 格式
<code>Observation:</code>	工具返回结果（由框架填，不由模型生成）	模型生成到 <code>Action Input:</code> 后停
<code>Final Answer:</code>	终止信号	框架检测到这个就退出循环
<code>{agent_scratchpad}</code>	之前的 Thought/Action/Observation 历史	多轮迭代时累积

少样本示例（few-shot examples）怎么选

ReAct 论文里发现 —— **prompt 里给 1-3 个范例**比纯 zero-shot 效果好得多。范例选择原则：

原则	解释
多样性	范例覆盖不同工具组合，不要全是"先 search 再 calculator"
代表性	范例匹配真实场景的复杂度（不要太简单也不要太罕见）
简洁	每个范例 5-8 步以内，避免占用过多 token
明确终止	范例必须以 <code>Final Answer:</code> 结尾，让模型学会"停"

少样本示例不是越多越好 —— 放 5 个之后边际效益急剧下降，且 token 成本线性上升。

节点 ① 上下文组装顺序

CODE

```
[System Prompt: ReAct ???]  
[Tools Description: ???]  
[Few-shot Example 1]  
[Few-shot Example 2]  
[Few-shot Example 3]  
[Question: ???]  
[Agent Scratchpad: ???Thought/Action/Observation]
```

小结一行：ReAct 模板的精髓在"格式约束 + 少样本示范"，节点 ① 上下文组装的胶水代码全在这里。

§1.5 ReAct 兄弟范式：补什么不足

「在基础流程中的位置：扩展 ReAct 在节点 ②③⑥⑦ 的能力。

ReAct 只是众多智能体推理范式中最早爆火的一个，2023-2024 年陆续出现了一系列"补丁"。

计划-执行 (Plan-and-Execute)

维度	内容
论文	Wang et al., 2023
核心改进	把节点 ② 拆成两步：先 计划 整个任务（输出 N 步计划清单），再 逐步执行 每一步
解决什么	ReAct 的"短视" —— 模型每次只看眼前一步，长任务容易迷路
缺点	计划阶段可能不准；每步执行都要回头对照计划，token 成本高
适用	任务步骤多（>5 步）、步骤间有强依赖

反思 (Reflexion)

维度	内容
论文	Shinn et al., 2023
核心改进	在节点 ⑥ 加一个"反思记忆" —— 每次任务失败后让模型总结"为什么错"，存进长期记忆，下次同类任务前注入
解决什么	ReAct 的"无记忆" —— 重复犯同样错误
缺点	需要明确的"成功/失败"信号；反思总结本身可能错
适用	有验证机制（编译通过 / 测试通过 / 用户反馈）的迭代场景

思维树（Tree-of-Thoughts, ToT）

维度	内容
论文	Yao et al., 2023（同 ReAct 作者）
核心改进	把节点 ② 从"线性思考"扩展为"树形探索" —— 每步生成多个候选思路，评估后选最优分支继续
解决什么	ReAct 的"局部最优陷阱" —— 一旦走错路就一条道走到黑
缺点	token 成本暴涨（同时维护多个分支）； 评估函数设计困难
适用	24 点游戏、复杂数独、创意写作、复杂规划

兄弟范式与 ReAct 对照

范式	改进的节点	增加的成本	何时选
ReAct	②③⑤	基线	默认起点
Plan-Execute	② 拆两步	+1 次大调用（规划）	任务长 / 步骤多
Reflexion	⑥ 加长期记忆	+1 次反思调用	可迭代场景
Tree-of-Thoughts	② 扩为树	×N 倍（N 个分支）	高复杂度规划

小结一行：ReAct 是"基线"，其他范式都在 ReAct 上某个节点做加法 —— 工程实践中先 ReAct，发现具体痛点再选对应兄弟范式。

§1.6 ReAct 的"原生竞争方案": Function Calling 与 Tool Use

「在基础流程中的位置: 节点 ③④ 跳过字符串解析。

问题: ReAct 的字符串解析很脆

ReAct 让模型输出 `Action: weather` 这种字符串, 框架要正则提取工具名。一旦模型输出 `Action: ???weather` 或 `Action: weather()` 就解析失败。

2023.06 OpenAI Function Calling 的解法

OpenAI 2023.06 发布的 Function Calling 是这么做的:

1. 在 API 请求里给一个 `functions` 数组 (每个函数有 name + description + JSON Schema 参数)
2. 模型不再输出文本, 而是返回一个特殊的 `function_call` 字段 (包含 name 和 arguments)
3. 这个字段是结构化 JSON, 不是字符串解析

协议对照

维度	ReAct（字符串解析）	OpenAI Function Calling	Anthropic Tool Use
工具描述位置	prompt 文本	API 字段 <code>functions</code> / <code>tools</code>	API 字段 <code>tools</code>
模型输出	文本 <code>Action:</code> <code>name\nAction</code> <code>Input: ...</code>	JSON <code>{"name": ..., "arguments": ...}</code>	XML 块 <code><tool_use></code>
解析方式	正则	字段访问	内置解析
并行调用	不支持	2023.11 起支持	支持
流式输出	全输出后解析	边流边解析	边流边解析

节点级别的影响

节点	ReAct (旧)	Function Calling / Tool Use (新)
① 上下文 组装	把工具描述写进 prompt 文本	通过 API 参数传
② 推理	用思维链让模型显式 Thought	模型仍然有 reasoning 字段（但框架不强制 ReAct 模板）
③ 决策路 由	框架正则解析 Action:	框架读 JSON 字段
④ 工具调 用	字符串解析后调用	字段访问后调用

现状

2026 年的实际玩法： - 新项目几乎都用 **Function Calling / Tool Use 原生协议**，不再用 ReAct 字符串解析 - **ReAct 的精神（Thought + Action + Observation）依然适用** —— 只是用 JSON 字段实现而非字符串 - **LangChain 的 Tool Calling Agent (§3.2)** 就是这套现代做法

小结一行： ReAct 是范式（精神层），Function Calling 是协议（接口层），两者相辅相成 —— 新项目用 Function Calling 实现 ReAct 思想。

§1.7 历史名作：AutoGPT 与 BabyAGI

「在基础流程中的位置：节点 ⑦ 循环控制的两种典型设计。

2023.03 AutoGPT 走红

AutoGPT 由 Toran Bruce Richards 于 2023.03 在 GitHub 发布，几周内拿到 100k+ star —— 第一个引爆 ReAct 范式的开源项目。

特性	AutoGPT 怎么做
节点 ② 推理	GPT-4 + 长 prompt（角色 / 目标 / 工具）
节点 ⑦ 循环	不限制次数，跑到任务完成或人类停
节点 ⑥ 状态	每步存到本地文件（持久化）
节点 ④ 工具	内置搜索 / 代码执行 / 文件读写 / 浏览网页
标志性 prompt	"You are AutoGPT, an AI designed to autonomously achieve a goal."

遗产：定义了"自主智能体"这个词的大众认知 —— 给个目标就自己跑。 **坑：**没有 token 限制，跑一晚上花 100 美元；经常卡死循环；任务完成质量低。

2023.04 BabyAGI 出现

Yohei Nakajima 几乎同时发布的 BabyAGI 走的是 Plan-Execute 路线：

CODE

```
Objective → [ ]Task List → [ ]Task → [ ]
[ ]
[ ]
[ ]
[ ]Task List
```


特性	BabyAGI 怎么做
节点 ② 推理	三个角色：任务创建器 / 任务优先级排序器 / 任务执行器
节点 ⑦ 循环	直到 Task List 空
节点 ⑥ 状态	用 Pinecone 向量数据库存历史任务（早期 RAG 实践）

遗产：把"多角色协作"思路普及 —— 后来的 Supervisor / Plan-Execute / Swarm 模式都受它启发。

这两个项目的影响

影响维度	内容
范式普及	让"ReAct 智能体"走出学术圈
框架孵化	LangChain 的 AgentExecutor 设计借鉴了 AutoGPT 循环
教训	暴露 ReAct 的所有局限（死循环 / token 爆 / 任务质量），催生 LangGraph
现状	项目本身已不活跃；其精神被各大现代框架吸收

小结一行：AutoGPT 让"自主智能体"破圈，BabyAGI 启发"多角色协作" —— 它们的局限直接催生了 LangGraph 这一代图式编排引擎。

§1.8 ReAct 局限性：每个局限对应基础流程的哪个节点

「在基础流程中的位置：揭示 ReAct 在节点 ①⑤④⑦⑥ 的全面短板。

局限	对应节点	表现	后续解法在哪
上下文爆炸	① 上下文组装	所有 Thought / Action / Observation 都塞进 prompt 历史，10 步后 token 翻 5 倍	RAG (§11.1) + State Reducer (§6.1) + 摘要 Memory (§2.7)
错误传播	⑤ 工具返回	一次工具错误调用，错误信息进 prompt 历史污染后续推理	错误分类 + Retry 边 (§6.7)
无并行	④ 工具调用	ReAct 默认串行：Thought 1 → Action 1 → Observation 1 → Thought 2，无法同时调多个工具	Function Calling 并行 (§1.6) + Send API (§6.2)
无中断	⑦ 循环控制	ReAct 跑起来就是一条道走到黑，中途人类无法插手	HITL (§5.4) + NodeInterrupt (§6.x)
无状态	⑥ 状态管理	重启对话就忘光，无法跨会话累积	Checkpoint (§5.3) + Store API (§6.5)
黑盒决策	③ 决策路由	模型输出 <code>Action: X</code> 后框架照做，没有"如果 X 不合法该怎么办"的机制	显式条件边 (§5.1)

小结一行：ReAct 的 6 大局限完美映射基础流程的 6 个节点 —— LangChain (§2-§3) 补节点级，LangGraph (§4-§6) 补流程级。

§1.9 业界用 ReAct 的真实场景

「在基础流程中的位置：节点 ②③⑤ 在产品中的具体形态。

产品	用 ReAct 思想做什么	框架
Claude Code (Anthropic)	编程任务的 Thought / Action / Observation 循环（读文件 / 改代码 / 跑测试）	自家 SDK + Tool Use
Cursor	编辑器内代码 agent 的对话循环	闭源，应内含 ReAct 思想
Devin (Cognition)	长程编程任务的多步推理	自家框架 + 沙箱
OpenAI Operator (Computer Use)	浏览器操作（点击 / 输入 / 截图）的 ReAct 循环	OpenAI Agents SDK
Anthropic Computer Use	桌面操作的 Thought / Action / Observation 循环	Anthropic SDK + 屏幕截图工具
客服机器人 （如 Klarna）	用户询单 → 查订单 → 查物流 → 提交退款	LangGraph
数据分析助手	写 SQL → 跑 → 看结果 → 改 SQL 的迭代	LangChain / LangGraph

真实场景的共同特征

特征	解释
任务步骤数不固定	简单查询 1 步，复杂排查 10+ 步
工具集相对固定	一个客服 agent 大概 10-20 个工具
中间结果决定后续路径	查不到订单就换思路
用户介入是常态	危险操作前要确认

小结一行：ReAct 的真实战场是"步骤不定、工具固定、需要中间反馈"的任务 —— 这正是 LangGraph 优化的核心。

§1.10 代际演进图：从 ReAct 到 2026（5 代叠加模型）

「在基础流程中的位置：把整个 Agent 范式发展史压成一条时间轴。理解 ReAct 在 5 代演进中是哪一代、被怎样叠加、为什么没死。

场景引入

读者读完前 9 节会有个困惑："ReAct 这么多坑，业界真在用吗？"答案是 —— 不直接用 ReAct，但它是所有现代范式的底层原语。本节用"5 代演进 + 6 层叠加"模型把这点说透。

5 代演进总览 (Mermaid #28)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    G0["? ? ? ? ?"]
    G0 --> G1["? ? ? ? ?"]
    G1 --> G2["? ? ? ? ?"]
    G2 --> G3["? ? ? ? ?"]
    G3 --> G4["? ? ? ? ?"]
    G4 --> G5["? ? ? ? ?"]
    G0 --- Chain-of-Thought
    Chain-of-Thought --- G1
    G1 --- ReAct
    ReAct --- G2
    G2 --- Plan-Execute
    Plan-Execute --- Reflexion-ToT
    Reflexion-ToT --- G3
    G3 --- LangGraph
    LangGraph --- MCP
    MCP --- G4
    G4 --- Reasoning-Models
    Reasoning-Models --- Computer-Use
    Computer-Use --- G5
    G5 --- A2A-Skills
    A2A-Skills --- Agent-as-Code
    Agent-as-Code --- G5
  
```

G0 --> G1 --> G2 --> G3 --> G4 --> G5

```

style G0 fill:#fef3c7
style G1 fill:#dbeafe
style G2 fill:#d1fae5
style G3 fill:#fce7f3,stroke-width:3px
style G4 fill:#e9d5ff
style G5 fill:#fed7aa
  
```

每代的核心创新

代	时间	核心创新	代表方案 / 项目
0	2022.01	让模型一步一步思考	Chain-of-Thought (Wei et al.)
1	2022.10	CoT + 工具调用	ReAct (Yao et al.)
2	2023	修补 ReAct 的"短视"	Plan-Execute / Reflexion / Tree-of-Thoughts
3	2024	系统级 Agent 架构	LangGraph / 多智能体 (Supervisor/Swarm) / MCP
4	2024.12+	模型自带推理能力 + 视觉操作	OpenAI o1/o3 / Anthropic Extended Thinking / DeepSeek-R1 / Computer Use / Devin
5	2026 形成中	Agent 网络化 + 工程化	A2A / AGNTCY / Anthropic Skills / Agent as Code / Ambient Agent

每代在 8 节点上的着力（速读版）

节点	0 代	1 代	2 代	3 代	4 代	5 代
① 上下文	+ CoT 提示	+ 工具描述 + ReAct 模板	+ 计划/ 反思	+ State Reducer	+ 视觉 + 推理	+ Skill 描述
② 推理	单次 CoT	Thought 显式	Plan/反思迭代	LLM 节点	模型自带深思考	跨 Agent 推理
③ 决策	无	字符串 Action	Planner 决定	条件边 / Command	模型给最终答案	A2A handoff
④ 工具	无	字符串解析	同 1 代	ToolNode + MCP	+ Computer Use	MCP server 网络
⑤ 返回	无	拼回 prompt	同 1 代	写回 State	+ 大返回截断	跨 Agent 返回
⑥ 状态	无	prompt 历史	+ 计划/ 反思记忆	Checkpoint + Store	长期工作记忆	git 化
⑦ 循环	无	for + max_steps	Plan 内迭代	recursion + interrupt	少而深	事件 + Cron
⑧ 输出	text	text	text	5 种 stream	+ 多模态	跨 Agent 输出

每代的优缺点对照

代	强在哪	弱在哪
0 CoT	简单、推理质量上	不能做事
1 ReAct	第一次能调工具	上下文爆炸、无规划、无中断、无状态、字符串解析脆
2 Plan-Execute	长任务稳、不乱跳	计划错全错
2 Reflexion	自我纠正、积累经验	需明确成败信号
2 ToT	跳出局部最优	token $\times N$ 、评估难
3 LangGraph	状态持久化、中断、时间旅行	学习曲线、需 Postgres
3 Multi-Agent	真团队协作	成本高、调度复杂
3 MCP	工具跨 LLM 复用	协议层不解决 Agent 编排
4 Reasoning Models	单步深思、步数少	单次延迟 5-30s、成本 5-10x
4 Computer Use	能做没 API 的事	截图理解仍弱、错误率高
4 长程自主 Agent	真自主、跨小时任务	沙箱 / 失败回收 / 成本
5 A2A	跨组织标准互通	早期、未稳
5 Skills	能力包装跨 Agent	概念新、生态早
5 Agent as Code	工程化、可回滚	需评估基础设施
5 Ambient Agent	主动工作	失控风险、成本管理

关键洞察 1：分层叠加而非替代（6 层模型）

每一代不替代上一代，而是**叠加在不同层**。这就是为什么 ReAct 没死。

层	来自哪几代	解决什么	代表
范式层（精神）	1 代 + 2 代	思考方式	ReAct / Plan-Execute / Reflexion / ToT
编排层（蓝图）	3 代	怎么把范式跑起来	LangGraph / Supervisor / Swarm
协议层（接口）	3 代 + 5 代	工具/Agent 怎么通信	MCP / A2A / Skills
模型层（大脑）	4 代	思考能力本身	Reasoning Models / Multimodal
运行层（手脚）	4 代	Agent 能操作什么	Sandbox / Computer Use / Browser Use
运维层（工程）	5 代	怎么上生产	Agent as Code / Eval / Observability

读法：一个真实的 2026 Agent 是这 6 层的组合，不是某一代的"纯产品"。

关键洞察 2：演进的 5 个核心规律

#	规律	含义
1	每代不替代上一代，叠加在不同层	ReAct 仍是原语
2	范式层趋稳 / 编排和模型层快速迭代	ReAct/Plan-Execute 6 年没变，编排层每年一代
3	协议层是 2025 后最大杠杆	MCP 把工具复用做成事实标准
4	运维层决定生产能力	评估和观测是分水岭
5	模型层吃掉范式层的复杂度	推理模型让"少而深"成为可能

2026 业界主流组合（按场景）

场景	推荐组合	用了哪几代
简单工具调用	Claude Sonnet 4.6 + Function Calling	1 代
多步客服	LangGraph Supervisor + Sonnet	1 + 3 代
复杂规划 / 数学 / 代码	LangGraph + Plan-Execute + o3/Opus 4.7	1+2+3+4 代
编程类（端到端）	Claude Code / Cursor / Devin	4 代
桌面自动化	Anthropic Computer Use + 沙箱	1+4 代
跨组织协作	LangGraph + MCP server	1+3+5 代
后台常驻	LangGraph + Cron + Ambient Agent	3+5 代
严肃生产	LangGraph + Langfuse + Promptfoo + Agent as Code	1+3+5 代

一句话总结

「**ReAct 没死，它降级成了"原语"** —— 第 4 代推理模型 + 第 3 代 LangGraph 编排 + 第 5 代 MCP 协议 才是 2026 严肃 Agent 的真实形态。

详见 [附录 L Agent 范式代际演进总表](#) —— 包含 5 代百科速查、6 层映射详细矩阵、custom_agent 代际定位。

§1.11 本章小结

#	核心结论
1	ReAct = Reasoning + Acting, 是范式不是框架 —— 让模型的"思考"和"行动"分两段显式输出
2	ReAct 的精髓是 Thought / Action / Observation 三段循环, 缺一不可
3	OpenAI Function Calling / Anthropic Tool Use 是 ReAct 的"原生协议"实现 —— 用 JSON 字段替代字符串解析
4	ReAct 的 6 大局限 (上下文爆 / 错误传播 / 无并行 / 无中断 / 无状态 / 黑盒) 完美映射基础流程的 6 个节点
5	AutoGPT / BabyAGI 是 ReAct 出圈的两个里程碑, 其局限催生了 LangGraph 这一代图式编排
6	5 代演进叠加模型: ReAct 是 1 代原语, 被叠加在 6 层 (范式/编排/协议/模型/运行/运维) 中而非被替代

§1.12 反模式速记

反模式	错在哪	正确做法
用 ReAct 字符串解析做新项目	解析脆弱 + 已被原生协议替代	用 OpenAI Function Calling 或 Anthropic Tool Use (§1.6)
给 ReAct 不加 max_iterations	死循环烧钱	必须设上限 (§3.5)
把 30 步任务硬塞 ReAct	上下文爆	改用 Plan-Execute 拆任务 (§1.5)
ReAct 跑失败不重试	一次错全错	加 Reflexion 总结失败 (§1.5)
期望 ReAct 多步任务跨会话续跑	ReAct 默认无状态	上 LangGraph Checkpoint (§5.3)
认为 ReAct 已被替代	ReAct 是原语，被叠加而非替代	理解 6 层叠加模型 (§1.10)
用 4 代推理模型还嵌套大量 1 代 ReAct 循环	推理模型把"多步思考"内化了，外层不必再循环很多次	推理模型场景减少 ReAct 循环步数 (§17.6)

§1.13 术语速查

术语	中文	含义
ReAct	推理-行动范式	Reasoning + Acting 同时输出的范式
Thought / Action / Observation	思考 / 行动 / 观察	ReAct 三段循环
Action Input	行动输入	工具参数
Final Answer	最终答案	ReAct 终止信号
Agent Scratchpad	智能体草稿板	累积的 Thought/Action/Observation 历史
Plan-and-Execute	计划-执行范式	先全盘规划再分步执行
Reflexion	反思范式	失败后总结记忆下次复用
Tree-of-Thoughts (ToT)	思维树	树形探索 + 评估剪枝
AutoGPT	自主 GPT	2023.03 引爆 ReAct 的开源项目
BabyAGI	婴儿通用智能	2023.04 多角色协作开源项目
Function Calling	函数调用	OpenAI 2023.06 的工具调用原生协议
Tool Use	工具使用	Anthropic 的工具调用协议
6 层叠加模型	6-layer stack	范式 / 编排 / 协议 / 模型 / 运行 / 运维
代际叠加	Generational layering	每代不替代上一代，叠加在不同层

§1.14 推荐下一章

下一章: [§2 LangChain：第一代框架的心智模型](#) —— ReAct 是范式, LangChain 是把它工程化的第一代尝试。

§2 LangChain：第一代框架的心智模型

§2.0 本章定位

项	内容
在基础流程中的位置	覆盖全部 8 个节点的"组件级"工具箱
与上下章的因果链	§1 ReAct 是范式（精神层），工程化它需要给每个节点提供工具箱 —— LangChain 是这个工具箱的第一代尝试。下一章 §3 用这个工具箱真的把 ReAct 跑起来
学完能做什么	(1) 复述 LangChain 六件套（LLM / Prompt / OutputParser / Memory / Retriever / Tool）对应到 8 节点；(2) 写 LCEL 管道表达式；(3) 区分 4 种 Memory 类型；(4) 用 <code>with_structured_output</code> 做现代结构化输出；(5) 加缓存 / 流式 / 重试 / 回退

§2.1 LangChain 解决了什么 — 2022 末的工程痛点

「在基础流程中的位置：节点 ① 到 ⑧ 全部 — 第一次给每个节点提供"组件级"工具箱。

场景引入

2022 年 10 月，开发者要做一个能查天气的智能体，得自己手写：

任务	手写代码量	问题
调 OpenAI / Cohere / Anthropic 接口	~50 行 / 厂商	三家 API 不一样
把 prompt 模板 + 变量拼起来	~30 行	没标准
ReAct 字符串解析	~80 行	边角 case 多
工具调用（HTTP / SQL / 文件）	~100 行 / 工具	工具间无统一接口
把检索结果（RAG）拼回 prompt	~50 行	多种检索器没标准
多轮对话历史管理	~40 行	截断 / 摘要策略要自己写

总计 350+ 行胶水代码，且每家厂商一份。

LangChain 的工程目标

Harrison Chase 2022 年 10 月开源 LangChain，目标是把这些“胶水代码”做成可复用的组件库：

节点	LangChain 组件	抽象掉的胶水
② 推理	LLM / ChatModel	各厂商 API 差异
① 上下文	PromptTemplate / ChatPromptTemplate	模板拼接
⑧ 输出	OutputParser / with_structured_output	输出解析
④ 工具	Tool / @tool	工具接口统一
⑥ 状态	Memory (4 种)	历史管理
① 增强	Retriever + VectorStore	RAG 检索
③ 决策	AgentExecutor + LCEL	流程编排

小结一行：LangChain 解决的核心问题是"用统一抽象消除厂商和工具的胶水代码"。

§2.2 历史脉络：v0.0 → v0.3 的四次重构

「在基础流程中的位置：理解版本演进，知道哪些 API 已废弃。

版本时间线

版本	时间	关键变化	节点级影响
v0.0	2022.10	初始发布，单包 <code>langchain</code>	节点 ① 到 ⑧ 第一次有了组件
v0.0.x	2023.01-2023.12	接口高频变化（每周大改）	老代码每月重写
v0.1	2024.01	拆包： <code>langchain-core</code> / <code>langchain</code> / <code>langchain-community</code>	减少依赖体积
v0.2	2024.05	LCEL 成熟、 <code>with_structured_output()</code> 推出、官方推荐迁 LangGraph	节点 ② ⑧ 现代化
v0.3	2024.09	老 <code>AgentExecutor</code> 正式退役，全面拥抱 LangGraph	节点 ③ ⑦ 推倒重来

版本变化的工程含义

时期	写法风格	现状
2022-2023	<code>LLMChain(llm=llm, prompt=prompt)</code>	已废弃，遇到老教程别学
2024.01 起	<code>prompt \ llm \ parser</code> （LCEL 管道）	当前推荐
2024.09 起	<code>create_react_agent(llm, tools)</code> 来自 <code>langgraph</code>	新项目首选（详见 §5.2）

升级到 v0.3 的破坏性改动清单

旧 (v0.1-v0.2)	新 (v0.3)
<code>from langchain.chat_models import ChatOpenAI</code>	<code>from langchain_openai import ChatOpenAI</code>
<code>LLMChain</code>	LCEL : <code>prompt \ llm \ parser</code>
<code>initialize_agent(...)</code>	<code>create_react_agent(...)</code>
<code>ConversationBufferMemory</code> 直接挂 chain	用 <code>RunnableWithMessageHistory</code>
<code>Tool(name=..., func=...)</code>	<code>@tool</code> 装饰器

小结一行：LangChain 三年四次大变身，新项目从 v0.3 起步、老项目按官方迁移指南升级（详见 §3.9）。

§2.3 核心抽象六件套对应到流程节点

「在基础流程中的位置：每个组件管哪个节点。

六件套总览

组件	节点	一句话职责
LLM / ChatModel	② 推理	调大模型
PromptTemplate / ChatPromptTemplate	① 组装	拼模板
OutputParser	⑧ 输出	解析 / 校验输出
Memory	⑥ 状态	多轮历史管理
Retriever + VectorStore	① 增强	RAG 检索
Tool / @tool	④ 工具	工具接口统一

每个组件的最小代码示例

CODE

```
# ② LLM
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# ① Prompt
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个助手"),
    ("user", "{question}"),
])

# ⑧ OutputParser
from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()

# LCEL
chain = prompt | llm | parser
result = chain.invoke({"question": "你好"})
```

LLM vs ChatModel 的区分

维度	LLM	ChatModel
接口	字符串 → 字符串	消息列表 → 消息
现代用法	已淘汰	推荐 （与现代 API 对齐）
典型类	OpenAI	ChatOpenAI

新项目只用 ChatModel。

小结一行：六件套是 LangChain 的"乐高积木"，LCEL 是把它们粘起来的"胶水"。

§2.4 LCEL 表达式语言：管道符的设计哲学

在基础流程中的位置：把节点 ①②⑧ 串起来的胶水。

LCEL（LangChain Expression Language）是什么

LangChain 表达式语言（LangChain Expression Language，简称 LCEL）—— 用 Python 的管道符把组件串起来：

```
chain = prompt | llm | parser
```

CODE

读作"把 prompt 输出喂给 llm，再喂给 parser"。

为什么用管道符

设计目标	LCEL 怎么实现
流式 (Streaming)	每个组件都支持 <code>astream()</code> ，全链路自动流式
异步 (Async)	每个组件都支持 <code>ainvoke()</code> ，全链路自动异步
批处理 (Batch)	每个组件都支持 <code>batch()</code> ，全链路自动并行批处理
可观测 (LangSmith 集成)	每个组件自动产生 trace
可组合	链可以再当组件用： <code>(prompt \llm) \ (another_prompt \llm)</code>

LCEL 与传统写法对照 (Mermaid #5)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph Old["Old LLMChain"]
        A1[Question] --> B1["LLMChain  
llm + prompt"]
        B1 --> C1[Answer]
    end

    subgraph New["New LCEL"]
        A2[Question] --> P[prompt]
        P --> L[llm]
        L --> Pa[parser]
        Pa --> C2[Answer]
    end

    style B1 fill:#fecaca,stroke:#ef4444
    style P fill:#d1fae5
    style L fill:#dbeafe
    style Pa fill:#fef3c7
```

LCEL 提供的辅助原语

原语	作用
<code>RunnablePassthrough()</code>	透传输入（占位符）
<code>RunnableParallel({...})</code>	并行多个分支
<code>RunnableLambda(func)</code>	把任意 Python 函数变成 Runnable
<code>RunnableBranch([(condition, runnable)])</code>	条件分支
<code>chain.with_retry(...)</code>	加重试
<code>chain.with_fallbacks([...])</code>	加回退

一个稍复杂的 LCEL 示例

CODE

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

retrieve_chain = (
    RunnableParallel({
        "context": retriever | format_docs,      # RAG 检索 + 格式化
        "question": RunnablePassthrough(),      # 透传问题
    })
    | prompt
    | llm
    | parser
)
```

读作：先并行检索文档 + 透传问题，组装到 prompt，过 llm，过 parser。

小结一行：LCEL 把“组件 + 管道符”当一等公民，让流式 / 异步 / 批处理 / 观测自动获得。

包	内容	安装大小	何时装
<code>langchain-core</code>	核心抽象 (Runnable / Prompt / OutputParser 接口)	很小	总是装
<code>langchain</code>	主流功能 (chains / agents 老接口 / 部分集成)	中	大多数项目
<code>langchain-community</code>	大量第三方集成 (向量库 / 文档加载器 / API 工具)	大	用到具体集成时
<code>langchain-openai</code> / <code>langchain-anthropic</code> / ...	厂商专属	各小	用谁装谁

CODE

```
pip install langchain langchain-openai

pip install langchain langchain-openai langchain-anthropic langchain-community
```

为什么拆包

痛点（v0.0 单包时代）	v0.1 拆包后
装一个 langchain 拖 200+ 依赖	core + 主包总共 < 30 个依赖
一家厂商接口变化要重发整个 langchain	各 partner 包独立发版
社区集成质量参差混在主包	community 单独，主包更稳

小结一行：v0.1 起的拆包架构让 LangChain 从"巨石"变"瑞士军刀" —— 按需选包。

§2.6 Runnable 接口：LCEL 背后的统一抽象

「在基础流程中的位置：所有节点的组件背后的统一契约。

Runnable 是什么

`Runnable` 是 `langchain-core` 里的抽象基类，所有 `LangChain` 组件都实现这个接口。它定义了一组标准方法：

```
class Runnable:
    def invoke(self, input): ...           # [?][?][?][?]
    def ainvoke(self, input): ...          # [?][?][?][?]
    def batch(self, inputs): ...           # [?][?][?][?]
    def abatch(self, inputs): ...          # [?][?][?][?]
    def stream(self, input): ...           # [?][?][?][?]
    def astream(self, input): ...          # [?][?][?][?]
    def __or__(self, other): ...           # [?][?][?]
```

CODE

为什么 Runnable 重要

不理解 Runnable 不算理解 LCEL：

用法	背后机制
<code>prompt llm</code>	<code>prompt.__or__(llm)</code> 返回新的 <code>RunnableSequence</code>
<code>chain.invoke(x)</code>	顺序调用每个组件的 <code>.invoke()</code>
<code>chain.astream(x)</code>	链路自动 token 级流式
<code>chain.batch([x, y, z])</code>	并行处理
<code>chain.with_config(...)</code>	给整个链加配置

自定义 Runnable 的两种方式

CODE

```
from langchain_core.runnables import RunnableLambda
from langchain_core.runnables import RunnableLambda

def my_func(x: dict) -> str:
    return x["question"].upper()

my_runnable = RunnableLambda(my_func)
chain = my_runnable | llm

from langchain_core.runnables import Runnable
from langchain_core.runnables import Runnable

class MyRunnable(Runnable):
    def invoke(self, input, config=None):
        return input.upper()
```

小结一行：Runnable 是 LangChain 的"USB 接口" —— 任何符合这个接口的东西都能插进 LCEL 管道。

§2.7 记忆模块四种深入对比

「在基础流程中的位置：节点 ⑥ 状态管理在 LangChain 时代的实现。

四种 Memory 的工程定位

类型	中文	节点 ⑥ 怎么存	节点 ① 怎么注入
<code>ConversationBufferMemory</code>	缓冲记忆	完整存所有历史	整段塞进 prompt
<code>ConversationSummaryMemory</code>	摘要记忆	用 LLM 总结历史成一段	摘要注入 prompt
<code>ConversationKGMemory</code>	知识图记忆	抽取实体 + 关系成知识图	相关三元组注入
<code>VectorStoreRetrieverMemory</code>	向量检索记忆	历史片段存向量库	与当前问题相关的检索注入

详细对比

维度	Buffer	Summary	KG	VectorStore
存储成本	O(N) 历史长度	O(1) 摘要	O(实体 × 关系)	O(N) 向量
节点 ① token 消耗	高（全塞）	低（只塞摘要）	中（塞相关三元组）	中（塞 top-k）
节点 ⑥ 写入成本	0（直接 append）	高（每轮 LLM 调用）	高（NER 抽取）	中（embedding）
失真风险	0	高（摘要丢细节）	中（NER 错）	低（语义相似匹配）
适用场景	短对话（< 10 轮）	长对话不在乎细节	实体关系密集（医疗 / 法律）	长对话需保留细节（推荐）

选哪种 Memory 的决策树

CODE

```
from langchain.memory import ConversationBufferMemory, ConversationSummaryMemory, ConversationKGMemory, VectorStoreRetrieverMemory
```

现代写法：RunnableWithMessageHistory

v0.2 起官方推荐用 RunnableWithMessageHistory 取代直接挂 Memory：

CODE

```

from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

store = {}

def get_session_history(session_id: str) -> ChatMessageHistory:
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="question",
    history_messages_key="history",
    [?]

result = chain_with_history.invoke(
    {"question": "..."},
    config={"configurable": {"session_id": "user-123"}},
    [?]

```

Memory 的根本缺陷

不管哪种 Memory，都是**会话级**（同一 session 内）。跨会话失忆。这就是为什么需要 LangGraph 的 Store API (§6.5)。

小结一行：LangChain Memory 解决会话内的"上下文工程"，但跨会话失忆 —— 这洞要等 LangGraph Store API 来补。

§2.8 文档加载器与文本分割器

「在基础流程中的位置：节点 ① 上下文增强（RAG）的入口。

Document Loaders 加载文档

LangChain 内置 100+ 种文档加载器（在 [langchain-community](#)）：

类别	代表加载器
文本	TextLoader / UnstructuredFileLoader
PDF	PyPDFLoader / PDFPlumberLoader
网页	WebBaseLoader / RecursiveUrlLoader
Office	Docx2txtLoader / UnstructuredExcelLoader
数据库	SQLDatabaseLoader
API	NotionDBLoader / ConfluenceLoader / GitHubIssuesLoader

Text Splitters 切片

切分策略	用法	适用
RecursiveCharacterTextSplitter	按字符递归切（先大段后小段）	默认推荐，通用
MarkdownHeaderTextSplitter	按 Markdown 标题切	文档型内容
CharacterTextSplitter	按字符简单切	老接口
TokenTextSplitter	按 token 切	严格控制 token 数
SemanticChunker	按语义边界切（用 embedding）	高精度

一个完整 RAG 数据准备示例

CODE

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

docs = PyPDFLoader("manual.pdf").load() # 1 1
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs) # 2 2
db = FAISS.from_documents(chunks, OpenAIEmbeddings()) # 3 3 3
retriever = db.as_retriever(search_kwargs={"k": 4}) # 4 4 4
```

小结一行：Document Loader + Text Splitter + Embedding + VectorStore 是 LangChain 的"RAG 四件套"，各组件之间通过 Document 对象统一。

§2.9 输出解析器三类与现代 with_structured_output

「在基础流程中的位置：节点 ⑧ 输出格式化。

三类 OutputParser

类型	代表类	用法
字符串型	<code>StrOutputParser</code>	把 ChatModel 输出转成字符串
结构化型（基于 Pydantic）	<code>PydanticOutputParser</code>	强制输出符合 Pydantic 模型的 JSON
自我修复型	<code>RetryOutputParser</code> / <code>OutputFixingParser</code>	解析失败后让 LLM 重试或修正

老写法：手动用 PydanticOutputParser

CODE

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel

class WeatherInfo(BaseModel):
    city: str
    temperature: float
    condition: str

parser = PydanticOutputParser(pydantic_object=WeatherInfo)
prompt = ChatPromptTemplate.from_messages([
    ("system", "[?][?][?][?][?][?]format_instructions[?][?]),
    ("user", "{input}"),
]).partial(format_instructions=parser.get_format_instructions())

chain = prompt | llm | parser
result = chain.invoke({"input": "[?][?][?][?][?][?][?][?][?][?]})
# WeatherInfo(city='[?][?][?][?][?]temperature=25.0, condition='[?][?])
```

问题：要手动管 `format_instructions`，复杂且易错。

现代写法：with_structured_output

v0.2 起官方推 `with_structured_output()`：

CODE

```
structured_llm = llm.with_structured_output(WeatherInfo)
result = structured_llm.invoke("???temperature=25.0, condition='?')
# WeatherInfo(city='?temperature=25.0, condition='?')
```

好处：

维度	老 PydanticOutputParser	现代 with_structured_output
实现机制	在 prompt 里加 JSON Schema 提示，靠模型遵守	用厂商原生的结构化输出 API（OpenAI Structured Outputs / Anthropic Tool Use）
成功率	中（取决于模型听话）	高（API 强制）
代码量	多（模板 + 解析器）	少（一行）
推荐	老项目维护	新项目首选

小结一行：节点 ⑧ 输出格式化从"prompt 哄模型"演化到"API 强约束"，新项目用 `with_structured_output`。

§2.10 缓存：LLM cache + 嵌入 cache

「在基础流程中的位置：节点 ② / 节点 ① 增强的成本优化。

为什么需要缓存

场景	没缓存	有缓存
同一问题反复问	每次调 LLM 花钱	第二次起从缓存返回
大量文档 embedding	每次重算	增量算
开发调试反复跑	烧钱	几乎免费

LLM 调用缓存

CODE

```
from langchain.cache import InMemoryCache, SQLiteCache
from langchain.globals import set_llm_cache

# 内存缓存
set_llm_cache(InMemoryCache())

# 数据库缓存
set_llm_cache(SQLiteCache(database_path=".llm_cache.db"))
```

设完后所有 LLM 调用自动走缓存。命中条件：**完全相同的输入**（包括 temperature 等参数）。

嵌入缓存（CacheBackedEmbeddings）

CODE

```
from langchain.embeddings import CacheBackedEmbeddings
from langchain.storage import LocalFileStore

store = LocalFileStore("./embedding_cache")
underlying = OpenAIEmbeddings()
cached_embed = CacheBackedEmbeddings.from_bytes_store(
    underlying, store, namespace=underlying.model
)

embedding = OpenAIEmbeddings
vec1 = cached_embed.embed_query("hello")
embedding = OpenAIEmbeddings
vec2 = cached_embed.embed_query("hello")
```

生产级缓存

后端	适用
InMemoryCache	单进程开发
SQLiteCache	单机持久化
RedisCache	多实例共享
RedisSemanticCache	语义相似匹配（不必完全一致）
MongoDBCache	已有 MongoDB
PostgresCache	已有 Postgres

小结一行：缓存是节点 ② / 节点 ① 的成本杠杆，开发用内存、生产用 Redis 或语义缓存。

§2.11 流式：astream / stream

「在基础流程中的位置：节点 ⑧ 输出格式化的体验关键。

为什么要流式

LLM 生成 200 字的回答可能要 3-5 秒。**流式**让用户每生成一个 token 就能看到，体感从"等 5 秒"变"看打字"。

LCEL 全链路流式

CODE

```
chain = prompt | llm | StrOutputParser()

???
for chunk in chain.stream({"question": "???"}):
    print(chunk, end="", flush=True)

???
async for chunk in chain.astream({"question": "???"}):
    print(chunk, end="", flush=True)
```

流式背后的机制

阶段	流式怎么发生
llm 输出 token	厂商 API 用 SSE / WebSocket 流式返回
LCEL 透传	每个 Runnable 实现 stream() 接口
OutputParser 转换	增量解析 (StrOutputParser 直接透传)
前端渲染	浏览器 SSE 或 WebSocket 接收

流式的限制

场景	能流吗	说明
纯文本输出	是	默认
结构化输出 (with_structured_output)	部分	完整 JSON 才能解析，不能 token 级流
Tool Calling Agent	复杂	tool_use 块 + 文本 token 混杂（详见 §3.x）
多步链	是	中间步骤完成才往下传，最末步流

小结一行：流式让 chat-style 智能体的体感大幅提升 —— 但结构化输出和工具调用流式有坑 (§3.x 会讲)。

§2.12 重试与回退：with_retry / with_fallbacks

在基础流程中的位置：横切节点 ② / ④ 的稳定性手段。

with_retry：失败重试

CODE

```
chain_with_retry = chain.with_retry(  
    retry_if_exception_type=(ConnectionError, TimeoutError),  
    wait_exponential_jitter=True,  
    stop_after_attempt=3,
```

?

参数	作用
<code>retry_if_exception_type</code>	哪些异常触发重试
<code>wait_exponential_jitter</code>	指数退避（避免雷击）
<code>stop_after_attempt</code>	最多重试几次

with_fallbacks：失败回退到备选

CODE

```
primary = ChatOpenAI(model="gpt-4o")
fallback = ChatAnthropic(model="claude-sonnet-4-6")

chain = (prompt | primary | parser).with_fallbacks(
    [prompt | fallback | parser]
)?
```

主链失败（如 OpenAI 限流）就跑备选链（Anthropic）。

何时用什么

失败类型	with_retry	with_fallbacks
临时网络错误	是	否
限流 / 配额	是（带退避）	更好（换厂商）
模型输出格式错	用 OutputFixingParser	否
厂商整体宕机	否	是

小结一行：重试管"瞬时错"，回退管"持续错"，组合用更稳。

§2.13 bind_tools 现代工具绑定

「在基础流程中的位置：节点 ② 推理 + ④ 工具调用的现代衔接。

老写法：Tool Calling Agent + AgentExecutor

CODE

```
llm_with_tools = llm.bind_tools(tools)
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools)
result = executor.invoke({"input": "..."})
```

现代写法：bind_tools + 自己跑循环

CODE

```
llm_with_tools = llm.bind_tools(tools)
response = llm_with_tools.invoke([HumanMessage(content="...")])
# response.tool_calls 包含调用信息
```

`bind_tools()` 把工具列表绑定到 LLM，模型输出会带 `tool_calls` 字段。不再需要 **AgentExecutor 黑盒** —— 用 LangGraph 的图式编排（详见 §5.1）显式控制循环。

bind_tools 的工作机制

厂商	bind_tools 背后调用
OpenAI	API 的 <code>tools</code> 参数（Function Calling）
Anthropic	API 的 <code>tools</code> 参数（Tool Use 块）
Google	<code>function_declarations</code> 字段

LangChain 自动适配三家协议（详见附录 K）。

小结一行：`bind_tools` 是新版工具绑定的标准 API —— 它替代了老的 `Tool` 类 + `AgentExecutor` 组合。

§2.14 LangSmith 观测平台

「在基础流程中的位置：横切所有节点的运行追踪。

LangSmith 是什么

LangChain 团队商业观测平台，与 **LangChain 框架解耦**（不用 LangChain 也能用 LangSmith）。

功能	说明
追踪（Trace）	每次 LCEL 调用的完整链路（每个 Runnable 输入输出 / token / 耗时）
评估（Eval）	在 LangSmith 数据集上跑回归测试
提示词管理	版本化 prompt，可 A/B 测试
注解 (Annotation)	标注 trace 给后续训练用
监控 (Monitoring)	Token 消耗 / 错误率 / 延迟趋势

启用 LangSmith

CODE

```
export LANGSMITH_API_KEY=...
export LANGSMITH_TRACING=true
export LANGSMITH_PROJECT=my-agent
```

启用后所有 LCEL 调用自动产生 trace，无需改代码。

LangSmith 在基础流程的作用

节点	LangSmith 看什么
① 组装	prompt 实际拼出来什么样
② 推理	模型 token 输入 / 输出 / 耗时
③ 决策	tool_calls 字段
④ 工具	每个工具的输入输出
⑦ 循环	共跑了几轮

小结一行：LangSmith 是 LangChain 生态的"观测中枢"，但不绑死 —— 严肃团队都开。

§2.15 业界目前怎么用 LangChain

「在基础流程中的位置：业界对 LangChain 的真实评估。

三类主流玩法

玩法	比例（粗估）	代表团队
完全用 LangChain（含 AgentExecutor）	少（< 10%）	老项目、教程
用 LangChain 当组件库 + LangGraph 编排	主流（~ 60%）	Klarna / Replit / LinkedIn
不用 LangChain，直接调 SDK + 手写胶水	增长中（~ 30%）	Anthropic 自家 / Cursor

各部分的去留

LangChain 组件	现状	去留
LCEL（管道符）	稳定	留
<code>with_structured_output</code>	现代	留
<code>bind_tools</code>	现代	留
Document Loaders / Text Splitters	仍是事实标准	留
Memory 模块	部分被 LangGraph State 替代	短对话留，长对话迁
<code>AgentExecutor</code>	官方建议迁出	走
<code>LLMChain</code> / <code>ConversationChain</code> 等老链类	已废弃	走
LangSmith 集成	与框架解耦	留（独立用也行）

小结一行：LangChain 已从"Agent 框架的代名词"退化为"组件胶水库"，新项目的默认是组件库 + LangGraph 编排。

§2.16 本章小结

#	核心结论
1	LangChain 解决了 2022 末的"胶水代码"问题 —— 用统一抽象消除厂商和工具差异
2	三年四次大版本（v0.0 → v0.3），新项目从 v0.3 起步、不学老 LLMChain / AgentExecutor
3	六件套对应基础流程：LLM（②） / Prompt（①） / OutputParser（⑧） / Memory（⑥） / Retriever（①） / Tool（④）
4	LCEL 用管道符把 Runnable 串起来，自动获得流式 / 异步 / 批处理 / 观测
5	现代写法： <code>with_structured_output</code> + <code>bind_tools</code> + LangGraph 编排 —— 这套组合替代老 AgentExecutor
6	业界主流是"组件库 + LangGraph"，纯 LangChain 智能体是少数派

§2.17 反模式速记

反模式	错在哪	正确做法
学 2023 教程的 <code>LLMChain</code> 写法	已废弃	用 LCEL 管道 (§2.4)
输出解析手写 <code>PydanticOutputParser</code>	易错 + 不优雅	用 <code>with_structured_output</code> (§2.9)
用 <code>AgentExecutor</code> 跑新项目	黑盒 + 无 checkpoint	用 LangGraph <code>create_react_agent</code> (§5.2)
不开 LangSmith / 不开缓存	看不见钱 + 烧钱	必开 (§2.10 / §2.14)
长对话用 <code>ConversationBufferMemory</code>	token 爆	用 SummaryMemory 或 VectorStoreMemory (§2.7)

§2.18 术语速查

术语	中文	含义
LCEL (LangChain Expression Language)	表达式语言	用 <code>[]</code> 管道符串联 Runnable
Runnable	可运行对象	LangChain 所有组件的基类
<code>bind_tools</code>	绑定工具	给 LLM 绑工具列表的现代 API
<code>with_structured_output</code>	结构化输出	强制模型输出符合 Pydantic 模型的 JSON
<code>with_retry</code> / <code>with_fallbacks</code>	重试 / 回退	LCEL 链上加稳定性
Memory	记忆	节点 ⑥ 的 LangChain 实现 (4 种)
Retriever	检索器	RAG 的检索抽象
VectorStore	向量库	embedding 存储抽象
Document Loader	文档加载器	100+ 种文件 / API 的统一加载
Text Splitter	文本分割器	长文档切片
LangSmith	LangSmith 观测平台	LangChain 团队商业观测产品
AgentExecutor	智能体执行器	老的 ReAct 循环黑盒 (已废弃)

§2.19 推荐下一章

下一章: [§3 用 LangChain 实现 ReAct \(流程跑通实战\)](#) —— 把 §2 的组件库真的用起来, 用 LangChain 写个完整的 ReAct 智能体并暴露其硬伤。

§3 用 LangChain 实现 ReAct（流程跑通实战）

§3.0 本章定位

项	内容
在基础流程中的位置	把整条流程用 LangChain 实现一遍，逐节点跑通
与上下章的因果链	§2 知道了 LangChain 是什么；本章用它把 §1 的 ReAct 流程真正跑起来，并暴露每个节点的硬伤，引出 §4 LangGraph 的必要性
学完能做什么	(1) 写出可运行的 LangChain ReAct 智能体；(2) 调试中间产物；(3) 列出 4 大节点级硬伤；(4) 知道从老 AgentExecutor 迁移到 LangGraph 的官方路径

§3.1 老接口（已废弃）：AgentExecutor + initialize_agent

「在基础流程中的位置：节点 ③ 决策路由 + ⑦ 循环控制的老黑盒。」

历史背景

2022.10 - 2024.05 期间，LangChain 智能体的"标准写法"是：

CODE

```

[?]

```

```

from langchain.agents import initialize_agent, AgentType
from langchain_openai import OpenAI

```

```

agent = initialize_agent(
    tools=[search_tool, calculator_tool],
    llm=OpenAI(temperature=0),
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True,

```

```

[?]

```

```

result = agent.invoke("[?]")

```

AgentExecutor 黑盒结构 (Mermaid #6)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    subgraph AE ["AgentExecutor ([?])"]
        Prompt["Prompt ([?])"]
        ReAct["[?] ReAct ([?])"]
        LLM["LLM ([?])"]
        Parser["AgentOutputParser ([?])"]
        Tools["Tools ([?])"]
        Loop["while not done: ([?])"]
        FinalAnswer["Final Answer ([?])"]
        end

        Q["Q ([?])"] --> AE
        AE --> R["R ([?])"]

        Prompt --> LLM
        LLM --> Parser
        Parser --> Action["Action: ([?])"]
        Action --> Tools
        Tools --> Loop
        Loop --> Prompt
        Loop --> FinalAnswer
        FinalAnswer --> R

        style AE fill:#fecaca,stroke:#ef4444,stroke-width:3px
        style Parser fill:#fef3c7,stroke:#f59e0b

```

黑盒在哪些节点出问题

节点	黑盒症状	后果
② 推理	用户看不见模型的中间 Thought（除非开 verbose）	调试靠人肉读日志
③ 决策	<code>AgentOutputParser</code> 用正则解析 <code>Action: ...</code> ，遇到模型输出格式不规范就报 <code>OutputParserException</code>	任务中断
⑥ 状态	中间产物（agent_scratchpad）只在 prompt 里，外部看不到	不能持久化
⑦ 循环	退出条件靠模型输出 <code>Final Answer:</code> 或 <code>max_iterations</code> 硬限	难精细控制

AgentType 的几种"个性"

老 `initialize_agent` 支持多种 Agent 类型：

AgentType	含义	现状
<code>ZERO_SHOT_REACT_DESCRIPTION</code>	经典 ReAct（字符串解析）	最常见，已废弃
<code>CHAT_ZERO_SHOT_REACT_DESCRIPTION</code>	ChatModel 版的 ReAct	已废弃
<code>OPENAI_FUNCTIONS</code>	用 OpenAI Function Calling	被 Tool Calling Agent 取代
<code>OPENAI_MULTI_FUNCTIONS</code>	多函数并行版	被 Tool Calling Agent 取代
<code>STRUCTURED_CHAT_ZERO_SHOT_REACT_DESCRIPTION</code>	结构化输入的 ReAct	已废弃

v0.3 起 `initialize_agent` 已发出 deprecation 警告。

小结一行：老 `AgentExecutor` 把节点 ③⑦ 黑盒化、字符串解析脆弱、不支持 checkpoint
—— 这就是为什么需要 LangGraph。

§3.2 现代做法：Tool Calling Agent + LCEL

「在基础流程中的位置：节点 ③ 用结构化字段替代字符串解析。

现代写法的演化

v0.2 起，LangChain 推荐：

CODE

```

from langchain_openai import ChatOpenAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o", temperature=0)

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个助手，请根据提供的信息回答问题。"),
    ("placeholder", "{chat_history}"),
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({"input": "你好，请问有什么可以帮助你的吗？"})

```

现代做法的全流程图（Mermaid #7）

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    Q["Q: 用户输入"] --> P["P: ChatPromptTemplate"]
    P --> L["L: llm.bind_tools(tools)"]
    L --> J["J: {node 3} tool_calls?"]
    J --> F["F: Final Output"]
    J --> T["T: ToolNode"]
    T --> M["M: Add ToolMessage"]
    M --> P

    style L fill:#dbeafe
    style J fill:#fce7f3
    style T fill:#d1fae5

```

与老接口的关键差异

维度	老 ZERO_SHOT_REACT	现代 create_tool_calling_agent
工具描述位置	prompt 文本 ("Tool 1: ...")	API 字段 <code>tools</code> (结构化)
模型输出	文本 <code>Action: name</code>	JSON <code>tool_calls</code> 字段
解析	正则 (脆弱)	字段访问 (稳)
多工具并发	不支持	支持
Pydantic 工具参数	弱	强 (schema 严格)

`create_tool_calling_agent` 的内部机制

它本质上是一个 LCEL 链:

CODE

```
①②③④⑤
agent = (
    ⑥⑦⑧⑨⑩
    "input": lambda x: x["input"],
    "agent_scratchpad": lambda x: format_to_tool_messages(x["intermediate_steps"]),
    ⑪⑫⑬⑭⑮
    | prompt
    | llm.bind_tools(tools)
    | ToolsAgentOutputParser() # ⑯⑰⑱tool_calls ⑲⑳㉑㉒㉓㉔
    ⑵
```

小结一行: 现代做法用结构化 `tool_calls` 字段替代 ReAct 字符串, 节点 ③ 决策路由从"正则猜"变成"字段读"。

§3.3 完整代码：50 行从 prompt 到 agent 到调用

「在基础流程中的位置：把节点 ① 到 ⑧ 全跑通。」

" "LangChain ????ReAct ??????????????????????"

? ? ? ? ? ?

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.globals import set_llm_cache
from langchain.cache import SQLiteCache
```

[illegible]

```
set_llm_cache(SQLiteCache(database_path=".llm_cache.db"))
```

[illegible]

@tool

```
def get_weather(city: str) -> str:
```

city

[illegible]

@tool

```
def calculator(expression: str) -> str:
```

expression Python

```
try:
```

```
return str(eval(expression, {"__builtins__": {}}, {}))
```

```
except Exception as e:
```

```
return f"?????e??"
```

```
tools = [get_weather, calculator]
```

[illegible]

```
llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

[illegible]

```
prompt = ChatPromptTemplate.from_messages([
```

```
("system", "????????????????????")
```

[illegible]

```
("placeholder", "{chat history}").
```

```
("user", "{input}"),
```

```
("placeholder", "{agent_scratchpad}"), # ??????????
```

22

? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?

```
agent = create_tool_calling_agent(llm, tools, prompt)
```

```
executor = AgentExecutor(
```

```
agent=agent,
```

```
tools=tools,
```

```
verbose=True,
```

? ? ? ?

```
max iterations=8,
```

[?] [?] [?] [?] [?] [?] [?] [?] [?]

```

return_intermediate_steps=True, # [?]
[?]

[?]
result = executor.invoke({
    "input": "[?]",
    "chat_history": [],
    [?]

[?]
print("[?]result['output'])
print("[?]len(result['intermediate_steps']))

```

期望的执行轨迹

CODE

```

> Entering new AgentExecutor chain...
[Tool call] get_weather(city='[?]')
Tool result: [?]
[Tool call] get_weather(city='[?]')
Tool result: [?]
[Tool call] calculator(expression='25 - 22')
Tool result: 3
[Final response] [?]
> Finished chain.

```

50 行代码各部分对应基础流程

行	干啥	节点
9-10	开缓存	② 优化
13-26	定义工具	④
29	创建 LLM	②
32-37	提示词模板	①
40	包装成 agent	②③
41-46	执行器	③⑦
49-52	调用	跑流程
55-56	输出	⑧

小结一行：50 行就能跑出可工作的 LangChain 智能体 —— 但每个节点都有问题，下面 §3.5 拆解。

§3.4 节点 ② 输出可见性：怎么看模型的"思考"过程

「在基础流程中的位置：节点 ② 推理的可观测性。

verbose=True 的输出格式

CODE

```
> Entering new AgentExecutor chain...

Invoking: `get_weather` with `{'city': '北京'}`
北京天气

Invoking: `get_weather` with `{'city': '上海'}`
上海天气

Invoking: `calculator` with `{'expression': '25 - 22'}`
7

> Finished chain.
```

拿到中间步骤的两种方式

CODE

```
from langchain.agents import AgentExecutor, Tool
from langchain.tools import Tool

return_intermediate_steps=True
executor = AgentExecutor(agent=agent, tools=tools, return_intermediate_steps=True)
result = executor.invoke({"input": "..."})
for action, observation in result["intermediate_steps"]:
    print(f"[Action] {action.tool}({action.tool_input})")
    print(f"[Obs] {observation}")

from langchain.agents import AgentExecutor, Tool
from langchain.tools import Tool

import os
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "..."
executor.invoke("...", trace=True, callbacks=)
```

流式查看中间过程

CODE

```
# astream_events [?] [?] [?] [?] [?] [?] [?] [?]
async for event in executor.astream_events({"input": "..."}, version="v2"):
    if event["event"] == "on_tool_start":
        print(f"[Tool start] {event['name']}({event['data']['input']})")
    elif event["event"] == "on_tool_end":
        print(f"[Tool end] {event['data']['output']}")
    elif event["event"] == "on_chat_model_stream":
        print(event["data"]["chunk"].content, end="", flush=True)
```

可见性局限

可见	不可见
工具的输入输出	模型为什么选这个工具（节点 ② 内部 reasoning）
最终回答	模型有几次"差点选错但最后选对"
共调几次工具	模型是否考虑了 Final Answer 但还是继续了

要看节点 ② 的"思考过程", 要么模型本身支持 reasoning 输出（如 Claude Extended Thinking、o1），要么用"Thought + Action + Observation"格式（即回到老 ReAct prompt）。

小结一行：现代 Tool Calling Agent 输出结构化但缺"思考过程"——要原汁原味的 ReAct Thought 字段，得改 prompt 强制要求。

§3.5 LangChain 实现 ReAct 的硬伤逐节点拆解

「在基础流程中的位置：本节是引出 LangGraph 的关键。

硬伤 1：节点 ⑥ 状态污染

CODE

```
def chat_history():
    result1 = executor.invoke({"input": "chat_history": []})
    result2 = executor.invoke({"input": "chat_history": []}) # chat_history
```

问题：两次 invoke 之间状态丢失。要持续对话得自己管 chat_history，且只是会话级 —— 进程重启或换实例就忘。

硬伤 2：节点 ③ AgentExecutor 黑盒难调试

问题	表现
模型选错工具	黑盒里发生，外面只看到结果错
工具被调了不该调的	没有"前置审批"挂载点
想动态加工具	必须重建 executor
想换循环逻辑	改不动 AgentExecutor 内部

硬伤 3：节点 ⑦ 循环只能粗放管

max_iterations=8 是唯一控制。没有：- 中途暂停问人类（HITL） - 卡死时优雅退出（细粒度信号） - 跳到指定步骤（time travel） - 失败时从某 checkpoint 重试

硬伤 4：节点 ⑤ 错误工具调用污染会话

CODE

```
@tool
def buggy_search(q: str) -> str:
    raise ValueError("API quota exceeded")

buggy_search, AgentExecutor.stringify, scratchpad
API quota exceeded
```

CODE

五大硬伤汇总

小结一行：LangChain 实现 ReAct 在每个节点都有可观察的硬伤，这五大硬伤就是 LangGraph 的设计动机。

「在基础流程中的位置：横切节点 ② / ④ / ⑤ 的观测手段。

Callbacks 是什么

不开 LangSmith 时，LangChain 提供本地的回调机制 —— 在每个组件运行时触发钩子。

内置事件

事件	何时触发	拿到什么
<code>on_chat_model_start</code>	LLM 调用开始	prompt
<code>on_chat_model_end</code>	LLM 调用结束	response + 用量
<code>on_chat_model_stream</code>	LLM 流式 token	chunk
<code>on_tool_start</code>	工具调用开始	input
<code>on_tool_end</code>	工具调用结束	output
<code>on_chain_start</code> / <code>on_chain_end</code>	链开始 / 结束	输入输出
<code>on_retriever_start</code> / <code>on_retriever_end</code>	检索器	query / docs

自定义 Callback Handler

CODE

```
from langchain_core.callbacks import BaseCallbackHandler

class TokenCountHandler(BaseCallbackHandler):
    def __init__(self):
        self.total_tokens = 0

    def on_chat_model_end(self, response, **kwargs):
        usage = response.llm_output.get("token_usage", {})
        self.total_tokens += usage.get("total_tokens", 0)

handler = TokenCountHandler()
result = executor.invoke({"input": "..."}, config={"callbacks": [handler]})
print(f"???handler.total_tokens??tokens")
```

Callbacks vs LangSmith 的选型

维度	自定义 Callbacks	LangSmith
部署成本	0	注册 + 配 API key
数据归属	本地	上传到 LangChain 云
历史回溯	自己存	内置
可视化	自己做	开箱即用
评估 / 注解	否	有
适合	简单观测 / 不能上云的环境	严肃团队

小结一行：Callbacks 是 LangSmith 的"低配版"，能不能上云决定选哪个。

三类常见错误

OutputParserException 处理

```
from langchain.agents import AgentExecutor

executor = AgentExecutor(
    agent=agent,
    tools=tools,
    handle_parsing_errors=True, # ???
    # handle_parsing_errors="???Action: <name>\nAction Input: <json>"
    ?
```

第 178 页

工具错误处理

CODE

```
@tool
def safe_search(q: str) -> str:
    """
    try:
        return real_search(q)
    except Exception as e:
        return f"[?] {str(e)[:200]}" # [?]

"""
```

自我修复解析器

CODE

```
from langchain.output_parsers import OutputFixingParser

parser = OutputFixingParser.from_llm(
    parser=PydanticOutputParser(pydantic_object=MySchema),
    llm=llm,
)

chain = prompt | llm | parser
```

解析失败时，OutputFixingParser 自动让 LLM 重写。

重试与回退（参考 §2.12）

CODE

```
chain_robust = chain.with_retry(
    retry_if_exception_type=(RateLimitError, APITimeoutError),
    stop_after_attempt=3,
).with_fallbacks([backup_chain])
```

小结一行：节点 ③⑤ 的错误处理三件套是 `handle_parsing_errors` + `OutputFixingParser` + `with_retry/with_fallbacks`，组合用最稳。

§3.8 业界目前怎么用 LangChain：迁出老接口

「在基础流程中的位置：业界对 LangChain ReAct 的真实评估。

真实情况

团队类型	选择	原因
2023 写的老项目	留在 v0.1 + AgentExecutor	不想改，能跑
2024 起的新项目	直接 LangGraph + LangChain 组件	官方推荐
严肃生产团队	LangGraph + LangSmith + 自部署	可控、可观测
快速 PoC	LangChain v0.3 现代写法	上手最快
不想被绑架的团队	直接 SDK + Pydantic + 自己写图	Anthropic 风格

业界案例对比

公司	用什么	为什么
Klarna	LangGraph	客服多步 + 检查点 + HITL
Replit	LangGraph	代码智能体 + 多智能体协作
LinkedIn	LangGraph	招聘助手 + 长会话
Anthropic 自家	直接 SDK	不想引入抽象
Cursor	闭源（推测自研）	编程智能体特化
Notion AI	LangChain v0.2（部分）	文档检索 + 简单链

何时仍该用 LangChain（无 LangGraph）

场景	用 LangChain 够
简单 RAG（一轮检索 + 回答）	是
单轮工具调用（无循环）	是
把现有 OpenAI 代码用 LCEL 改写	是
多步 Agent + 中断 + 持久化	否，必须 LangGraph

小结一行：LangChain 仍在用，但场景缩小到"简单管道 / 组件库" —— Agent 工作迁到 LangGraph。

§3.9 从 LangChain 老接口迁移到 LangGraph 的官方迁移指南

「在基础流程中的位置：节点 ③⑦ 的迁移路径。

官方迁移文档位置

LangGraph 文档站有专门的 `langgraph/how-tos/migrate-from-langchain` 章节。核心是把 `AgentExecutor` 替换成 LangGraph 的 `create_react_agent`。

迁移对照表

老 (LangChain)	新 (LangGraph)
<code>from langchain.agents import create_tool_calling_agent, AgentExecutor</code>	<code>from langgraph.prebuilt import create_react_agent</code>
<code>agent = create_tool_calling_agent(llm, tools, prompt)</code>	(一行下)
<code>executor = AgentExecutor(agent=agent, tools=tools)</code>	<code>agent = create_react_agent(llm, tools, prompt=prompt)</code>
<code>result = executor.invoke({"input": "????"})</code>	<code>result = agent.invoke({"messages": [HumanMessage(content="...")]})</code>
<code>chat_history</code> 字段	<code>messages</code> 字段 (统一为 <code>BaseMessage</code> 列表)
<code>agent_scratchpad</code> 占位	LangGraph State 自动管理
<code>max_iterations=8</code>	<code>recursion_limit=8</code> (在 config 中)
<code>handle_parsing_errors=True</code>	不需要 (结构化 <code>tool_calls</code> 不会解析错)
<code>return_intermediate_steps=True</code>	默认所有 <code>messages</code> 都返回

最小迁移示例

CODE

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({"input": " "})

from langgraph.prebuilt import create_react_agent
agent = create_react_agent(llm, tools, prompt=prompt)
result = agent.invoke({"messages": [{"role": "user", "content": " "}]})
```

迁移后立刻得到的好处

好处	实现
自动 checkpoint（会话续跑）	加 <code>checkpointer=MemorySaver()</code>
Human-in-the-loop	加 <code>interrupt_before=["tools"]</code>
流式中间步骤	<code>agent.astream_events(...)</code>
可视化图结构	<code>agent.get_graph().draw_mermaid()</code>

详见 §5.2 - §5.4。

小结一行：迁移官方提供一行替换，迁移后立刻拿到 checkpoint / HITL / 可视化四大好处。

§3.10 本章小结

#	核心结论
1	LangChain 实现 ReAct 的现代写法是 <code>create_tool_calling_agent</code> + <code>AgentExecutor</code> —— 用结构化 <code>tool_calls</code> 字段替代字符串解析
2	即使用现代写法, <code>AgentExecutor</code> 仍是黑盒 —— 节点 ③⑦ 不可控
3	五大硬伤逐节点对应: ⑥ 状态会话级 / ③ 黑盒难调 / ⑦ 循环粗放 / ⑤ 错误污染 / ⑥ 不能时间旅行
4	Callbacks 是无 LangSmith 时的本地观测手段, 能拿 token / 工具事件
5	错误处理三件套: <code>handle_parsing_errors</code> + <code>OutputFixingParser</code> + <code>with_retry/with_fallbacks</code>
6	业界主流: 新项目直接 <code>langgraph.prebuilt.create_react_agent</code> (一行替换)

§3.11 反模式速记

反模式	错在哪	正确做法
用 <code>initialize_agent(...)</code> 写新代码	v0.3 已发 deprecation	用 <code>create_tool_calling_agent</code> 或 迁 LangGraph
工具内异常直接抛	污染 prompt 历史	工具内 try/except 返回错误字符串
不设 max_iterations	死循环烧钱	必设上限
不开 verbose / 不接 LangSmith	调试靠肉眼	必开
期望 AgentExecutor 跨进程续跑	状态进程内	迁 LangGraph + Checkpointer

§3.12 术语速查

术语	中文	含义
AgentExecutor	智能体执行器	老黑盒（包含 prompt + llm + parser + 循环）
<code>create_tool_calling_agent</code>	工具调用智能体	v0.2 起的现代构造器
<code>agent_scratchpad</code>	智能体草稿板	中间 Thought/Action/Observation 的 prompt 占位
<code>intermediate_steps</code>	中间步骤	工具调用历史的列表形式
<code>handle_parsing_errors</code>	解析错误处理	自动让模型修正格式错的输出
<code>OutputFixingParser</code>	自我修复解析器	解析失败时让 LLM 重写
<code>BaseCallbackHandler</code>	回调处理器基类	自定义观测钩子
<code>OutputParserException</code>	输出解析异常	模型输出不符合预期
<code>RateLimitError</code>	限流错误	厂商限流
<code>astream_events</code>	异步流式事件	看链路所有内部事件

§3.13 推荐下一章

下一章: [§4 LangGraph：第二代框架的心智模型](#) —— §3 暴露的五大硬伤逐节点对应 LangGraph 的核心改进。

§4 LangGraph：第二代框架的心智模型

§4.0 本章定位

项	内容
在基础流程中的位置	把基础流程从"链"升级为"图"，重做节点 ⑥ 状态、节点 ③ 决策、节点 ⑦ 循环
与上下章的因果链	§3 暴露了 LangChain 在节点 ⑥/③/⑦ 的硬伤，本章讲 LangGraph 怎么从根本上改
学完能做什么	(1) 解释"链 → 图"的工程动机；(2) 解释 Pregel 超步执行模型；(3) 写五行最小图；(4) 区分 StateGraph / CompiledGraph；(5) 用三种执行接口 invoke / stream / batch

§4.1 LangGraph 起源：2024 年 LangChain 团队的反思

「在基础流程中的位置：直面 §3 暴露的五大硬伤。」

起源故事

2024 年 1 月，LangChain 团队在博客里反思过去一年：

「 "We realized that for production agents, the chain-of-X mental model is not enough. Agents need to be **graphs** — with cycles, conditional branches, and explicit state."

直译：链式心智模型不够 —— 智能体需要图（有循环、有条件分支、有显式状态）。

五大硬伤 → 解法对照（Mermaid #8）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph LC ["LangChain AgentExecutor"]
        H1["⑥ AgentExecutor"]
        H2["③ AgentExecutor"]
        H3["⑦ max_iterations"]
        H4["⑤"]
        H5["⑥"]
    end

    subgraph LG ["LangGraph"]
        S1["Checkpoint"]
        S2["Store API"]
        S3["recursion_limit + interrupt"]
        S4["try/except + retry"]
        S5["get_state_history"]
    end

    H1 --> S1
    H2 --> S2
    H3 --> S3
    H4 --> S4
    H5 --> S5

    style H1 fill:#fecaca,stroke:#ef4444
    style H2 fill:#fecaca,stroke:#ef4444
    style H3 fill:#fecaca,stroke:#ef4444
    style H4 fill:#fecaca,stroke:#ef4444
    style H5 fill:#fecaca,stroke:#ef4444
    style S1 fill:#d1fae5,stroke:#10b981
    style S2 fill:#d1fae5,stroke:#10b981
    style S3 fill:#d1fae5,stroke:#10b981
    style S4 fill:#d1fae5,stroke:#10b981
    style S5 fill:#d1fae5,stroke:#10b981
```

时间线

时间	事件
2024.01	LangGraph 0.0.x 发布
2024.05	LangGraph 0.1, StateGraph + Reducer + Checkpoint 成熟
2024.10	LangGraph 0.2, Functional API + Send + Store + 子图
2025.Q4	LangGraph 0.2.x 持续迭代, 加入推理模型支持、 Command 对象
2026.Q1	LangGraph Platform GA

设计动机的根本

LangChain 的 AgentExecutor 是"管道 + while 循环", 本质是**线性序列**。但智能体的真实形态是图:

智能体的真实形态	LangChain 表达	LangGraph 表达
节点之间有循环	max_iterations 硬限	边是循环的
不同决策走不同路径	内部 if/else	条件边
中间状态需要持久化	无	State + Checkpoint
一些节点要并行	无	Send + 并发
流程中要暂停问人	无	interrupt

小结一行：LangGraph 是 LangChain 团队对"链式不够"的官方回应 —— 升级心智模型到图。

§4.2 核心模型转变：从"链"到"图"

「在基础流程中的位置：基础流程的"形状"从线变成图。

链 vs 图的工程含义

维度	链 (Chain)	图 (Graph)
几何	有向无环 + 线性	有向图 (含循环)
节点	顺序执行	任意拓扑 (条件 / 并行 / 循环)
状态	节点间传值	共享 State 字典
控制流	写死的顺序	由边和条件决定
调试	看输入输出	看每个节点的状态变化
持久化	不天然	一等公民 (Checkpoint)

状态图 (State Graph) 的工程含义

LangGraph 的核心抽象是状态图 (State Graph, 简称 SG) :

元素	工程含义
State	整个图共享的字典（节点 ⑥ 状态管理）
Node	一个函数，读 State，返回 State 的部分更新（节点 ②④⑤）
Edge	节点之间的连线，可以是静态 / 条件（节点 ③⑦）
Reducer	State 字段的合并规则（覆盖 / 累加 / 自定义）
Checkpoint	State 的一个时间快照，可保存 / 恢复

图思维的核心好处

好处	在基础流程的哪
任意拓扑（不限链式）	③ 决策路由可以任意复杂
显式状态共享	⑥ 状态管理一等公民
边可以是循环的	⑦ 循环控制天然
边可以是条件的	③ 决策可显式
可视化天然	调试体验提升
持久化天然	节点 ⑥ 跨调用可恢复

小结一行：从链到图不是工程美学问题，是必要性问题 —— 真实智能体的拓扑需要循环和分支。

§4.3 三要素深入：状态 / 节点 / 边

在基础流程中的位置：State 对应节点 ⑥，Node 对应节点 ②④⑤，Edge 对应节点 ③⑦。

State：图的"共享内存"

CODE

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages] # 消息列表
    counter: int # 计数器
    metadata: dict # 元数据
```

State 字段	工程含义
字段名	节点之间共享的"插槽"
类型	TypedDict / Pydantic / dataclass
Annotated[..., reducer]	该字段的合并规则

Node：状态变换函数

CODE

```
def my_node(state: AgentState) -> dict:
    # 从 state 中取出 messages
    messages = state["messages"]
    # 调用 LLM
    new_msg = llm.invoke(messages)
    # 将新消息添加到 state 的 messages 列表中
    return {"messages": [new_msg], "counter": state["counter"] + 1}
```

节点签名	含义
输入	<code>state: State</code> (完整 state)
返回	<code>dict</code> (部分更新, 由 reducer 合并到 state)
同步 / 异步	都支持 (<code>async def my_node(state)</code>)
可副作用	是 (调 LLM / 工具 / 数据库)

Edge：连接节点的"路径"

边类型	用法	工程含义
静态边	<code>graph.add_edge("a", "b")</code>	跑完 a 一定跑 b
条件边	<code>graph.add_conditional_edges("a", router_fn, {"x": "b", "y": "c"})</code>	跑完 a, 调 router_fn 返回字符串决定下一节点
入口边	<code>graph.add_edge(START, "a")</code>	起点
出口边	<code>graph.add_edge("b", END)</code>	终点
循环边	<code>graph.add_edge("c", "a")</code>	显式循环

三要素与基础流程的对应

基础流程节点	LangGraph 元素
① 上下文组装	节点函数体内拼 prompt
② 推理	LLM 节点
③ 决策路由	条件边
④ 工具调用	ToolNode
⑤ 工具返回	写回 State
⑥ 状态管理	State + Reducer + Checkpoint
⑦ 循环控制	循环边 + recursion_limit + interrupt
⑧ 输出格式化	出口节点

小结一行：State / Node / Edge 三要素一一对应基础流程的关键节点，把"管道 + while"换成"图 + 状态机"。

§4.4 超步并行执行模型（Pregel）

在基础流程中的位置：节点级别的并发执行机制。

Pregel 的来源

Pregel 是 Google 2010 年发表的图计算论文（用于 PageRank 这类大规模图算法）。LangGraph 借用了它的执行模型：

CODE

```
Superstep 1: [?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?]State [?][?][?][?][?][?][?][?][?]
Superstep 2: [?][?][?]State [?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?]
[?][?][?]
[?][?][?][?][?][?][?][?][?][?][?][?]
```

拓扑	执行
$A \rightarrow [B, C] \rightarrow D$	A 跑完, B 和 C 并行 跑 (同一超步), 都跑完 $\rightarrow D$
$A \rightarrow B \rightarrow C$ 线性	一步一步顺序
$A \rightarrow A$ 循环	每超步跑一次 A

与传统执行模型对比

模型	执行方式	何时用
链 (Chain)	严格线性	LangChain
DAG (无环图)	拓扑排序	Airflow / Prefect
Pregel (超步)	同步并行批	LangGraph
Actor / 消息驱动	完全异步	Erlang / Akka

Pregel 模型对开发者意味着什么

影响	解释
可推理性	同一超步内节点不能依赖其他节点的"中间结果", 避免竞态
并发优化	框架自动并行同超步内节点, 不用写 <code>asyncio.gather</code>
调试友好	每超步是个清晰的"快照点"
Checkpoint 自然	屏障点天然是在写 <code>checkpoint</code> 的位置

小结一行：Pregel 给 LangGraph 提供了"同步并行"的执行模型 —— 简单图也能拿到自动并发。

§4.5 五行最小图 (Hello World)

在基础流程中的位置：用最小代码示例演示节点 ②⑥⑦⑧。

最小可运行图

CODE

```

from langgraph.graph import StateGraph, START, END
from typing import TypedDict

???State
class State(TypedDict):
    value: int

???
def add_one(state: State) -> dict:
    return {"value": state["value"] + 1}

???
graph = StateGraph(State)
graph.add_node("incrementer", add_one)
graph.add_edge(START, "incrementer")
graph.add_edge("incrementer", END)
app = graph.compile()

???
result = app.invoke({"value": 0})
print(result) # {"value": 1}

```

三要素关系 (Mermaid #9)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph SG["StateGraph"]
        S["State"]
        N1["Node A"]
        N2["Node B"]
        E1["Edge"]
    end

    Start([START]) --> N1
    N1 --> S
    S --> N2
    N2 --> S
    N1 --- E1 --- N2
    N2 --> End([END])

    style S fill:#fef3c7,stroke:#f59e0b,stroke-width:3px
    style N1 fill:#dbeafe,stroke:#3b82f6
    style N2 fill:#dbeafe,stroke:#3b82f6
    style E1 fill:#fce7f3,stroke:#ec4899
```

升级到带条件分支的图

CODE

```
def router(state: State) -> str:
    return "doubler" if state["value"] > 5 else "incrementer"

def double(state: State) -> dict:
    return {"value": state["value"] * 2}

graph = StateGraph(State)
graph.add_node("incrementer", add_one)
graph.add_node("doubler", double)
graph.add_conditional_edges(
    START,
    router,
    {"incrementer": "incrementer", "doubler": "doubler"},
)
graph.add_edge("incrementer", END)
graph.add_edge("doubler", END)
app = graph.compile()
```

`add_conditional_edges` 三个参数：源节点、路由函数（返回字符串）、字符串到节点名的映射。

节点函数的几种返回值类型

返回值	含义
<code>dict</code>	部分 State 更新
<code>Command(update={...}, goto="next_node")</code>	同时更新 state 和路由 (0.2.30+)
<code>Command(update=..., goto=END)</code>	强制终止

小结一行：5 行最小图揭示了 StateGraph 的三个核心 API (add_node / add_edge / compile)
+ 条件边 / Command 是稍进阶的工具。

§4.6 编译与可视化：StateGraph vs CompiledGraph

「在基础流程中的位置：理解构建期和执行期的区别。

两个对象的区别

对象	何时存在	能做什么
<code>StateGraph</code>	编译前	加节点、加边、加条件路由
<code>CompiledGraph</code>	<code>graph.compile()</code> 之后	invoke / stream / batch / 拿可视化

调用 `compile()` 后图变不可变，所有节点和边都被锁定，开始可执行。

CompiledGraph 的执行接口

接口	同步 / 异步	输入	输出
<code>invoke(input, config)</code>	同步	单输入	最终 state
<code>ainvoke(input, config)</code>	异步	单输入	最终 state
<code>stream(input, config)</code>	同步	单输入	迭代器（每步 state 增量）
<code>astream(input, config)</code>	异步	单输入	异步迭代器
<code>batch(inputs, config)</code>	同步	多输入	多个最终 state
<code>abatch(inputs, config)</code>	异步	多输入	多个最终 state
<code>astream_events(input, config, version)</code>	异步	单输入	内部事件流

config 参数的作用

CODE

```
result = app.invoke(
    {"value": 0},
    config={
        "configurable": {"thread_id": "user-123"}, # 节点ID
        "recursion_limit": 25, # 递归深度
        "tags": ["dev"], # LangSmith 标签
        "metadata": {"user_id": "abc"},
        "callbacks": [my_callback], # 回调函数
    }
)
```

config 字段	作用
configurable.thread_id	Checkpoint 续跑的会话标识（详见 §5.3）
recursion_limit	节点 ⑦ 循环上限（默认 25）
tags / metadata	给 LangSmith 用
callbacks	同 LangChain Callback

可视化：自带 Mermaid 渲染

CODE

```
print(app.get_graph().draw_mermaid())
app.get_graph().draw_mermaid_png("graph.png")
```

输出可直接贴到 Mermaid 编辑器看图。调试和文档双用。

异步全栈支持

CODE

```
def sync_node(state):  
    # Sync node logic  
  
async def async_node(state):  
    result = await some_async_call()  
    # Async node logic  
  
# Graph definition and invocation logic
```

调用方根据需要选 `invoke` 或 `ainvoke`：

调用方式	节点支持
<code>invoke</code>	同步节点 + 异步节点（自动跑事件循环）
<code>ainvoke</code>	同步节点 + 异步节点

推荐：节点尽量写成 `async def`，调 `ainvoke` —— 在 Web 服务里能拿满并发。

小结一行：`compile()` 是构建期到执行期的分界线，`CompiledGraph` 提供 6 种执行接口 + 可视化。

§4.7 业界目前怎么用 LangGraph：典型案例

「在基础流程中的位置：业界采用真实数据。

Klarna：客服智能体

维度	内容
场景	用户问退款 / 订单 / 物流
节点	意图识别 → 信息查询（多工具）→ 决策 → 答复
用 LangGraph 的关键	多工具并发（Send 扇出）+ 长会话 Checkpoint + HITL（危险操作前确认）
公开数据	处理 2/3 客服流量、平均处理时间从 11 分钟降到 2 分钟

Replit：编程智能体

维度	内容
场景	在 Replit IDE 中由用户描述要写的功能，AI 自动写代码 / 跑测试 / 修改
节点	规划 → 编辑文件 → 跑测试 → 看错误 → 修改（循环）
用 LangGraph 的关键	长循环（recursion_limit 高）+ 多智能体协作（编辑器 + 测试者）
公开数据	"Replit Agent" 产品的核心

LinkedIn：招聘助手

维度	内容
场景	招聘者描述岗位，AI 检索匹配候选人
节点	意图理解 → RAG 检索（多种向量库）→ 排序 → 生成解释
用 LangGraph 的关键	多检索源并发 + State 共享

Elastic / Uber：内部知识助手

维度	内容
场景	员工问"如何配置 X" / 文档检索
节点	检索 → 验证 → 生成 → 引用注入
用 LangGraph 的关键	评估循环（生成 → 自评 → 必要时重检索）

共同模式

模式	出现频率
多工具并发（Send）	几乎所有客服 / 检索
Checkpoint 续跑	长任务 / 多轮对话
HITL（人工审批）	涉及钱 / 写操作
评估反思循环	RAG / 编程
多智能体（Supervisor）	复杂任务分工

小结一行：业界采用 LangGraph 的真实场景集中在"多工具 + 长流程 + 高可控"的生产智能体。

§4.8 本章小结

#	核心结论
1	LangGraph 起源于 LangChain 团队 2024 年的反思 —— 链式不够，必须图
2	五大硬伤逐节点对应 LangGraph 的核心改进（状态 / 决策 / 循环 / 错误 / 时间旅行）
3	三要素 State / Node / Edge 一一对应基础流程的核心节点
4	Pregel 超步并行执行模型让"同步并行"自动获得
5	StateGraph（构建期）→ compile() → CompiledGraph（执行期），后者提供 6 种执行接口
6	业界主流采用 LangGraph 的场景：多工具 + 长流程 + Checkpoint + HITL

§4.9 反模式速记

反模式	错在哪	正确做法
不 compile 直接用 StateGraph	没法 invoke	必须 <code>graph.compile()</code>
节点函数返回完整 State	不必要、低效	只返回需要更新的字段
不设 recursion_limit	默认 25 太低或太高	显式设置匹配场景
节点之间用全局变量传值	破坏图的可推理性	全部走 State
同步节点里同步调远程 API	Pregel 屏障被阻塞	改 async + ainvoke

§4.10 术语速查

术语	中文	含义
StateGraph	状态图	LangGraph 的图构造器
CompiledGraph	已编译图	compile() 后可执行的图
State	状态	图共享的字典
Node	节点	状态变换函数
Edge	边	节点之间的连线
Reducer	归约器	State 字段的合并规则
Checkpoint	检查点	State 的快照
Pregel	超步并行模型	Google 2010 论文, LangGraph 借用
Superstep	超步	一轮节点并行执行
START / END	起点 / 终点	LangGraph 的特殊节点
<code>add_conditional_edges</code>	加条件边	根据路由函数返回值决定下一节点
<code>recursion_limit</code>	递归上限	节点 ⑦ 循环次数硬限
<code>Command</code>	命令对象	同时更新 state + 路由 (0.2.30+)
<code>astream_events</code>	异步流式事件	看图内部所有事件

§4.11 推荐下一章

下一章：[§5 用 LangGraph 实现 ReAct（同任务对比 §3）](#) —— 把 §4 的概念落到代码，用 LangGraph 重写 §3 的同一个任务。

§5 用 LangGraph 实现 ReAct（同任务对比 §3）

§5.0 本章定位

项	内容
在基础流程中的位置	把整条流程用 LangGraph 重做一遍，每节点对照 §3
与上下章的因果链	§3 LangChain 写 ReAct, §5 用 LangGraph 写同任务 —— 看代码差异
学完能做什么	(1) 手写 LangGraph ReAct 循环；(2) 用 <code>prebuilt create_react_agent</code> ；(3) 加 Checkpoint 三种保存器；(4) 加 HITL 暂停审批；(5) 配置 <code>thread_id</code> ；(6) 处理 <code>GraphRecursionError</code>

§5.1 手写 ReAct 循环：State + 条件边

「在基础流程中的位置：用 LangGraph 重做节点 ②③④⑤⑥⑦。」

场景引入

§3 用 LangChain 实现了 ReAct 智能体（查天气 + 计算）。本节用 LangGraph 重写 **完全相同的任务**，看代码差异。

完整代码：手写 ReAct 循环


```

return END

#####
graph = StateGraph(AgentState)
graph.add_node("agent", call_model)
graph.add_node("tools", tool_node)

graph.add_edge(START, "agent")
graph.add_conditional_edges("agent", should_continue, {
    "tools": "tools",
    END: END,
})

#####
graph.add_edge("tools", "agent") # #####

app = graph.compile()

#####
result = app.invoke({
    "messages": [HumanMessage(content="#####")],
    #####
})

#####
for msg in result["messages"]:
    print(f"[{type(msg).__name__}]", msg.content[:100])

```

图的可视化 (Mermaid #10)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    Start([START]) --> Agent["agent #####  
call_model"]
    Agent -->|tool_calls #####| Tools["tools #####  
ToolNode"]
    Agent -->|#####tool_calls| End([END])
    Tools -->|#####Agent

    style Agent fill:#dbeafe,stroke:#3b82f6
    style Tools fill:#d1fae5,stroke:#10b981

```

这段代码与 §3.3 LangChain 实现的关键差异

维度	LangChain (§3.3)	LangGraph (§5.1)
节点 ③ 路由	AgentExecutor 黑盒	<code>should_continue</code> 显式函数
节点 ⑥ 状态	<code>agent_scratchpad</code> 字段（黑盒）	<code>messages</code> State 字段（可见可改）
节点 ⑦ 循环	AgentExecutor 内部 while	显式循环边 <code>tools → agent</code>
工具节点	<code>Tool</code> 列表 + executor 自调	<code>ToolNode</code> 显式节点
错误处理	<code>handle_parsing_errors</code>	在节点内 try/except
中间步骤可见	要 <code>return_intermediate_steps=True</code>	默认所有 messages 在 state 里

小结一行：手写 LangGraph ReAct 把每个节点和路由都暴露出来，可控可调试 —— 但前提是画图时清晰。

§5.2 prebuilt create_react_agent

「在基础流程中的位置：把 §5.1 的 30 行图压缩到 1 行。

一行版本

CODE

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(
    llm,
    tools,
    prompt="???",
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "???"}],
})
```

create_react_agent 的内部就是 §5.1

打开源码可以看到，`create_react_agent` 内部就是 §5.1 那张图（StateGraph + agent 节点 + tools 节点 + 条件边 + 循环）+ 一些便利封装。

何时用 prebuilt vs 自己手写

场景	用什么
标准 ReAct（一个 LLM + 一组工具）	<code>create_react_agent</code> 一行
需要自定义路由逻辑	手写图
多个 LLM 协作（Supervisor）	手写图（详见 §6.3）
需要插入预处理 / 后处理节点	手写图
需要并发分支（Send）	手写图

prebuilt 的可配置参数

CODE

```
agent = create_react_agent(
    model=llm,
    tools=tools,
    prompt="...",
    state_schema=CustomState,
    checkpointer=MemorySaver(),
    interrupt_before=["tools"],
    interrupt_after=[],
    debug=True,
    # 自定义提示词
    # 自定义 State
    # 自定义工具
    # 自定义 breakpoint (自定义工具)
```

小结一行：`create_react_agent` 是"标准 ReAct 一行版"，不满足时改用手写图（见 §6 进阶）。

§5.3 检查点：三种保存器 + thread_id 续跑

「在基础流程中的位置：节点 ⑥ 状态管理升级为持久化。

检查点（Checkpoint）解决什么

回顾 §3.5 硬伤 1：LangChain 的 Memory 是会话级 + 进程内。LangGraph 的检查点机制把 State 写到外部存储，让智能体跨进程、跨会话续跑。

三种内置 Saver

Saver	用法	适用
MemorySaver	进程内 dict	开发 / 单进程演示
SqliteSaver	SQLite 数据库	单机持久化 / 个人项目
PostgresSaver	Postgres 数据库	生产 / 多实例 / 大规模

MemorySaver 示例

CODE

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
agent = create_react_agent(llm, tools, checkpointer=memory)

thread_id = "user-123"
config = {"configurable": {"thread_id": "user-123"}}
result1 = agent.invoke(
    {"messages": [{"role": "user", "content": "Hello"}]},
    config=config,
)

thread_id, config = agent.get_state(configurable={"thread_id": "user-123"})
result2 = agent.invoke(
    {"messages": [{"role": "user", "content": "Hello"}]},
    config=config,
)

# result2.messages
```

Postgres 生产级示例

CODE

```
from langgraph.checkpoint.postgres import PostgresSaver
from psycopg_pool import ConnectionPool

DB_URI = "postgresql://user:pass@localhost:5432/checkpoints"
pool = ConnectionPool(DB_URI)

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # 创建 4 张表

    agent = create_react_agent(llm, tools, checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "user-123"}}
    result = agent.invoke({"messages": [...]}), config=config)
```

`setup()` 会在 Postgres 中创建 4 张表： - `checkpoints`：State 快照 - `checkpoint_writes`：节点写入记录 - `checkpoint_blobs`：大字段二进制 - `checkpoint_migrations`：版本管理

Checkpoint 写入时序 (Mermaid #11)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
sequenceDiagram
    participant U as User
    participant A as Agent (CompiledGraph)
    participant N1 as Node A
    participant N2 as Node B
    participant CP as Checkpointer

    U->>A: invoke(input, thread_id=X)
    A->>CP: load(thread_id=X)
    CP-->>A: State

    Note over A: 
    A->>N1: state
    N1-->>A: update
    A->>CP: save(checkpoint_1)

    Note over A: 
    A->>N2: state
    N2-->>A: update
    A->>CP: save(checkpoint_2)

    A-->>U: state
```

thread_id 的语义

概念	说明
<code>thread_id</code>	一次"会话"或"任务"的唯一标识
同 <code>thread_id</code>	相同 <code>thread_id</code> 的多次 <code>invoke</code> 会自动接上历史
不同 <code>thread_id</code>	完全隔离
命名建议	<code>user-{user_id}</code> / <code>session-{uuid}</code>

config 与 configurable 运行时

`config` 是每次调用时传的运行时配置：

CODE

```

result = agent.invoke(
    input,
    config={
        "configurable": {
            "thread_id": "user-123",          # checkpoint ⑥
            "checkpoint_ns": "weather",       # ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲
            "user_id": "abc",                 # ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲
            ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲
            "recursion_limit": 50,            # ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲
            "tags": ["prod"],
            ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲
            ①
        }
    }

    ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲config
def my_node(state, config: RunnableConfig):
    user_id = config["configurable"]["user_id"]
    ①②③④⑤⑥⑦⑧⑨⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲

```

小结一行：Checkpoint + thread_id 让节点 ⑥ 从"会话内"升级到"跨会话持久化"，三种 Saver 按规模选。

§5.4 人在回路：interrupt() 与 NodeInterrupt

「在基础流程中的位置：节点 ⑦ 循环控制 + 节点 ④ 工具调用前的人工审批。

为什么需要 HITL

场景	不要 HITL	要 HITL
查天气	否	否
提交退款 100 元	否	是（钱）
删生产数据	否	是（不可逆）
发送营销邮件给所有用户	否	是（影响大）
改代码并合 PR	否	是（生产风险）

静态 breakpoint：interrupt_before / interrupt_after

最简单的 HITL —— 在编译图时声明"工具节点前必须暂停"：

```

from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
agent = create_react_agent(
    llm, tools,
    checkpointer=memory,
    interrupt_before=["tools"],  # 在调用工具前中断
)

config = {"configurable": {"thread_id": "user-123"}}

# 调用工具
invoke_result = agent.invoke(
    {"messages": [{"role": "user", "content": "请帮我查询一下天气"}]},
    config=config,
)

# 获取当前状态
state = agent.get_state(config)
if state.next == ("tools",):
    print(f"当前状态为 tools，工具调用次数为 {state.values['messages'][-1].tool_calls}")
    user_approve = input("是否继续调用工具 (y/n)? ")

    if user_approve == "y":
        # 继续调用工具
        agent.invoke(None, config=config)
    else:
        # 更新状态
        state = agent.get_state(config)
        agent.update_state(
            config,
            {"messages": [ToolMessage(content="工具调用失败，请重试", tool_call_id=...)]},
        )
        agent.invoke(None, config=config)

```

动态中断：interrupt() 函数

LangGraph 0.2 起支持节点内动态中断（不必在编译时静态声明）：

CODE

```

from langgraph.types import interrupt, Command

def review_node(state):
    last_msg = state["messages"][-1]
    if "???" in last_msg.content:
        approval = interrupt({"action": "???", "amount": 1000})
        if approval == "approved":
            return {"messages": [ToolMessage(content="???")]}
        else:
            return {"messages": [ToolMessage(content="???")]}
    return {}

???Command(resume=...)
agent.invoke(Command(resume="approved"), config=config)

```

暂停-恢复时序 (Mermaid #12)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

sequenceDiagram
    participant U as ?
    participant App as Web App
    participant A as Agent
    participant CP as Checkpointer

    U->>App: ???
    App->>A: invoke(input, thread_id=X)
    A->>A: ???
    A->>CP: save(state with next=tools)
    A-->>App: ???status=interrupted
    App-->>U: ???

    Note over U: ???

    U->>App: ?
    App->>A: invoke(None, config) ?
    A->>CP: load(state)
    A->>A: ?tools ?
    A->>A: ?agent ?
    A-->>App: ???
    App-->>U: "???"

```


三种 HITL 模式

模式	实现	适用
静态 breakpoint	<code>interrupt_before=["tools"]</code>	简单"工具前必停"
动态 <code>interrupt()</code>	节点内调用	条件性暂停 ("超过 1000 元才停")
<code>update_state</code>	修改历史 state	修正错误 / 注入纠偏

小结一行：HITL 三件套（静态 breakpoint / 动态 interrupt / update_state）覆盖"暂停 - 审批 - 修正"全场景。

§5.5 与 §3 LangChain 实现的逐行对比表

「在基础流程中的位置：把两种实现拉通看每个节点的差异。

同任务两种实现

节点	LangChain (§3.3)	LangGraph (§5.1)	行数差异
工具定义	<code>@tool</code> 装饰器	<code>@tool</code> 装饰器 (相同)	0
LLM 创建	<code>ChatOpenAI(...)</code>	<code>ChatOpenAI(...)</code>	0
工具绑定	<code>tools=[...]</code> 传 executor	<code>llm.bind_tools(tools)</code>	-1 (更直接)
状态管理	<code>agent_scratchpad</code> (黑盒)	<code>messages</code> State + <code>add_messages</code>	+3 (更显式)
节点 ② 推理	在 AgentExecutor 内部	<code>call_model</code> 函数	+3 (独立函数)
节点 ④⑤ 工具	AgentExecutor 内部循环调	<code>ToolNode(tools)</code>	+1
节点 ③ 路由	黑盒 <code>AgentOutputParser</code>	<code>should_continue</code> 函数	+5 (显式)
图构建	<code>AgentExecutor(agent, tools)</code>	<code>StateGraph</code> + <code>add_node</code> + <code>add_edge</code> + <code>compile</code>	+6
调用	<code>executor.invoke({"input": ??????})</code>	<code>app.invoke({"messages": ??????})</code>	0

总行数：LangChain ~50 行 / LangGraph ~60 行（因为节点 ②③ 显式化）

各节点的"可控性"对比

节点	LangChain 可控性	LangGraph 可控性
① 上下文	通过 prompt 模板	节点函数内自由组装 + State 读取
② 推理	配置 LLM	任意函数节点
③ 决策	黑盒	显式（路由函数）
④ 工具	Tool 列表	ToolNode（可替换）
⑤ 返回	内部处理	写回 State
⑥ 状态	Memory（会话级）	State + Checkpoint（持久）
⑦ 循环	max_iterations	recursion_limit + interrupt + 条件边
⑧ 输出	OutputParser	出口节点

调试体感对比

任务	LangChain	LangGraph
看每步 state	<code>return_intermediate_steps=True</code> 拿 list	<code>app.get_state(config)</code> 拿完整快照
中间改 state	不支持	<code>update_state()</code> 直接改
跨进程续跑	不行	<code>thread_id + checkpointer</code>
暂停问人	不行（没原生）	<code>interrupt_before</code> 一行加
看图结构	不直观	<code>app.get_graph().draw_mermaid()</code>
时间旅行	不行	<code>get_state_history()</code>

小结一行：同任务 LangGraph 多 10 行代码，但每个节点变成"显式可控"，生产环境的可调试性指数级提升。

§5.6 GraphRecursionError 与递归保护

「在基础流程中的位置：节点 ⑦ 循环控制的安全网。

触发条件

LangGraph 默认 `recursion_limit=25`。一旦图跑超 25 个超步（包括循环回去的次数），抛 `GraphRecursionError`：

CODE

```
from langgraph.errors import GraphRecursionError

try:
    result = app.invoke(input, config={"recursion_limit": 25})
except GraphRecursionError as e:
    print(f"GraphRecursionError: {e}")
```

为什么会死循环

原因	解决
模型反复要求调同一个工具	在 prompt 里强调"不要重复调用"
工具返回总是错，模型不知道放弃	工具内 try/catch 标记错误 + 路由识别
路由函数逻辑错	加日志查
故意需要长循环	调高 <code>recursion_limit</code> （如 100）

调整 recursion_limit

CODE

```
result = app.invoke(input, config={"recursion_limit": 100}) # 成功
```

检测死循环的硬指标

指标	阈值
同工具重复调用	≥ 3 次连续
总循环次数	> 默认 25
token 消耗	单次会话 > 50k
时长	单次会话 > 5 分钟

可以在节点函数里加监测：

CODE

```
def call_model(state):
    if state.get("loop_count", 0) > 10:
        return {"messages": [AIMessage(content="???")]}
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response], "loop_count": state.get("loop_count", 0) + 1}
```

小结一行：`recursion_limit` 默认 25 是常见任务的合理上限，超出说明设计有问题或场景需要显式调高。

§5.7 时间旅行：分支重放

「在基础流程中的位置：节点 ⑥ 状态管理的"穿越"能力。

时间旅行能做什么

能力	用法
看图跑过哪些超步	<code>app.get_state_history(config)</code>
回到任意 checkpoint	<code>config = state.config</code> 后再 <code>invoke</code>
修改某 checkpoint 后续跑（分支）	<code>update_state</code> 改完后 <code>invoke</code>

看历史

CODE

```
config = {"configurable": {"thread_id": "user-123"}}

for state in app.get_state_history(config):
    print(f"state.metadata['step'] state.next")
    print(f" state: {state.values}")
    print(f" checkpoint_id: {state.config['configurable']['checkpoint_id']}")
```

分支重放：从历史 checkpoint 创新分支

CODE

```
checkpoint
target_checkpoint = list(app.get_state_history(config))[3] # 

config 
new_config = target_checkpoint.config

state 
app.update_state(
    new_config,
    {"messages": [HumanMessage(content="")]},
)
result = app.invoke(None, config=new_config)
```

时间旅行的工程价值

场景	价值
调试：回到错的那步看 state	调试效率大幅提升
A/B 测试：从同一中间状态分两个分支	比较不同后续策略
用户"撤回"：从历史某点重新选	客服场景必备
审计：完整还原任务执行链	合规 / 事后分析

小结一行：时间旅行让节点 ⑥ 从"现在"扩展到"历史"和"分支"，是 LangGraph 区别于 LangChain 的杀手级能力之一。

§5.8 业界目前怎么用 LangGraph

「在基础流程中的位置：业界对 LangGraph 的真实采纳情况。

何时必须 LangGraph

场景	LangChain 够	必须 LangGraph
单轮 RAG	是	否
一次工具调用	是	否
简单链式调用	是	否
多步循环 + 中断	否	是
跨会话续跑	否	是
多智能体协作	否	是
HITL 审批流	否	是
长流程审计 / 时间旅行	否	是

常见的"LangChain → LangGraph"迁移触发点

触发点	例子
长会话 token 爆	50 轮对话 prompt 30k token
用户要求"接着上次聊"	跨会话续跑
业务要求"操作前必须批"	HITL
多人协作（销售 + 客服 + 物流）	多智能体
定时任务（每天扫描）	后台 agent + Cron

业界采纳的现实

LangGraph 在 2024 年下半年到 2026 年初已成为生产级智能体的事实标准 —— 但**新项目仍有相当比例选择不用任何框架**（直接 SDK + 自写图），这种风格 Anthropic 自家最典型。

小结一行：LangGraph 已是生产智能体的主流选择，但"框架最小化"派也在增长。

§5.9 本章小结

#	核心结论
1	LangGraph 手写 ReAct 把每个节点和路由暴露出来 —— 比 LangChain 多 10 行代码、多 100 倍可控性
2	<code>create_react_agent</code> 是"标准 ReAct 一行版"，本质是 §5.1 那张图的便利封装
3	三种 Checkpoint 保存器（Memory / SQLite / Postgres）按规模选
4	thread_id 是会话标识 —— 同 thread_id 自动续跑历史
5	HITL 三件套：静态 breakpoint / 动态 interrupt / update_state，覆盖暂停-审批-修正
6	<code>recursion_limit</code> 默认 25 是死循环的安全网
7	时间旅行 + 分支重放是 LangGraph 区别于 LangChain 的杀手级能力

§5.10 反模式速记

反模式	错在哪	正确做法
不传 thread_id 用 checkpointer	不能续跑	必须 <code>configurable.thread_id</code>
MemorySaver 上生产	重启丢数据	用 PostgresSaver
不设 recursion_limit	默认 25 可能不够	按场景调（如 long-running 任务设 100）
同 thread_id 跑不同任务	历史污染	每个任务独立 thread_id
HITL 让前端轮询 state	浪费请求	用 <code>astream_events</code> 推送中断事件
节点之间用全局变量	破坏可推理性	全部走 State

§5.11 术语速查

术语	中文	含义
<code>create_react_agent</code>	创建 ReAct 智能体	LangGraph prebuilt 一行构造器
<code>ToolNode</code>	工具节点	langgraph.prebuilt 的工具调用节点
Checkpointner	检查点保存器	State 持久化后端
MemorySaver / SqliteSaver / PostgresSaver	三种保存器	按规模选
<code>thread_id</code>	会话标识	Checkpoint 的隔离单位
<code>interrupt_before</code> / <code>interrupt_after</code>	静态断点	编译时声明
<code>interrupt()</code>	动态中断	节点内调用
<code>Command(resume=...)</code>	恢复命令	从中断处续跑
<code>update_state()</code>	更新状态	在历史 state 上做修改
<code>get_state_history()</code>	获取历史	看所有超步快照
<code>GraphRecursionError</code>	图递归错	超 recursion_limit
<code>RunnableConfig</code>	运行时配置	invoke 时传的 config 字典类型

§5.12 推荐下一章

下一章：[§6 LangGraph 进阶（为生产环境完善每个节点）](#) —— 把基础 LangGraph 知识扩展到生产级（State 高级用法 / Send / 多智能体 / Functional API / Store / 流式 / 部署）。

§6 LangGraph 进阶（为生产环境完善每个节点）

§6.0 本章定位

项	内容
在基础流程中的位置	完善节点 ⑥ 状态、节点 ④ 工具、节点 ⑦ 循环、节点 ⑧ 输出
与上下章的因果链	§5 跑通基础版，本章上生产必备的进阶能力
学完能做什么	(1) 设计复合 State + Reducer + Command；(2) 用 Send 做动态扇出；(3) 写 Supervisor / Plan-Execute / Swarm 多智能体；(4) 用 Functional API 装饰器；(5) 配 Store API 长期记忆；(6) 配五种流式模式 + LangSmith；(7) 集成 ToolNode / MCP；(8) 部署到 LangGraph Server / Platform

§6.1 节点 ⑥ 状态深入：State / Reducer / add_messages / Command

「在基础流程中的位置：节点 ⑥ 状态管理的高级用法。

三种 State 定义方式

方式	类型	优势	劣势
<code>TypedDict</code>	Python 内置类型注解	最轻量、官方默认	无运行时验证
<code>Pydantic BaseModel</code>	数据校验库	运行时验证 + JSON Schema	序列化开销
<code>dataclass</code>	Python 内置	中庸、有默认值机制	无运行时验证

TypedDict（推荐默认）

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]
    user_id: str
    counter: int
    metadata: dict
```

CODE

Pydantic（需要校验时）

CODE

```
from pydantic import BaseModel, Field
from typing import Annotated
from langgraph.graph import add_messages

class State(BaseModel):
    messages: Annotated[list, add_messages] = Field(default_factory=list)
    user_id: str
    counter: int = 0
    metadata: dict = Field(default_factory=dict)
```

Reducer 三种语义

State 字段的 `Annotated[..., reducer]` 指定该字段如何"合并"节点的部分更新：

Reducer	工程含义	例子
默认（无 Annotated）	覆盖（last-write-wins）	<code>counter: int</code> 后写覆盖前写
<code>add_messages</code>	消息列表自动追加 + 去重	<code>messages</code> 字段
<code>operator.add</code>	列表 / 数字累加	<code>Annotated[list[str], operator.add]</code>
自定义函数	任意合并逻辑	见下

Reducer 三种语义图 (Mermaid #13)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph Override["Override (Last-Write-Wins)"]
        O1["state.counter = 5"]
        O2["Node 1 counter=10"]
        O3["state.counter"]
        O1 --> O2
        O2 --> O3
    end

    subgraph Append["add_messages"]
        A1["state.messages = [m1, m2]"]
        A2["Node 1 m3"]
        A3["state.messages = [m1, m2, m3]"]
        A1 --> A2
        A2 --> A3
    end

    subgraph Custom["Reducer"]
        C1["state.scores = {x: 5}"]
        C2["Node 1 y: 8"]
        C3["state.scores = {x: 5, y: 8}"]
        C1 --> C2
        C2 --> C3
    end

    style O3 fill:#fef3c7
    style A3 fill:#d1fae5
    style C3 fill:#dbeeaf
```

自定义 Reducer 示例

CODE

```
def merge_dicts(left: dict, right: dict) -> dict:
    return {**left, **right}

class State(TypedDict):
    scores: Annotated[dict, merge_dicts]
```

add_messages 的隐藏魔法

`add_messages` 不只是简单 `append`, 还做:

行为	说明
追加新消息	默认行为
通过 <code>id</code> 去重	同 id 的消息会被替换而非重复
通过 <code>id</code> 删除	传 <code>RemoveMessage(id=x)</code> 删除已有消息
自动转换	dict 类自动转为对应 <code>BaseMessage</code>

CODE

```
from langchain_core.messages import RemoveMessage

def filter_node(state):
    return {"messages": [RemoveMessage(id=state["messages"][0].id)]}
```

Command 对象 (0.2.30+ 推荐)

老写法只能返回 `dict` 更新 `state`，要路由得靠条件边。新写法用 `Command` 同时做这两件事：

CODE

```
from langgraph.types import Command

def my_node(state) -> Command:
    return Command(
        update={"messages": [AIMessage(content="...")]}, # state
        goto="next_node", # 
    )
```

Command 字段	作用
<code>update={...}</code>	state 部分更新
<code>goto="node_name"</code>	跳转到指定节点
<code>goto=END</code>	强制终止
<code>goto=[Send("node", payload)]</code>	触发 Send 扇出

好处：把"决策"和"动作"合并到节点函数内，省一个独立的 router 函数。

小结一行：State 用 TypedDict + add_messages 是默认配方，Reducer 控制合并语义，Command 把更新+路由合并 —— 这三者覆盖 99% 场景。

§6.2 节点 ⑦ 高级循环：子图与 Send API

「在基础流程中的位置：节点 ⑦ 循环控制的"扇出 / 收敛"扩展。

子图：把图当节点用

CODE

```
from langgraph.graph import StateGraph

# 子图
sub = StateGraph(SubState)
sub.add_node("planner", plan_step)
sub.add_node("executor", exec_step)
sub.add_edge(START, "planner")
sub.add_edge("planner", "executor")
sub.add_edge("executor", END)
sub_compiled = sub.compile()

# 主图
main = StateGraph(MainState)
main.add_node("preprocess", preprocess)
main.add_node("subagent", sub_compiled) # 子图
main.add_node("postprocess", postprocess)
main.add_edge(START, "preprocess")
main.add_edge("preprocess", "subagent")
main.add_edge("subagent", "postprocess")
main.add_edge("postprocess", END)
```

状态映射：input_schema / output_schema

子图的 State 类型可以与主图不同，通过 `input_schema` / `output_schema` 映射：

CODE

```
sub = StateGraph(SubState, input=MainState, output=MainState)
```

Send API：动态扇出

老路由：路由函数返回 1 个目标节点名。**Send** 让节点函数同时触发 **N** 个子任务，每个带不同输入：

CODE

```

from langgraph.types import Send

def fanout(state):
    for item in state["items"]:
        return [Send("worker", {"item": item}) for item in state["items"]]

graph.add_conditional_edges("dispatcher", fanout)
graph.add_node("worker", worker_node)

```

`Send("worker", payload)` 含义：以 `payload` 为输入跑 `worker` 节点。

Send 的扇出图 (Mermaid #14)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    Start([START]) --> D[dispatcher]
    D -->|Send worker, item=A| W1[worker A]
    D -->|Send worker, item=B| W2[worker B]
    D -->|Send worker, item=C| W3[worker C]
    W1 --> Agg[aggregator]
    W2 --> Agg
    W3 --> Agg
    Agg --> End([END])

    style D fill:#fce7f3,stroke:#ec4899
    style W1 fill:#d1fae5
    style W2 fill:#d1fae5
    style W3 fill:#d1fae5
    style Agg fill:#dbeeaf

```

Map-Reduce 模式

Send 是 LangGraph 实现 Map-Reduce 的天然方式：

CODE

```
class State(TypedDict):
    items: list # 列表
    results: Annotated[list, operator.add] # Map 列表

def map_node(state):
    return [Send("worker", {"item": x}) for x in state["items"]]

def worker(state):
    # state 列表 Send 列表 payload {"item": ...}
    return {"results": [process(state["item"])]}

def reduce_node(state):
    final = aggregate(state["results"])
    return {"final": final}

graph.add_conditional_edges("start", map_node)
graph.add_node("worker", worker)
graph.add_edge("worker", "reduce")
graph.add_node("reduce", reduce_node)
```

并行节点的 Reducer 协作

Send 触发多个节点并行写 state，必须用支持累加的 Reducer：

Reducer	并行写安全？
默认（覆盖）	不安全 — 后写覆盖前写
<code>add_messages</code>	安全 — 自动 append
<code>operator.add</code>	安全 — 列表累加
自定义合并函数	取决于是否满足结合律和交换律

小结一行：子图把图当节点用、Send 动态扇出、Map-Reduce 是组合技 —— 节点 ⑦ 循环不再是单线程的 while。

§6.3 多智能体模式：Supervisor / Plan-Execute / Swarm

「在基础流程中的位置：节点 ② 推理升级为多个智能体协作。」

三种主流多智能体拓扑

模式	特征	适用
Supervisor	一个监督者 + N 个工作者，监督者决定调谁	客服 / 分流路由
Plan-Execute	先规划全部步骤，再串行执行	长任务 / 步骤明确
Swarm	任意智能体可"交接"给任意智能体（去中心化）	复杂角色协作

Supervisor 拓扑（Mermaid #15）

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Start([Start]) --> Sup[Supervisor]
    Sup --> R[Researcher]
    Sup --> W[Writer]
    Sup --> C[Critic]
    R --> Sup
    W --> Sup
    C --> Sup
    Sup --> End([End])

    style Sup fill:#fce7f3,stroke:#ec4899,stroke-width:3px
    style R fill:#dbeafe
    style W fill:#dbeafe
    style C fill:#dbeafe
```

CODE

```

def supervisor(state):
    llm_with_tools.invoke(state["messages"])
    if not decision.tool_calls:
        return Command(goto=END)
    next_agent = decision.tool_calls[0]["name"] # 
    return Command(update={"messages": [decision]}, goto=next_agent)

def researcher(state):
    result = research(...)
    return Command(update={"messages": [...]}, goto="supervisor")

graph = StateGraph(State)
graph.add_node("supervisor", supervisor)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_node("critic", critic)
graph.add_edge(START, "supervisor")

```

适用场景：客服分流、销售-工程-售后流转。

Plan-Execute 拓扑 (Mermaid #16)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    Start([Start]) --> P[Planner]
    P --> E1[Executor Step 1]
    E1 --> E2[Executor Step 2]
    E2 --> E3[Executor Step 3]
    E3 --> R[Re-planner]
    R --> P
    R --> End([End])

    style P fill:#fef3c7,stroke:#f59e0b,stroke-width:3px
    style P2 fill:#fef3c7,stroke:#f59e0b
    style E1 fill:#d1fae5
    style E2 fill:#d1fae5
    style E3 fill:#d1fae5

```


CODE

```
def planner(state):
    """
    1. Analyze the current state and identify the goal.
    2. Break down the goal into smaller, manageable steps.
    3. Return a list of steps to follow.
    """
    plan = llm.invoke([SystemMessage("你是一个规划师，请根据当前状态制定计划。"), state["input"]])
    return {"plan": parse_steps(plan)}

def executor(state):
    """
    1. Take the next step from the plan.
    2. Execute the step using the provided tools.
    3. Return the result of the step.
    """
    next_step = state["plan"][state["step_idx"]]
    result = run_step(next_step)
    return {"results": [result], "step_idx": state["step_idx"] + 1}

def is_done(state):
    """
    1. Check if the current step is the last step in the plan.
    2. Return True if done, False otherwise.
    """
    return state["step_idx"] >= len(state["plan"])
```

适用场景：研究助手、长程编程任务、复杂报告生成。

Swarm 拓扑 (Mermaid #17)

 [Mermaid 图表 — 请在 [HTML 中](#) 查看交互式渲染]

```
Start([? ?]) --> S[Sales Agent]
S -.handoff.-> T[Technical Agent]
T -.handoff.-> B[Billing Agent]
    B -.handoff.-> S
S -.?[? ?][? ?][? ?][? ?][? ?]End([? ?])
    T -.?[? ?][? ?][? ?][? ?][? ?]End
        B -.?[? ?][? ?][? ?][? ?][? ?]End

style S fill:#dbeafe
style T fill:#d1fae5
style B fill:#fef3c7
```

CODE

```
def sales_agent(state):
    response = llm_with_handoff_tools.invoke(state["messages"])
    if "transfer_to_technical" in tool_calls:
        return Command(goto="technical_agent")
    if "transfer_to_billing" in tool_calls:
        return Command(goto="billing_agent")
    return Command(update={"messages": [response]}, goto=END)
```

LangGraph 官方有独立包 `langgraph-swarm` 实现这套交接模型。

三种模式选型

维度	Supervisor	Plan-Execute	Swarm
决策中心	中央监督者	Planner	各智能体平等
控制流	树状	线性 + 重规划	网状
适用任务	流程明确分支多	步骤多需规划	角色协作复杂
调试难度	中	低	高
推荐起点	是	否（需求清晰再上）	否（先 Supervisor）

小结一行：先从 Supervisor 起步（覆盖 80% 场景），任务长且步骤明确上 Plan-Execute，复杂角色协作才上 Swarm。

§6.4 函数式接口：@entrypoint / @task（0.2 引入）

在基础流程中的位置：替代 StateGraph 的"装饰器风格"。

为什么有 Functional API

部分开发者觉得 `StateGraph + add_node + add_edge` 太"图论"，希望用更接近 Python 函数的方式写智能体。LangGraph 0.2 推出 Functional API 满足这个偏好。

用法

CODE

```

from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import MemorySaver

@task
def call_llm(messages: list) -> str:
    return llm.invoke(messages).content

@task
def call_tool(name: str, args: dict) -> str:
    return tools_map[name].invoke(args)

@entrypoint(checkpointer=MemorySaver())
def my_agent(messages: list, *, previous: list = None) -> list:
    history = previous or []
    history.extend(messages)

    while True:
        response = call_llm(history).result() # task Future
        history.append(response)
        if not has_tool_calls(response):
            break
        for tc in response.tool_calls:
            tool_result = call_tool(tc["name"], tc["args"]).result()
            history.append(tool_result)
    return history

result = my_agent.invoke([HumanMessage("...")])

```

Functional API 与 StateGraph 对比

维度	StateGraph	Functional API
心智模型	图（节点 + 边）	函数（带 task 调用）
控制流	显式（边）	隐式（Python if/while）
可视化	<code>draw_mermaid()</code>	不可可视化
调试	看每个超步 state	看 task Future
推荐	复杂多智能体	单 agent 主循环

previous 参数

Functional API 通过 `previous` 参数支持续跑：每次 `invoke` 时框架自动注入上次的返回值。这就是 Functional API 的"State"。

小结一行：Functional API 给函数式爱好者一个 `@entrypoint + @task` 的替代写法，但复杂场景仍推荐 StateGraph。

§6.5 长期记忆：Store API + BaseStore

「在基础流程中的位置：节点 ⑥ 状态管理的"跨会话"层。

Checkpoint vs Store 的层次

层	范围	典型存什么
State（节点间）	单次 invoke	工具调用历史
Checkpoint（会话内）	同 thread_id 的多次 invoke	长对话历史
Store（跨会话）	跨 thread_id	用户偏好、长期记忆、知识

Store API

CODE

```
from langgraph.store.memory import InMemoryStore
from langgraph.store.postgres import PostgresStore # ???

store = InMemoryStore()

# namespace 8888888888888888
namespace = ("user_memories", "user-123")

???
store.put(namespace, key="favorite_color", value={"color": "blue", "since": "2024"})
store.put(namespace, key="diet", value={"diet": "vegetarian"})

???
mem = store.get(namespace, "favorite_color")
print(mem.value) # {"color": "blue", "since": "2024"}

????
all_mems = store.search(namespace)
```

在图中使用 Store

CODE

```
from langgraph.graph import StateGraph
from langgraph.checkpoint.memory import MemorySaver

def call_model(state, *, store): # 1. 从 store 中获取上下文
    user_id = state["user_id"]
    namespace = ("user_memories", user_id)
    memories = store.search(namespace)

    context = "\n".join(m.value["text"] for m in memories)
    enriched_messages = [
        SystemMessage(f"2. 根据上下文回答问题，上下文: {context}"),
        *state["messages"],
    ]
    response = llm.invoke(enriched_messages)

    # 3. 将回答存入 store
    if "3. 将回答存入 store" in state["messages"][-1].content:
        store.put(namespace, key=str(uuid.uuid4()), value={"text": "..."})

    return {"messages": [response]}

# 4. 创建 store 并初始化 agent
store = MemorySaver()
agent = create_react_agent(llm, tools, store=store, checkpoint=checkpoint)
```

Store 后端

Store	用法
InMemoryStore	开发
PostgresStore	生产
自定义实现 BaseStore	接 MongoDB / Redis / 向量库

向量检索 Store（语义记忆）

CODE

```
from langgraph.store.memory import InMemoryStore
from langchain_openai import OpenAIEmbeddings

store = InMemoryStore(
    index={"embed": OpenAIEmbeddings(), "dims": 1536}
)
store.put(namespace, "mem1", {"text": "???????"})
store.put(namespace, "mem2", {"text": "?????"})

results = store.search(namespace, query="????")

```

小结一行：Store API 补 Checkpoint 的洞 —— 跨会话的长期记忆，结合 embedding 索引可实现语义召回。

§6.6 节点 ⑧ 流式输出五模式 + LangSmith 集成

「在基础流程中的位置：节点 ⑧ 输出格式化的体验关键。

五种 stream 模式对比

CODE

```
async for chunk in agent.astream(input, config, stream_mode="..."):
    print(chunk)
```

stream_mode	输出内容	何时用
values	每超步后的完整 state	看每步的全貌（默认）
updates	每节点的部分更新 dict	只关心变化
messages	LLM token 级流	前端打字机效果
debug	内部事件（include node start/end）	调试
custom	用 dispatch_custom_event 自己发的事件	业务自定义

values 模式

CODE

```
async for state in agent.astream(input, stream_mode="values"):
    print("state:", state)
```

updates 模式

CODE

```
async for update in agent.astream(input, stream_mode="updates"):
    # update {"node_name": {"messages": [...]}
    for node, data in update.items():
        print(f"node{node}data")
```

messages 模式（前端最常用）

CODE

```
async for msg, metadata in agent.astream(input, stream_mode="messages"):
    if isinstance(msg, AIMessage) and msg.content:
        print(msg.content, end="", flush=True) # token
```


custom 模式

CODE

```
from langgraph.config import get_stream_writer

def my_node(state):
    writer = get_stream_writer()
    writer({"progress": "loading", "step": 1}) # [?] [?] [?] [?] [?]
    [?] [?] [?] [?] [?] [?]
    writer({"progress": "loading", "step": 2})
    return {...}

[?] [?] [?] [?] [?]
async for event in agent.astream(input, stream_mode="custom"):
    print(event) # {"progress": "loading", "step": 1} [?]
```

多模式组合

CODE

```
async for chunk in agent.astream(
    input,
    stream_mode=["values", "messages"], # [?] [?] [?]
    [?] [?]
    if chunk[0] == "values":
    [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
    elif chunk[0] == "messages":
    [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
```

astream_events 看一切

CODE

```
async for event in agent.astream_events(input, version="v2"):
    if event["event"] == "on_chat_model_stream":
        print(event["data"]["chunk"].content, end="")
    elif event["event"] == "on_tool_start":
        print(f"\n[Tool] {event['name']}")
```

流式时序图 (Mermaid #18)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
sequenceDiagram
    participant U as 用户
    participant A as Agent
    participant L as LLM
    participant T as Tool

    U->>A: astream(input, mode=messages)
    A->>L: 111LLM
    L-->>A: token 1
    A-->>U: yield AIMessage chunk
    L-->>A: token 2
    A-->>U: yield AIMessage chunk
    L-->>A: tool_calls
    A->>T: 1111
    T-->>A: 1111
    A->>L: 111LLM
    L-->>A: token N
    A-->>U: yield AIMessage chunk
```

LangSmith 集成

CODE

```
export LANGSMITH_API_KEY=...
export LANGSMITH_TRACING=true
export LANGSMITH_PROJECT=my-langgraph-agent
```

启用后所有 invoke / stream 自动产生 trace —— 在 LangSmith 网页可看每个节点输入输出、耗时、token、报错。

小结一行：五种 stream 模式覆盖"看 state / 看 update / 看 token / 调试 / 业务"五种诉求，配合 LangSmith 是生产观测标配。

§6.7 节点 ④ 工具集成：@tool / ToolNode / MCP 适配器

在基础流程中的位置：节点 ④ 工具调用的现代实现。

@tool 装饰器：定义工具

CODE

```
from langchain_core.tools import tool

@tool
def search(query: str, top_k: int = 5) -> str:
    """Search for information about {query}"""
    return real_search(query, top_k)

"""Search for information about {query}"""
docstring = """JSON Schema"""
```

Pydantic 参数模型（复杂参数）

CODE

```
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(description="Search query")
    top_k: int = Field(default=5, ge=1, le=20)

@tool(args_schema=SearchInput)
def search(query: str, top_k: int = 5) -> str:
    """Search for information about {query}"""
    return real_search(query, top_k)
```

ToolNode：并发调用多工具

CODE

```
from langgraph.prebuilt import ToolNode

tool_node = ToolNode([search, calculator, weather])
graph.add_node("tools", tool_node)
```

ToolNode 在收到含多个 `tool_calls` 的 AIMessage 时**自动并发调用**，比串行调快。

工具错误处理

CODE

```
@tool
def safe_search(query: str) -> str:
    try:
        return real_search(query)
    except Exception as e:
        return f"[? ? ? ?] {str(e)[:200]}"

[? ? ? ?]ToolNode [? ?]
tool_node = ToolNode(
    tools,
    handle_tool_errors=True, # [? ? ?]True, [? ? ?]catch [? ?]ToolMessage
[?]
```

MCP 适配器：动态发现外部工具

模型上下文协议（Model Context Protocol, MCP）是 Anthropic 推出的工具发现协议。

`langchain-mcp-adapters` 让 LangGraph 可挂任意 MCP server 的工具：

CODE

```
from langchain_mcp_adapters.client import MultiServerMCPClient

async with MultiServerMCPClient({
    "filesystem": {
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path"],
        "transport": "stdio",
[? ? ? ? ? ?]
        "web_search": {
            "url": "http://localhost:8080/mcp",
            "transport": "streamable_http",
[? ? ? ? ? ?]
    }) as client:
    tools = await client.get_tools()
    agent = create_react_agent(llm, tools)
    result = await agent.ainvoke({"messages": [HumanMessage("...")]})
```

详见 §16.1 MCP 协议详解。

小结一行：@tool 是定义入口、ToolNode 是并发执行节点、MCP 适配器让外部工具生态接入——节点 ④ 在 LangGraph 时代生态友好。

§6.8 通道（Channels）深入

「在基础流程中的位置：节点 ⑥ Reducer 背后的实现机制。

Channel 是什么

LangGraph 内部把 State 的每个字段当成一个**通道**（Channel）—— 节点向通道写入"消息"，框架根据通道类型决定如何合并。

内置 Channel 类型

Channel 类	行为	默认用法
LastValue	覆盖式（最后写入胜出）	字段无 Annotated 时默认
Topic	节点内私有缓冲（不持久化）	不常用
BinaryOperatorAggregate	用二元运算符（如 +）累加	Annotated[list, operator.add]
EphemeralValue	临时通道，每超步清空	节点间一次性传

显式选 Channel

CODE

```
from langgraph.channels import LastValue, BinaryOperatorAggregate
import operator

class State(TypedDict):
    counter: Annotated[int, operator.add]      # ??
    user_id: Annotated[str, LastValue]        # ???????
    logs: Annotated[list, operator.add]       # ?????
```

自定义 Channel

CODE

```
from langgraph.channels import Channel

class MyMaxChannel(Channel):
    ?????????????????????
    def update(self, values):
        return max(values, default=self.value)
```

Channels 与 Pregel 超步的关系

Pregel 阶段	Channel 行为
超步开始	Channel 把累积写入合并成新值
节点读 state	节点看到合并后的值
节点写更新	写入对应 Channel 的"待合并"队列
超步屏障	等本超步所有节点写完

小结一行：Channels 是 Reducer 的实现层 —— 一般不需要直接用，但理解它有助于排查并行场景的 state 异常。

§6.9 Send vs add_conditional_edges 选择决策

「在基础流程中的位置：节点 ③ 决策路由的两种工具的选型。

决策表

维度	add_conditional_edges	Send
目标节点数	1 个（路由函数返回一个名字）	N 个（返回 Send 列表）
输入定制	共享 state	每个 Send 带独立 payload
执行模式	单线程	并行
用法语义	"选一条路走"	"扇出多个任务"

用 add_conditional_edges 的场景

- 客服意图分流（去 A 流程 / B 流程 / C 流程）
- 工具调用 / 直接回答的分支
- 失败重试的 retry 边

用 Send 的场景

- 对一组数据（list）每项跑相同处理（Map-Reduce）
- 多智能体并行调研
- 多检索源同时调（hybrid retrieval）

二者可以组合

小结一行：单选用 `add_conditional_edges`，扇出用 `Send` —— 二者背后机制不同，`Send` 在 Pregel 超步内并发跑。

「在基础流程中的位置：横切所有节点的可视化工具。」

LangGraph 团队的 desktop 应用 —— 把图可视化、可点击调试。本地启动后能:

功能	说明
看图结构	每个节点 / 边可视化
实时跑	输入触发后逐节点高亮
看 state 变化	每超步快照可看完整 state
时间旅行	拖拽回到任意 checkpoint 改 state 续跑
编辑 thread	直接改 messages 列表
HITL 控制	暂停 / 恢复 / 注入消息

启动方式

CODE

```

[?] [?] [?] [?] [?] [?] [?]
pip install langgraph-cli[inmem]

[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] langgraph.json [?] [?]
cat > langgraph.json << EOF

[?]

    "graphs": {
        "agent": ". /my_agent.py:agent"
    }
    [?] [?] [?] [?]
    "env": ".env"

[?]

EOF

[?] [?] [?] [?] [?] [?] [?]

langgraph dev

[?] [?] [?] [?] [?] [?] [?] [?] http://localhost:8123

```

Studio 与 LangSmith 的关系

维度	LangGraph Studio	LangSmith
部署	本地 desktop	云端（或自托管）
主要用途	开发 / 调试	生产追踪 / 评估
可视化	图结构 + 实时执行	trace 树
时间旅行	是（核心功能）	仅看不改
数据归属	本地	云

小结一行：Studio 是开发期"图调试器"，LangSmith 是生产期"飞行记录仪"，两者互补。

§6.11 LangGraph Server / Platform / Cloud 三种部署形态

「在基础流程中的位置：把图从本地脚本变成生产服务。

三种形态对比

形态	部署方式	适用
嵌入式（自己跑）	把 LangGraph 当库放进自己的 FastAPI / Flask	已有服务集成
LangGraph Server（自托管开源）	<code>langgraph dev</code> / Docker 起独立服务	想要 LangGraph 自带的 API
LangGraph Platform / Cloud	LangChain 商业托管	不想运维

LangGraph Server 自带的 API

`langgraph up` 启动后自动暴露 RESTful + SSE :

端点	作用
<code>POST /threads</code>	创建 thread（会话）
<code>POST /threads/{id}/runs/stream</code>	在 thread 上跑图（SSE 流式）
<code>GET /threads/{id}/state</code>	看当前 state
<code>POST /threads/{id}/state</code>	改 state（HITL）
<code>GET /threads/{id}/history</code>	看历史
<code>POST /assistants</code>	注册一个智能体配置

Docker 部署

CODE

```
langgraph build -t my-agent
docker run -p 8000:8000 my-agent
```

Platform / Cloud 的额外能力

能力	说明
自动扩缩容	按负载
后台任务（Cron）	内置调度器
多 assistant 管理	一个工程多个智能体
监控	集成 LangSmith
鉴权	API Key / OAuth
价格	按 thread / token 计

部署形态决策

CODE

```

LangGraph Server
LangGraph API + LangGraph Server
LangGraph Platform / Cloud
FastAPI
```

小结一行：嵌入式最灵活、Server 提供标准 API、Cloud 完全托管 —— 按数据合规和运维能力选。

§6.12 LangGraph SDK 客户端库

「在基础流程中的位置：跨语言调用 LangGraph Server。

Python SDK

CODE

```
from langgraph_sdk import get_client

client = get_client(url="http://localhost:8123")

???thread
thread = await client.threads.create()

?????
async for chunk in client.runs.stream(
    thread_id=thread["thread_id"],
    assistant_id="agent",
    input={"messages": [{"role": "user", "content": "..."}]},
    stream_mode="messages",
    ??
    print(chunk)
```

TypeScript SDK

CODE

```
import { Client } from "@langchain/langgraph-sdk";

const client = new Client({ apiUrl: "http://localhost:8123" });

const thread = await client.threads.create();
const stream = client.runs.stream(thread.thread_id, "agent", {
  input: { messages: [{ role: "user", content: "..."}] },
  streamMode: "messages",
  ???

  for await (const chunk of stream) {
    console.log(chunk);
    ?
```

何时用 SDK

场景	选择
同进程内调图	直接 <code>app.invoke</code>
跨服务调（Python）	Python SDK
前端调（Web）	TS SDK + 浏览器或 Node
多语言后端调	RESTful 直接调（任意语言）

小结一行：SDK 是 LangGraph Server / Platform 的客户端封装，跨服务 / 跨语言调图必备。

§6.13 后台任务 + 定时调度（Cron）

「在基础流程中的位置：节点 ⑦ 触发方式从"用户请求"扩展到"事件 / 定时"。

后台任务的定位

不是所有智能体都是"用户问 → 答" —— 有的需要：

场景	触发方式
每天凌晨扫描工单	Cron
用户上传文件后触发处理	Webhook / Event
监控告警触发自动诊断	Event
长任务后台跑（不阻塞用户）	后台

Cron 调度

LangGraph Platform / Server 内置 Cron：

CODE

```
???langgraph.json ?
{
  "graphs": {
    "agent": "./my_agent.py:agent"
  }
  "crons": [
    {
      "schedule": "0 9 * * 1",      // 每周一上午9点
      "graph_id": "agent",
      "input": {"messages": [{"role": "user", "content": "???"}]}
    }
  ]
}
```

或通过 SDK 动态创建：

CODE

```
await client.crons.create(
  cron="0 9 * * 1",
  payload={
    "assistant_id": "agent",
    "input": {"messages": [...]}
  }
)
```

后台任务（不阻塞用户）

CODE

```

??run, ??
run = await client.runs.create(
    thread_id=thread["thread_id"],
    assistant_id="agent",
    input={...},
)

status = await client.runs.get(thread["thread_id"], run["run_id"])

```

常驻智能体（Ambient Agent）

把后台 + Cron 组合起来 —— 做"持续工作的智能体"，详见 §17.2。

小结一行：Cron 和后台任务把 LangGraph 从"请求-响应"扩展到"持续运行"，是常驻智能体的基础设施。

§6.14 检查点的索引策略 + 容量管理

「在基础流程中的位置：节点 ⑥ 持久化的生产规模问题。」

Checkpoint 数据量估算

每超步写一个 checkpoint。100 轮对话 × 每轮平均 5 节点 = 500 checkpoint 行。

字段	大小估算
messages 列表（10 条 200 token 平均）	~20 KB
元数据 + 节点状态	~5 KB
单 checkpoint 总大小	~25 KB
100 轮对话累计	12 MB
1 万用户 × 平均 50 轮 = 50 万会话	6 TB

Postgres 索引建议

CODE

```
-- LangGraph 表结构
CREATE INDEX checkpoints_thread_id_idx ON checkpoints(thread_id);
CREATE INDEX checkpoints_thread_id_step_idx ON checkpoints(thread_id, step);

-- 按线程ID和创建时间排序
CREATE INDEX checkpoints_thread_id_created_at_idx
ON checkpoints(thread_id, created_at DESC);

-- 分区表
CREATE TABLE checkpoints_2026_01 PARTITION OF checkpoints
FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
```

TTL / 清理策略

策略	实施
删除 N 天前 checkpoint	Cron 定期 DELETE
仅保留每会话最近 K 个	按 thread_id 取 top K, 删其余
归档到冷存储	移到 S3 后删主库

CODE

```
truncate table checkpoints;
DELETE FROM checkpoints WHERE created_at < NOW() - INTERVAL '30 days';
```

大字段处理

State 里塞了大文档（PDF 全文 / 大图片 base64）会让 checkpoint 超大。正确做法：

错误	正确
把 PDF 文本塞进 state	把 PDF 存 S3，state 存 URL
把图片 base64 塞进 messages	用图片 URL（厂商支持）
把整个数据库结果塞进 state	存 ID 列表，需要时按 ID 查

小结一行：Checkpoint 是 LangGraph 的命脉，但要按规模做索引、TTL 和大字段外挂 —— 不然数据库会被吃光。

§6.15 混合架构：LangChain 组件 + LangGraph 编排（业界默认答案）

在基础流程中的位置：业界对 LangChain / LangGraph 关系的真实定位。

不是"二选一"，是"分层用"

节点	用 LangChain 组件	用 LangGraph 编排
① 上下文	<code>ChatPromptTemplate</code> / <code>Document Loader</code> / <code>Text Splitter</code>	节点函数自由组装
② 推理	<code>init_chat_model</code> / <code>llm.bind_tools</code>	LLM 节点
③ 决策	—	条件边
④ 工具	<code>@tool</code> 装饰器	<code>ToolNode</code>
⑤ 返回	—	写回 State
⑥ 状态	(老 Memory 已被替代)	State + Checkpoint + Store
⑦ 循环	—	<code>recursion_limit</code> + <code>interrupt</code>
⑧ 输出	<code>with_structured_output</code> / <code>OutputParser</code>	出口节点 + 流式

典型混合架构代码

CODE

```
from langchain_openai import init_chat_model
from langchain_core.tools import tool
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode

# LangChain [?][?]
llm = init_chat_model("gpt-4o").bind_tools(tools)

@tool
def my_tool(...):
    [?][?][?][?][?][?]

# LangGraph [?][?]
graph = StateGraph(State)
graph.add_node("agent", lambda s: {"messages": [llm.invoke(s["messages"])]})
graph.add_node("tools", ToolNode([my_tool]))
graph.add_conditional_edges("agent", should_continue, {...})
graph.add_edge(START, "agent")
graph.add_edge("tools", "agent")
agent = graph.compile()
```

业界采用比例（粗估）

玩法	比例
LangChain 组件 + LangGraph 编排	~60%
纯 LangGraph（不用 LangChain 组件）	~15%
纯 LangChain（v0.3 新法）	~10%
不用任何框架（直接 SDK）	~15%

小结一行：混合架构是业界默认答案 —— LangChain 当组件库（不重新造轮子）+ LangGraph 当编排引擎（图式状态机）。

§6.16 本章小结

#	核心结论
1	State 用 TypedDict + add_messages 是默认配方，Reducer 控制合并语义，Command 同时更新 + 路由
2	子图把图当节点用、Send 动态扇出、Map-Reduce 是组合技 —— 节点 ⑦ 不再是单线程 while
3	多智能体三模式：先 Supervisor 起步、任务长上 Plan-Execute、复杂角色协作上 Swarm
4	Functional API (@entrypoint / @task) 给函数式爱好者一个替代写法
5	Store API 补 Checkpoint 跨会话失忆的洞，结合 embedding 索引可语义召回
6	五种 stream 模式 + LangSmith 是生产观测标配
7	工具集成：@tool / ToolNode / MCP 适配器 三件套
8	LangGraph Studio（开发期）+ LangSmith（生产期）= 完整可观测性
9	三种部署形态：嵌入 / Server 自托管 / Platform Cloud
10	混合架构（LangChain 组件 + LangGraph 编排）是业界默认答案，约 60% 项目采用

§6.17 反模式速记

反模式	错在哪	正确做法
State 字段无 Reducer 时 Send 并行写	后写覆盖前写	必须用 <code>operator.add</code> 或 <code>add_messages</code>
把 PDF 全文塞 messages	Checkpoint 爆大	存 URL 引用
用 Swarm 跑简单任务	调试地狱	先 Supervisor
不开 stream 模式跑 60 秒任务	用户体验差	用 <code>stream_mode=messages</code> 流式
Postgres checkpoint 不分区 / 不 TTL	表越来越大	加分区 + Cron 删 30 天前
Studio + LangSmith 都不开	调试黑盒	开发用 Studio, 生产开 LangSmith

§6.18 术语速查

术语	中文	含义
Reducer	归约器	State 字段的合并规则
<code>add_messages</code>	消息追加器	自动追加 + 去重的 reducer
<code>operator.add</code>	累加	Python 内置 reducer
Command	命令对象	同时更新 state + 路由
Subgraph	子图	一张图作为另一张图的节点
<code>Send</code>	发送原语	动态扇出多个并行子任务
Map-Reduce	映射-归约	经典并行计算模式
Supervisor	监督者	中央决策的多智能体模式
Plan-Execute	计划-执行	先规划后串行执行
Swarm	蜂群	去中心化交接的多智能体
Functional API	函数式接口	@entrypoint / @task 装饰器
<code>BaseStore</code> / Store	跨会话存储	Checkpoint 之上的长期记忆层
Channel	通道	Reducer 的实现层
LangGraph Studio	可视化调试器	桌面应用
LangGraph Server	服务器形态	自托管的 LangGraph 服务
LangGraph Platform / Cloud	商业托管	LangChain 公司的 SaaS
LangGraph SDK	客户端 SDK	Python / TS 的客户端库
MCP (Model Context Protocol)	模型上下文协议	Anthropic 推出的工具协议

术语	中文	含义
langchain-mcp-adapters	MCP 适配器	把 MCP 工具接入 LangGraph

§6.19 推荐下一章

下一章: [§7 三者深度对比 + 业界主流玩法](#) —— 把 ReAct / LangChain / LangGraph 三种实现拉通，做横向 PK 和选型决策。

§7 三者深度对比 + 业界主流玩法

§7.0 本章定位

项	内容
在基础流程中的位置	把 8 个节点拉通，三种实现并排比较
与上下章的因果链	三件套讲完，本章做横向 PK 和选型决策。下一章 §8 把决策落到用户工程
学完能做什么	(1) 同任务三实现并列对比；(2) 8 节点逐节点对照；(3) 选型决策树；(4) 成本 / 延迟 / 测试 / 错误处理 / 迁移路径多维对比

§7.1 同任务三种实现并列：纯 ReAct prompt / LangChain / LangGraph

「在基础流程中的位置：把 8 节点的三种实现并列对照。

任务定义

带搜索工具的客服智能体：用户问"昨天我下单的那个 Sony WH-1000XM5 现在到哪了？"，智能体要： 1. 用搜索工具查订单 2. 用搜索工具查物流 3. 综合回答

实现 1: 纯 ReAct prompt (无框架)

CODE

```

from openai import OpenAI SDK
import openai, json

REACT_PROMPT = """
- search_order(query): 
- search_logistics(order_id): 

Thought: <>
Action: <>
Action Input: <JSON >
Observation: <>
Thought: 
Final Answer: <>

question
"""

def search_order(query): return f"ID 12345, Sony WH-1000XM5"
def search_logistics(order_id): return f"SF1234, "
TOOLS = {"search_order": search_order, "search_logistics": search_logistics}

def react_loop(question, max_steps=8):
    history = REACT_PROMPT.format(question=question)
    for _ in range(max_steps):
        response = openai.ChatCompletion.create(
            model="gpt-4o",
            messages=[{"role": "user", "content": history}],
            stop=["Observation:"],
        ).choices[0].message.content

        history += response
        if "Final Answer:" in response:
            return response.split("Final Answer:")[1].strip()

    action = response.split("Action:")[1].split("\n")[0].strip()
    action_input = json.loads(response.split("Action Input:")[1].split("\n")[0].strip())

    observation = TOOLS[action](**action_input)
    history += f"\nObservation: {observation}\n"
    return "[ ]"

print(react_loop("Sony WH-1000XM5 "))

```

实现 2: LangChain Tool Calling Agent

CODE

```

"""LangChain 工具调用 Agent 实现"""
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate

@tool
def search_order(query: str) -> str:
    """根据查询 ID 查找订单信息"""
    return f"订单 ID 12345, 产品: Sony WH-1000XM5"

@tool
def search_logistics(order_id: str) -> str:
    """根据订单 ID 查找物流信息"""
    return f"订单 ID {order_id} 物流单号 SF1234, 预计送达日期 2024-05-20"

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个智能助手，负责处理用户的查询。"),
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}"),
])

llm = ChatOpenAI(model="gpt-4o")
tools = [search_order, search_logistics]

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=8)
result = executor.invoke({"input": "Sony WH-1000XM5 耳机物流信息"})
print(result["output"])

```

实现 3: LangGraph create_react_agent

CODE

```

"""LangGraph 实现 create_react_agent"""
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

@tool
def search_order(query: str) -> str:
    """根据查询返回订单信息"""
    return f"订单ID 12345, Sony WH-1000XM5"

@tool
def search_logistics(order_id: str) -> str:
    """根据订单ID返回物流信息"""
    return f"订单ID {order_id} SF1234, 物流信息"

agent = create_react_agent(
    ChatOpenAI(model="gpt-4o"),
    [search_order, search_logistics],
    prompt="你是一个物流助手，请根据用户查询提供订单和物流信息。",
    checkpoint=MemorySaver(), # 启用checkpoint
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Sony WH-1000XM5 物流信息"}]},
    config={"configurable": {"thread_id": "user-123"}},
)

print(result["messages"][-1].content)

```

行数 / 特性 对比

维度	纯 ReAct	LangChain	LangGraph
代码行数	80	30	20
工具描述	手拼 prompt	@tool docstring	@tool docstring
路由解析	字符串正则	黑盒 OutputParser	字段读取
Checkpoint	不支持	不支持	一行加
HITL	不支持	不支持	一行加
流式	难	中等	一行加
调试	看 prompt 拼接	verbose=True	astream_events 全可见

小结一行：同任务 LangGraph 行数最少、能力最全 —— 这就是为什么业界主流。

§7.2 8 节点逐节点对比表（核心对照）

本节是全书最关键的总览表。

节点	纯 ReAct	LangChain	LangGraph
① 上下文 组装	手写 prompt 模板 + 拼字符串	<code>ChatPromptTemplate</code> + <code>MessagesPlaceholder</code>	节点函数自由组装 + State 注入
② 大模型 推理	直接调 OpenAI API	<code>ChatModel</code> + <code>bind_tools</code>	LLM 节点 + 任意函数
③ 决策 路由	正则提取 <code>Action:</code> <code><name></code>	<code>AgentExecutor</code> 黑盒	显式 <code>add_conditional_edges</code>
④ 工具 调用	字符串解析后 查字典调用	<code>@tool</code> + <code>AgentExecutor</code> 内部调	<code>@tool</code> + <code>ToolNode</code> (并发)
⑤ 工具 返回	拼回 prompt history	<code>AgentExecutor</code> 内部处理	写回 <code>state.messages</code> (auto)
⑥ 状		<code>Memory</code> (4 类, 会话级)	<code>State</code> + <code>Checkpoint</code> + <code>Store</code>

节点	纯 ReAct	LangChain	LangGraph
态管理	全在 prompt 里（爆炸）		
⑦ 循环控制	for 循环 + max_steps	<code>max_iterations</code> 硬限	<code>recursion_limit</code> + <code>interrupt</code> + 条件边
⑧ 输出格式化	提取 <code>Final Answer:</code>	<code>OutputParser</code> / <code>with_structured_output</code>	出口节点 + 5 种 stream 模式

核心结论

节点	纯 ReAct → LangChain 改善点	LangChain → LangGraph 改善点
①	模板抽象化	模板不再绑死、状态可注入
②	API 厂商封装	任意函数节点 + bind_tools
③	字段化（不再字符串）	黑盒 → 显式条件边
④	装饰器统一	并发 + MCP
⑤	自动处理	显式 State
⑥	Memory 模块	Checkpoint + Store 跨会话
⑦	max_iterations	interrupt + 时间旅行
⑧	OutputParser	5 种 stream 模式

小结一行：每个节点都能看到从纯 ReAct → LangChain → LangGraph 的演化轨迹，节点③⑥⑦改善最显著。

§7.3 选型决策树 (Mermaid #19)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Start([Start]) --> WF[WF: Airflow / Prefect]
    Start --> Q1((Q1: RAG))
    Q1 --> Q2((Q2: LangChain LCEL + Retriever))
    Q2 --> SDK[SDK: OpenAI/Anthropic SDK]
    SDK --> Q3((Q3: HITL / LangGraph))
    Q3 --> LG[LangGraph + Checkpointer]
    Q3 --> LCT[LangChain create_tool_calling_agent]
    LG --> LG
    LCT --> LG

    style WF fill:#fef3c7
    style SDK fill:#dbeafe
    style LC fill:#d1fae5
    style LCT fill:#fce7f3
    style LG fill:#e9d5ff,stroke-width:3px
```

决策维度优先级

1. **首选**：直接 SDK + 自己写状态机（如果团队能力够）
2. **简单 RAG**：LangChain LCEL
3. **简单 Agent（无中断、单会话）**：LangChain create_tool_calling_agent（短期可用）
4. **生产 Agent**：LangGraph
5. **多智能体协作**：LangGraph Supervisor / Plan-Execute

小结一行：决策树以"是否需要状态/循环/中断"为分水岭，超过这条线必须 LangGraph。

§7.4 多维对比表

维度	纯 ReAct	LangChain	LangGraph
学习曲线	易（看一篇博客）	中（看完官方文档）	中-难（图思维 + Pregel）
生态	无	大量 Loader / 集成	集成 LangChain 的全部
可调试性	看 prompt	verbose / LangSmith	Studio + LangSmith + 时间旅行
可维护性	差（脆弱字符串解析）	中（黑盒难追问）	好（显式图）
性能	一般（无并发）	一般	好（Pregel 自动并行）
生产级稳定性	差	中	好
招聘市场	无	多人会	渐成主流
Anthropic 推荐	不推	"看情况"	推（自家 docs 用）
2026 趋势	学习用	当组件库	当编排引擎

小结一行：六个维度看 LangGraph 综合最优，但有学习曲线 —— 团队投入应匹配项目复杂度。

§7.5 成本对比：token / 调用费用

同任务下三种实现的 token 消耗差异在 2-3 倍量级 —— 因为 ReAct 字符串格式比结构化 tool_calls 字段冗长得多。开 Prompt Caching 后还能再降一半。

同任务 token 消耗实测（粗估）

维度	纯 ReAct	LangChain	LangGraph
Prompt 模板大小	大（手拼，含格式约束）	中（框架优化）	中
每轮 token 累积	高（Thought/Action/Observation 全塞）	中	中（messages 列表）
工具调用解析	模型输出 50-100 token 字段	模型直接结构化 JSON（10-30 token）	同 LangChain
单次调用 token	~3000 (3 步循环)	~2000	~2000

结论：LangChain / LangGraph 比纯 ReAct 省 30-40% token —— 因为用结构化 tool_calls 字段而不是 Thought/Action/Observation 字符串。

调用次数

实现	LLM 调用次数
纯 ReAct (3 工具)	3 次 (每步一个 LLM 调用)
LangChain (多工具并发)	同
LangGraph (用 Send 并发工具)	仍是每个工具调用前 LLM 决策一次, 但 ToolNode 并发执行多工具, 总耗时下降

一次复杂任务的成本估算

10 步 ReAct 任务用 GPT-4o :

实现	输入 token	输出 token	成本 (美元)
纯 ReAct	~50k	~10k	~\$0.40
LangChain	~30k	~6k	~\$0.24
LangGraph	~30k	~6k	~\$0.24
LangGraph + Prompt Caching	~10k (命中缓存)	~6k	~\$0.10

Prompt Caching (\$17.5) 能让成本进一步降 50%-70%。

小结一行：现代框架 (LangChain/LangGraph) 比纯 ReAct 省 30-40% token, 加上 Prompt Caching 能再降一半。

§7.6 延迟对比：串行 / 并行 / 流式

实现	串行延迟 (10 步任务)	并行延迟	流式 TTFT (首 token)
纯 ReAct	~20s	不支持	不支持
LangChain	~15s	不支持 (AgentExecutor 串行)	中
LangGraph	~15s	~6s (Send 并发)	快 (stream_mode=messages)

TTFT (Time To First Token) 的重要性

用户体验关键指标。LangGraph 流式让用户在 200-500ms 就看到第一个字，体验从"等 15 秒"变"看打字"。

并发收益

CODE

```
# Send 10 个任务并行
async for chunk in agent.astream({"messages": [...]} , stream_mode="messages"):
    print(chunk)
10 个任务串行 10s → 3 个任务并行 max(2s)=2s
```

小结一行：LangGraph 在并发场景比 LangChain 快 2-3 倍，流式体感差异巨大。

§7.7 测试策略对比

测试层	纯 ReAct	LangChain	LangGraph
单元测试（纯逻辑）	工具函数单测	工具单测 + Runnable 单测	工具单测 + 节点函数单测
集成测试（链路）	mock LLM 跑全流程	mock LLM + executor.invoke	mock LLM + app.invoke
端到端测试	真 API 跑用例	同	同
评估（语义正确性）	看输出对不对	LangSmith eval	LangSmith eval
时间旅行测试	不支持	不支持	支持 （从某 checkpoint 回放）

LangGraph 节点函数测试示例

CODE

```
def test_router_node():
    state = {"messages": [HumanMessage("???")]}
    result = should_continue(state)
    assert result == "tools"

def test_llm_node():
    fake_llm = FakeListLLM(responses=["{'tool_calls': [...]}"])
    state = {"messages": [HumanMessage("...")]}
    result = call_model(state)
    assert "tool_calls" in result["messages"][0]
```

评估对比表

工具	适用
LangSmith Eval	LangChain / LangGraph 原生
Promptfoo	跨框架 / yaml 配置
DeepEval	pytest 风格
Inspect AI	严肃评估

小结一行：LangGraph 节点函数 + 时间旅行让单元测试和回放测试更友好。

§7.8 错误处理与重试策略对比

错误层	纯 ReAct	LangChain	LangGraph
LLM 限流	自己写重试	<code>with_retry</code>	节点内 try + retry 边
工具异常	自己 catch	<code>handle_tool_errors=True</code>	节点内 try + 路由到 retry 节点
解析错误	自己写正则补丁	<code>OutputFixingParser</code>	不存在（结构化字段）
死循环	自己设 max_steps	<code>max_iterations</code>	<code>recursion_limit</code>
厂商宕机	手动切	<code>with_fallbacks</code>	节点内切
局部失败可重跑	不支持	不支持	支持（从 checkpoint 续跑）

小结一行：LangGraph 把错误处理粒度细化到节点级 + 提供"从断点恢复"能力。

§7.9 版本与迁移：从 LangChain v0.x → LangGraph 0.x

「迁移核心是把 `AgentExecutor` 替换成 `create_react_agent`，多数项目能一行替换；复杂场景再扩展为完整 StateGraph。详见附录 J 完整速查表。

老 LangChain 项目的现实选择

项目状态	推荐
仍在用 v0.0.x（2023 老代码）	必须升 v0.3 以上，否则维护困难
用 v0.1-v0.2 但仅简单链	可以不动
用 AgentExecutor 跑生产	强烈建议迁 LangGraph
用 LCEL + with_structured_output	OK，可继续

迁移步骤

步骤	老 LangChain	新 LangGraph
1	<code>from langchain.agents import AgentExecutor</code>	<code>from langgraph.prebuilt import create_react_agent</code>
2	<code>executor = AgentExecutor(agent, tools)</code>	<code>agent = create_react_agent(llm, tools)</code>
3	<code>result = executor.invoke({"input": q})</code>	<code>result = agent.invoke({"messages": [{"role": "user", "content": q}]})</code>
4	加 Memory	加 <code>checkpoint=MemorySaver()</code>
5	复杂逻辑	改写成 StateGraph

迁移踩坑

坑	解法
<code>chat_history</code> 字段名变了	用 <code>messages</code>
<code>agent_scratchpad</code> 不存在了	LangGraph 内部管理
Memory 类直接挂不行	用 <code>checkpointner</code> 替代
AgentExecutor 配置项不对应	<code>recursion_limit</code> / <code>interrupt_before</code> 替代

详见附录 J 完整迁移速查表。

小结一行：迁移核心是把 AgentExecutor 替换成 `create_react_agent`，复杂场景再扩展为 StateGraph。

§7.10 2026 年初业界真实选型快照

公司 / 团队	选择	公开信息
Klarna	LangGraph	客服智能体核心，处理 2/3 流量
Replit	LangGraph	Replit Agent 编程助手
LinkedIn	LangGraph	招聘助手 SQL Bot
Elastic	LangGraph	客服 + 内部知识
Uber	LangGraph	Developer Platform / Code Reviewer
Anthropic 自家	直接 SDK	Claude Code / 内部工具
Cursor / Windsurf	闭源（推测自研）	编程智能体
OpenAI	OpenAI Agents SDK	自家产品（2024.10 推）
国内大厂	Dify / Coze / 自研	多元
学术 / Notebook	LangChain v0.3	简单 RAG 演示

选型逻辑

情境	选择逻辑
团队 ≤ 5 人 / 简单任务	LangChain 或直接 SDK
团队 ≥ 10 人 / 严肃生产	LangGraph + LangSmith
不想被框架绑架	直接 SDK + 手写图（Anthropic 风格）
只用 OpenAI 生态	可考虑 OpenAI Agents SDK
国内 + 数据合规	Dify / Coze / 自研

小结一行：业界主流是 LangGraph + LangSmith，但"不用框架"派也有存在感（尤其 Anthropic 自家）。

§7.11 何时不该用三件套

场景	该用什么
业务流程预定义、不需要 LLM 自主决策	Airflow / Temporal / Prefect 等传统工作流
一次性 Prompt + 一次返回	直接 OpenAI / Anthropic SDK
简单 RAG（一轮检索 + 答）	LlamaIndex（更专注 RAG）
编程类智能体	Claude Code / Cursor / Aider 等专门工具
国内 / 合规要求严	Dify / Coze / 自研
团队全 TS	Vercel AI SDK / Mastra
评估 / 提示词管理工作流	Promptfoo / Braintrust

小结一行：三件套适合"通用智能体场景"，垂直领域有更专门的工具链。

§7.12 本章小结

#	核心结论
1	同任务三种实现：纯 ReAct 80 行、LangChain 30 行、LangGraph 20 行 —— LangGraph 行数最少能力最全
2	8 节点逐节点对比中，节点 ③⑥⑦ 改善最显著（黑盒 → 显式 / 会话级 → 跨会话 / max_iterations → interrupt）
3	选型决策核心分水岭：是否需要状态 / 循环 / 中断 —— 超过这条线必须 LangGraph
4	现代框架（LC/LG）比纯 ReAct 省 30-40% token，加 Prompt Caching 再降一半
5	LangGraph 在并发场景比 LangChain 快 2-3 倍，流式 TTFT 体感最优
6	业界主流选型：LangGraph + LangSmith，但"不用框架"派也存在
7	7 类垂直场景（工作流 / 简单 RAG / 编程 Agent / 合规等）有比三件套更专门的选择

§7.13 反模式速记

反模式	错在哪	正确做法
简单 prompt 任务也上 LangGraph	杀鸡用牛刀	直接 SDK
长会话用 LangChain Memory	跨进程丢	上 LangGraph Checkpointer
不评估直接上生产	不知好坏	先 LangSmith eval / Promptfoo
不开 Prompt Caching	多花 50% 钱	必开 (§17.5)
选 Swarm 当起步	调试地狱	先 Supervisor

§7.14 术语速查

术语	中文	含义
TTFT (Time To First Token)	首字延迟	用户从发起到看到第一个字的耗时
Prompt Caching	提示词缓存	厂商对相同 prompt 前缀缓存，省 token
LangSmith Eval	LangSmith 评估	LangSmith 内置的评估工具
Promptfoo	Promptfoo	跨框架的 prompt 评估工具

§7.15 推荐下一章

下一章：[§8 用三件套落地 custom_agent（实现视角）](#) —— 把通用结论具体到当前工程，给出接入设计草案。

§8 用三件套落地 custom_agent（实现视角）

§8.0 本章定位

项	内容
在基础 流程中的 位置	把 8 节点流程映射到 custom_agent 工程的具体模块
与上下 章的因 果链	§7 给了通用选型结论，本章具体到用户工程；下一章 §9 起 Part B 拉开生态视角
学完能 做什么	(1) 知道 custom_agent 现有架构；(2) 选三种接入点中的合适层；(3) 设计 Sessions ↔ thread_id 映射；(4) 设计 Skills ↔ LangGraph 节点的关系；(5) 列渐进迁移 3 步路径；(6) 知道当前不必上的判断条件

§8.1 现有架构速览

「声明：以下基于公开仓库结构推测。实际实现以代码为准。」

三层结构

层	路径	职责（推测）
Web Console（前端）	apps/web-console	Next.js 用户界面、聊天面板、知识库面板、技能面板
API Server（后端 API）	services/api-server	FastAPI，提供 sessions / chat / kb / skills / api_keys 接口
Gateway（网关）	services/gateway	流量路由、鉴权、限流（推测）

已有的核心概念

从仓库结构看到：

概念	文件	当前形态
会话（Sessions）	api-server/db/chat.py	数据库表
知识库（KB）	api-server/db/kb.py	数据库 + RAG
技能（Skills）	api-server/db/skills.py	数据库 + Skill 配置
工作区（Workspaces）	api-server/db/workspaces.py	多租户单元
API Keys	api-server/db/api_keys.py	鉴权

小结一行：custom_agent 是 FastAPI 后端 + Next.js 前端 + 网关三层架构，已有 sessions / KB / skills / workspaces 四个核心领域概念。

§8.2 8 节点流程到用户工程模块的映射表

基础流程节点	custom_agent 现有模块（推测）	待引入（用 LangGraph）
① 上下文组装	routes/chat.py 拼 messages	LangGraph 节点函数从 State 取
② 大模型推理	LLM 客户端调 OpenAI/Anthropic	<code>bind_tools(llm)</code>
③ 决策路由	暂无（一次直答）	条件边 <code>add_conditional_edges</code>
④ 工具调用	Skills 系统	<code>ToolNode</code> + Skills 适配器
⑤ 工具返回	Skills 调用结果	写回 State
⑥ 状态管理	Sessions 数据库	<code>PostgresSaver</code> Checkpoint
⑦ 循环控制	暂无	<code>recursion_limit</code> + <code>interrupt</code>
⑧ 输出格式化	routes/chat.py SSE	<code>astream</code> messages 模式

小结一行：现有工程在节点 ①②④⑤⑥⑧ 已有基础（拼 prompt / 调 LLM / 调 Skills / 存会话 / SSE），缺节点 ③⑦（决策路由 + 循环控制）—— 这是 LangGraph 该补的位置。

§8.3 接入点决策矩阵

三个候选接入点的核心权衡：**实施速度 vs 长期可扩展性**。MVP 阶段优先速度，业务复杂后迁独立服务。

三个候选接入点

接入点	实现	优点	缺点
网关层	<code>services/gateway</code> 内嵌 LangGraph	流量入口统一	gateway 应该薄、不该承载业务逻辑
接口服务层	<code>services/api-server</code> 内嵌 LangGraph	与现有 sessions / Skills 同进程，最方便	api-server 变重
独立智能体服务	新建 <code>services/agent-runtime</code>	独立扩缩容、独立部署	多一个服务、跨服务调用

决策矩阵

维度	网关层	api-server 内嵌	独立 agent service
实施速度	不推荐	快（1-2 周）	慢（4-6 周）
可观测性	中	中	好（独立日志 / 监控）
运维成本	中	低（同一服务）	高
扩缩容	受 gateway 限制	与 api-server 绑定	独立
推荐起步	否	是	第二阶段

推荐路径

- 1. **MVP**: 内嵌 `api-server`，新增 `routes/agent.py` 路由调用 LangGraph
- 2. **业务复杂后**: 抽出独立 `services/agent-runtime`，api-server 转为代理
- 3. **网关层**: 始终保持薄

小结一行：MVP 阶段建议内嵌 api-server（最快），业务起来后再抽独立 agent service。

§8.4 与现有 RAG / Skills / Sessions / KB 的关系

「 现有 4 个核心概念（Sessions / KB / Skills / Workspaces）——对应到 LangGraph 的概念（thread_id / RAG 节点 / @tool / checkpoint_ns），无需推倒重做。

集成架构（Mermaid #20）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    User[User Web Console] --> Gateway[Gateway]
    Gateway --> API[api-server FastAPI]

    subgraph API_INNER [api-server]
        Routes["/chat"]
        LG[LangGraph CompiledGraph]
        Routes --> LG
    end

    subgraph Nodes [LangGraph]
        N1["agent LLM"]
        N2["tools ToolNode"]
        N3["rag KB"]
        N1 --> N2
        N2 --> N1
        N1 -. " " .-> N3
        N3 --> N1
    end

    LG --> Nodes

    LG <--> CP["PostgresSaver Checkpoint"]
    LG <--> ST["PostgresStore"]
    Nodes --> KB["KB"]
    Nodes --> Skills["Skills ToolNode"]
    Nodes --> Sessions["Sessions thread_id"]

    style LG fill:#e9d5ff,stroke:#a855f7,stroke-width:3px
    style CP fill:#fef3c7
    style ST fill:#fef3c7
```

各模块的角色变化

现有模块	接入 LangGraph 后的角色
Sessions 表	与 LangGraph thread_id 一一映射，Sessions.id 即 thread_id
KB（知识库）	包装成 RAG 节点（节点 ① 增强）
Skills	每个 Skill 包装成 @tool，挂到 ToolNode
Workspaces	与 LangGraph checkpoint_ns（命名空间）映射，做多租户隔离
API Keys	用于 LangGraph Server 的鉴权（如果走独立部署）

小结一行：现有概念无需推倒重做 —— Sessions ↔ thread_id、Skills ↔ tool、Workspaces ↔ checkpoint_ns、KB ↔ RAG 节点四个映射建好就能接入。

§8.5 渐进式迁移路径：3 步走

第 1 步：纯 SDK 升级到 LCEL（不引 LangGraph）

改动	说明
routes/chat.py 用 init_chat_model + ChatPromptTemplate	比直接 SDK 抽象一层
用 bind_tools 让模型能调 Skills	节点 ④ 标准化
用 with_structured_output 做结构化回复	节点 ⑧ 标准化
加 LangSmith trace	观测

收益：LCEL 化、可观测，但架构本质未变。

第 2 步：加 ReAct 范式（仍不引 LangGraph）

改动	说明
用 <code>langchain.agents.create_tool_calling_agent</code>	让模型能多步调工具
Skills 全部包装成 <code>@tool</code>	工具系统对接
加 verbose / LangSmith	调试

收益：模型能多步调 Skills 完成任务，但仍是会话级、无 checkpoint、无 HITL。

第 3 步：上 LangGraph（生产级）

改动	说明
把第 2 步的 AgentExecutor 替换为 <code>create_react_agent</code>	一行迁移
加 <code>PostgresSaver</code> ，Sessions.id → thread_id	跨会话续跑
加 <code>interrupt_before=["tools"]</code> （按需）	HITL
加 <code>astream_events</code> 给前端流式	体验提升
加 <code>Store</code> 长期记忆	用户偏好持久化

收益：完整生产能力。

```

[?] [?] [?] [?] [?] [?] [?] SDK [?]
[?] [?] [?] [?] [?] [?] [?] [?] [?]
Step 1: LCEL [?] [?] [?] [?] LangSmith
[?] [?] [?] [?] [?] [?] [?] [?] [?]
Step 2: ReAct + Tool Calling Agent
[?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?]
Step 3: LangGraph + Checkpoint + HITL
[?] [?] [?] [?] [?] [?] [?] [?] [?]
[?] [?] [?] [?] Send [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] [?] Store

```

§8.6 部署形态四选一

推荐顺序

- 第 312 页

- 2. 业务起来：容器化独立部署
- 3. 可选：上 Platform / Cloud（如果数据合规允许）

小结一行：先嵌入最简单，业务起来再独立 —— 别一开始上 Platform / Cloud。

§8.7 Sessions 与 thread_id 映射设计

一对一映射

CODE

```
session = create_session(workspace_id="w1", user_id="u1")
session.id = "session_abc123"

# LangGraph
config = {"configurable": {
    "thread_id": session.id,
    "checkpoint_ns": session.workspace_id,
    "user_id": session.user_id,
}}
result = agent.invoke({"messages": [...]}, config=config)
```

Schema 兼容设计

现有 Sessions 表无须改，新加 `langgraph_enabled: bool` 字段：

Session.langgraph_enabled	行为
False（兼容老逻辑）	走老的直接 SDK 路径
True	走 LangGraph 路径

灰度发布时只对一部分 session 开。

历史消息的迁移

老 Sessions 表里的历史 messages 怎么"接"到 LangGraph？

方案	说明
写入 LangGraph checkpoint	第一次启用时把历史灌入 thread
仅新会话用 LangGraph	老会话仍走老逻辑
双写	同时写 sessions 表 + checkpoint, 逐步淘汰

推荐：第二种最稳，不动老数据。

小结一行： Sessions.id 直接当 thread_id、Sessions.workspace_id 当 checkpoint_ns —— Sessions 表无需改 Schema 即可接入。

§8.8 Skills 与 LangGraph 节点 / 子图的关系

Skills 系统的两种映射方式

映射	说明	适合
每个 Skill → 一个 @tool	简单 Skill（一次调用得结果）	大部分 Skill
每个 Skill → 一个子图	复杂 Skill（多步流程）	少数高级 Skill

简单 Skill 转 Tool

CODE

```
from langchain_core.tools import tool

def skill_to_tool(skill_def: SkillDefinition):
    """将 Skill 转换为 Tool"""
    @tool(
        name=skill_def.name,
        description=skill_def.description,
        args_schema=skill_def.args_schema,
    )
    def skill_runner(**kwargs):
        return execute_skill(skill_def.id, kwargs)
    return skill_runner

"""将 Skills 转换为 workspace 的 Tools"""
def get_tools_for_workspace(workspace_id):
    skills = db.query_skills(workspace_id=workspace_id)
    return [skill_to_tool(s) for s in skills]
```

复杂 Skill 转子图

CODE

```
"""将 Skill 转换为 StateGraph 的子图"""
def build_data_analysis_subgraph():
    sub = StateGraph(SubState)
    sub.add_node("extract", extract_data)
    sub.add_node("analyze", run_analysis)
    sub.add_node("plot", make_plot)
    sub.add_edge(START, "extract")
    sub.add_edge("extract", "analyze")
    sub.add_edge("analyze", "plot")
    sub.add_edge("plot", END)
    return sub.compile()

"""将 Skill 添加到 main 中"""
main.add_node("data_analysis_skill", build_data_analysis_subgraph())
```

小结一行：Skills 系统是 ToolNode 的天然来源 —— 简单 Skill 转 @tool、复杂 Skill 转子图。

§8.9 API 兼容层设计：既有 RESTful 接口怎么挂 LangGraph

现状（推测）

`routes/chat.py` 的接口大概形如：

CODE

```
@router.post("/chat/sessions/{session_id}/messages")
async def send_message(session_id: str, msg: MessageInput):
    session = db.get_session(session_id)
    history = db.get_messages(session_id)
    llm = LLM
    response = await llm.complete(history + [msg])
    db.save_message(session_id, response)
    return {"output": response}
```

加 LangGraph 后的兼容设计

CODE

```
@router.post("/chat/sessions/{session_id}/messages")
async def send_message(session_id: str, msg: MessageInput):
    session = db.get_session(session_id)

    if not session.langgraph_enabled:
        return await legacy_send(session_id, msg)

    langgraph = LangGraph
    config = {"configurable": {
        "thread_id": session_id,
        "checkpoint_ns": session.workspace_id,
        "user_id": session.user_id,
    }}
    result = await agent.ainvoke(
        {"messages": [{"role": "user", "content": msg.content}]},
        config=config,
    )
    return {"output": result["messages"][-1].content}
```

SSE 流式接口

CODE

```
@router.post("/chat/sessions/{session_id}/messages/stream")
async def stream_message(session_id: str, msg: MessageInput):
    config = {"configurable": {"thread_id": session_id}}

    async def event_generator():
        async for chunk in agent.astream(
            {"messages": [{"role": "user", "content": msg.content}]},
            config=config,
            stream_mode="messages",
            # 1000 tokens
            yield f"data: {json.dumps({'content': chunk.content})}\n\n"

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

小结一行：通过 `langgraph_enabled` 字段做 A/B 切换，老接口契约保持不变 —— 灰度发布最稳。

§8.10 多租户隔离：thread_id 命名空间 + checkpoint 表分区

多租户的 LangGraph 配置

CODE

```
config = {"configurable": {
    "thread_id": session_id,
    "checkpoint_ns": workspace_id,    # 1000 tokens
    # 1000
```

`checkpoint_ns` 让不同 workspace 的 thread 完全隔离 —— 即便 thread_id 重复也不会冲突。

Postgres 分区策略

CODE

```
CREATE TABLE checkpoints (
    thread_id TEXT,
    checkpoint_ns TEXT,
    checkpoint_id TEXT,
) PARTITION BY HASH (checkpoint_ns);

CREATE TABLE checkpoints_part_0 PARTITION OF checkpoints
FOR VALUES WITH (modulus 16, remainder 0);
```

跨租户的 Store 隔离

CODE

```
# Store namespace
namespace = ("user_memories", workspace_id, user_id)
store.put(namespace, key, value)
```

安全检查

风险	防御
A workspace 拿到 B workspace 的 checkpoint	API 层强制 workspace_id 匹配
Skills 跨 workspace 调用	tool 工厂只加载当前 workspace 的 Skills
Store 跨租户读写	namespace 第一段始终 workspace_id

小结一行：多租户用 `checkpoint_ns` + Postgres 分区 + Skill 工厂三层隔离 —— 安全核心是 API 层永远验证 workspace_id。

§8.11 当前不必上的判断条件

判断	解释
业务还没复杂到需要"循环 + 中断 + 跨会话"中任意一个	直接 SDK 够
团队 < 3 人，没人懂 LangGraph	学习成本暂时不划算
没有 LangSmith / 自家观测	上 LangGraph 没法调试
用户量小 (< 100 DAU)，还没碰到性能瓶颈	优先做产品
数据合规不允许使用 LangChain 系生态	自研或选 Dify / Coze

推荐先做的事

1. **观测先行**：先接 LangSmith / Langfuse，看清现状
2. **评估先行**：建 Promptfoo 用例集，量化质量
3. **简单优化**：开 Prompt Caching (§17.5) 能立刻省钱
4. **再决定**：观测和评估有了，再决定是否上 LangGraph

小结一行：上 LangGraph 不是越早越好 —— 先把"看见"和"度量"做好，再决定升级时机。

§8.12 本章小结

#	核心结论
1	custom_agent 现有架构是 FastAPI + Next.js + Gateway 三层，已有 Sessions / KB / Skills / Workspaces 四个核心概念
2	8 节点流程映射：现有工程在节点 ①②④⑤⑥⑧ 已有基础，缺节点 ③⑦
3	接入点选 api-server 内嵌（MVP），业务复杂后抽独立 agent service
4	三步迁移路径：直接 SDK → LCEL → ReAct → LangGraph，1-2 个月完成
5	Sessions.id ↔ thread_id 一对一、Workspaces ↔ checkpoint_ns、Skills ↔ @tool 三个映射建好即可接入
6	简单 Skill 转 @tool、复杂 Skill 转子图
7	API 兼容层用 <code>langgraph_enabled</code> 字段做灰度，老逻辑保留
8	多租户用 <code>checkpoint_ns</code> + Postgres 分区 + Skill 工厂三层隔离
9	当前不必上的 5 大条件 —— 先做观测和评估，再决定升级

§8.13 反模式速记

反模式	错在哪	正确做法
一上来就用独立 agent service	早期增加运维负担	先嵌入 api-server
Sessions 表大改适配 LangGraph	风险大	加 <code>langgraph_enabled</code> 字段灰度
Skills 全部转子图	大材小用	简单的转 @tool 即可
无 checkpoint_ns 跨租户	数据泄漏风险	必须按 workspace 隔离
不灰度全量切 LangGraph	出问题难回滚	langgraph_enabled 5% → 50% → 100%

§8.14 术语速查

术语	中文	含义
Session	会话	用户的一次对话上下文
Workspace	工作区	多租户的隔离单元
Skill	技能	用户工程中可调用的能力（类似 Tool）
<code>checkpoint_ns</code>	检查点命名空间	LangGraph 多租户隔离字段
灰度发布	Canary release	部分流量先试新逻辑

§8.15 推荐下一章

下一章: [§9 全景速览（基础链路总览）](#) —— Part B 开篇，从用户工程拉开视角看整个智能体生态。

Part B — 生态全景手册

「2026 年初快照。每个分类回答"它扩展或重做基础流程的哪个节点"。

§9 全景速览

§9.0 本章定位

项	内容
在基础流程中的位置	给 16 大类生态画一张总览图，叠加 8 节点高亮
与上下章的因果链	Part A 钻进了三件套，本部分拉开视角看整个生态
学完能做什么	(1) 知道 16 大类生态的分布；(2) 看懂"框架 vs 协议 vs 平台"边界；(3) 学会按需查 vs 通读两种使用法

§9.1 16 大类对应到 8 节点（全景图）

「本节是 Part B 的脊梁。

16 大类生态全景 (Mermaid #21)



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    subgraph FW["Framework"]
        F1["LangChain / Haystack / Semantic Kernel"]
        F2["LangGraph / Burr / Inngest"]
        F3["CrewAI / AutoGen/AG2 / OpenAI Agents"]
        F4["PydanticAI / Instructor / Marvin"]
        F5["DSPy / TextGrad / Trace"]
    end

    subgraph DM["Domain Models"]
        D1["RAG LlamaIndex / Cognita"]
        D2["Claude Code / Cursor / Devin / Cline"]
    end

    subgraph TS["Tooling / SDKs"]
        T1["Vercel AI SDK"]
        T2["Mastra / Inngest Agent Kit"]
    end

    subgraph LC["LangChain Ecosystem"]
        L1["Flowise / Langflow / n8n"]
        L2["Dify / Coze / AppBuilder"]
    end

    subgraph CL["Cloud Providers"]
        C1["AWS Bedrock / GCP Vertex / Azure AI Foundry / Databricks"]
    end

    subgraph IF["Infrastructure"]
        I1["Sandbox: E2B / Daytona / Modal / Replicate"]
        I2["Braintrust / Inspect AI / Promptfoo"]
        I3["LangSmith / Langfuse / Helicone"]
    end

    subgraph PR["Providers / Integrations"]
        P1["MCP / A2A / AGNTCY"]
    end

    subgraph TR["Tools / Reasoning"]
        N1["Agent as Code"]
        N2["Ambient Agent"]
        N3["Reasoning Models"]
    end

```

```
style F1 fill:#dbeafe
style F2 fill:#dbeafe
style F3 fill:#dbeafe
style D1 fill:#d1fae5
style D2 fill:#d1fae5
style T1 fill:#fef3c7
style L1 fill:#fce7f3
style L2 fill:#fce7f3
style C1 fill:#e9d5ff
style I1 fill:#fed7aa
style P1 fill:#fecaca
style N1 fill:#ddd6fe
```

各类对应基础流程的哪些节点

类别	主要影响节点	角色
通用框架	①-⑧ 全	编排引擎
RAG 框架	① 上下文增强	节点 ① 专精
编程类智能体	①-⑧ 全	端到端产品（不是框架）
JS/TS 框架	①-⑧ 全	平行实现
低代码	①-⑧ 全	可视化封装
云厂商	①-⑧ 全 + 部署	托管平台
Sandbox	④ 工具运行	节点 ④ 专精
评估	横切 ⑥⑧	横切观察
观测	横切全部	横切观察
协议	④ 工具协议	标准化
趋势	节点 ②⑦ 演化	范式扩展

小结一行：16 大类按"功能定位 + 影响节点"组织 —— 通用框架管全流程、专门派管特定节点、基础设施横切观察、协议规范化接口。

§9.2 看图视角：按"功能定位"而非"流量大小"

很多生态图按"使用率排版"，热门工具占大块。本手册反其道：**按工程定位排** —— 一个小众工具如果填补了独特的位置（如 MCP），地位等同于 LangChain 这种巨头。

三个看图原则

原则	解释
按节点定位	每个工具回答"它解决基础流程哪个节点的问题"
按生态层次	框架 / 协议 / 平台 / 工具是不同层
按时代	2022 第一代 / 2024 第二代 / 2025 新兴 不混为一谈

小结一行：先看一个工具属于哪一层 + 影响哪个节点，再看它的市场地位。

§9.3 各类关系：互补 / 替代 / 横切

三种关系

关系	例子
互补	LangChain 组件 + LangGraph 编排（业界默认）
替代	LangChain 与 PydanticAI（同样定位简洁框架）
横切	LangSmith 观测、E2B Sandbox（与框架解耦）

关系矩阵

工具 A vs B	关系
LangChain vs LangGraph	互补
LangGraph vs CrewAI	替代（多智能体场景）
LangSmith vs Langfuse	替代
LlamaIndex vs LangChain Retriever	替代
MCP vs OpenAI Function Calling	互补（协议层 vs 用法层）
Claude Code vs Cursor	替代（编程类智能体）

小结一行：互补 / 替代 / 横切三类关系不混淆，选型时先识别一个工具的"角色定位"。

§9.4 本手册使用法

使用方式	适合
按需查	想了解某个特定工具时跳到对应章节
通读	全局摸清生态
决策时查	配合 §18 选型决策矩阵使用

每个分类按"通用结构"组织：起源 / 定位 / 与其他工具的关系 / 适用场景 / 业界采用度 / 反模式。

小结一行：本手册三种使用法，配合 §18 决策矩阵和 §19 用户工程对照，是选型工具书。

§9.5 本章小结

#	核心结论
1	Part B 拉开视角，从单一三件套（Part A）扩展到整个 16 大类智能体生态
2	16 大类按"功能定位"组织（不按热度），每类回答"它扩展或重做基础流程的哪个节点"
3	工具关系三类：互补（LangChain 组件 + LangGraph 编排） / 替代（LangChain vs PydanticAI） / 横切（LangSmith 观测）
4	选型必须先识别工具的"角色定位"（哪一层 + 哪个节点）再看市场地位
5	本手册使用法：按需查 / 通读 / 决策时查 三种姿势配合 §18 决策矩阵 + §19 用户工程对照

§9.6 反模式速记

反模式	错在哪	正确做法
按"热度"选工具	热门 ≠ 适合你	按场景 + 节点定位选 (§18)
把"互补"工具当"替代"对比	选型逻辑错位	先识别工具关系 (§9.3)
跳过 Part B 直接选	视野太窄	至少扫一遍 16 大类全景

§9.7 术语速查

术语	中文	含义
Ecosystem	生态	围绕某主题的工具 / 框架 / 标准 / 社区集合
Vertical	垂直领域	专门做某类任务的工具（如编程类智能体）
Horizontal	横切	跨越多个垂直的工具（如评估 / 观测）
Protocol	协议	标准化接口（如 MCP）

§9.8 推荐下一章

下一章：§10 通用框架家族 —— 16 大类的第一类，5 个分支（第一代 / 第二代 / 多智能体 / 类型安全 / 编译式）逐个讲。

§10 通用框架家族

§10.0 本章定位

项	内容
在基础流程中的位置	节点 ① 到 ⑧ 全覆盖的"通用编排框架"
与上下章因果链	§9 全景图给了概览，本章深入"通用框架"这一类的 5 个分支
学完能做什么	(1) 区分第一代 / 第二代 / 多智能体 / 类型安全 / 编译式 5 个分支；(2) 知道每个分支代表工具；(3) 复述 Anthropic Building Effective Agents 五大模式

§10.1 第一代框架：LangChain / Haystack / Semantic Kernel

LangChain（详见 §2-§3）

维度	内容
起源	2022.10, Harrison Chase
当前定位	组件库（不再是 Agent 框架）
主要用法	LCEL + Document Loader + with_structured_output

Haystack

维度	内容
出品方	deepset 公司（德国）
起源	2019（早于 LangChain）
强项	RAG / 文档问答 / pipeline 化
现状	在欧洲企业较多采用，全球热度低于 LangChain
与 LangChain 区别	pipeline 思维（不是 chain），更强类型

CODE

```
# Haystack [?][?]
from haystack import Pipeline
from haystack.components.retrievers import InMemoryEmbeddingRetriever
from haystack.components.generators import OpenAIGenerator

pipe = Pipeline()
pipe.add_component("retriever", retriever)
pipe.add_component("generator", OpenAIGenerator())
pipe.connect("retriever", "generator")
```

Semantic Kernel（微软）

维度	内容
出品方	微软
起源	2023.04
语言	C# / Python / Java（跨语言）
强项	.NET 生态、企业级（与 Azure AD / Microsoft 365 集成）
现状	微软 Azure 客户基本盘

三者对比

维度	LangChain	Haystack	Semantic Kernel
语言	Python（+JS/TS）	Python	C#/Python/Java
强项	生态 / 集成多	RAG	企业 .NET
区域	全球	欧洲偏重	微软客户

小结一行：第一代框架三巨头按地域和生态各占一片 —— 全球 LangChain、欧洲 Haystack、Microsoft 客户 Semantic Kernel。

§10.2 第二代状态图：LangGraph / Burr / Inngest Agent Kit

LangGraph（详见 §4-§6）

主流第二代框架。

Burr (DAGWorks)

维度	内容
起源	2024, DAGWorks 公司（开源团队）
心智	状态机 + Action 函数
与 LangGraph 区别	更轻量、纯函数式、不依赖 LangChain
适合	不想被 LangChain 生态绑架的团队

CODE

```
# Burr ❸❸
from burr.core import State, action, ApplicationBuilder

@action(reads=["counter"], writes=["counter"])
def increment(state: State) -> State:
    return state.update(counter=state["counter"] + 1)

app = ApplicationBuilder().with_actions(increment).with_state(counter=0).build()
```

Inngest Agent Kit

维度	内容
出品	Inngest（事件驱动 workflow 平台）
心智	事件驱动 + 智能体
强项	与 Inngest 自家工作流引擎集成
语言	TypeScript（详见 §12）

三者对比

维度	LangGraph	Burr	Inngest Agent Kit
心智	状态图	状态机 + Action	事件 + Agent
主要语言	Python	Python	TS
学习曲线	中-难	中	中
生态	大（含 LangChain）	小	中

小结一行： 第二代框架以 LangGraph 为主流、Burr 是轻量替代、Inngest 是 TS 事件驱动选项。

§10.3 多智能体专门: CrewAI / AutoGen / OpenAI Agents SDK / Camel-AI / MetaGPT

CrewAI

维度	内容
起源	2023.11, João Moura
心智	角色驱动 (Agent + Task + Crew)
适合	快速 PoC、营销 / 内容生成
商业化	CrewAI Enterprise (2024)
与 LangGraph 区别	角色更显式, 但灵活性低

CODE

```
# CrewAI 快速入门
from crewai import Agent, Task, Crew

researcher = Agent(role="资深研究员", goal="收集最新行业趋势")
writer = Agent(role="内容撰写员", goal="撰写引人入胜的文章")

task1 = Task(description="研究并总结未来3年的市场趋势", agent=researcher)
task2 = Task(description="根据研究结果撰写一篇关于未来趋势的文章", agent=writer)

crew = Crew(agents=[researcher, writer], tasks=[task1, task2])
result = crew.kickoff()
```


AutoGen → AG2 / AutoGen Studio

维度	内容
起源	2023.10，微软研究院
心智	多智能体对话
2024 末分裂	团队分家 → AG2（社区分支）+ AutoGen Studio（微软官方）
强项	学术 / 研究风格
现状	分裂后热度有所下降

OpenAI Agents SDK（前身 Swarm）

维度	内容
起源	2024.10（Swarm 实验）→ 2026.Q1 GA
心智	Handoff 模型（智能体间交接）
强项	OpenAI 自家、轻量
适合	OpenAI 全栈用户

Camel-AI

维度	内容
起源	2023, KAUST 学术团队
心智	角色扮演 (Role-playing)
强项	学术研究
现状	学术领域采用

MetaGPT

维度	内容
起源	2023, DeepWisdom (中国)
心智	软件公司模拟 (PM / Architect / Engineer / QA)
强项	软件工程类任务
现状	中文社区影响力大

五者对比

工具	心智	适用	推荐度
CrewAI	角色 + Task	快速 PoC	起步可用
AutoGen / AG2	多 Agent 对话	研究	一般
OpenAI Agents SDK	Handoff	OpenAI 全栈	跟 OpenAI 绑定时用
Camel-AI	Role-playing	学术	学术圈
MetaGPT	软件公司模拟	编程任务	中文社区

小结一行：多智能体专门派各有定位 —— 严肃生产推荐用 LangGraph Supervisor / Plan-Execute（更通用），CrewAI 适合快 PoC。

§10.4 类型安全派：PydanticAI / Instructor / Marvin

PydanticAI

维度	内容
起源	2024.12, Pydantic 团队
强项	类型安全 + Pydantic 模型 + Logfire 集成
增长	2025 增长最快的智能体框架之一
适合	Pydantic 重度用户、严格类型团队

CODE

```
# PydanticAI [?][?]
from pydantic_ai import Agent
from pydantic import BaseModel

class WeatherResult(BaseModel):
    city: str
    temperature: float

agent = Agent(
    "openai:gpt-4o",
    result_type=WeatherResult,
    system_prompt="[?][?][?][?][?][?][?][?]"
)

result = agent.run_sync("[?][?][?][?][?]")
print(result.data) # WeatherResult(city='[?][?][?][?]'temperature=25.0)
```

Instructor (Jason Liu)

维度	内容
起源	2023, Jason Liu
强项	结构化输出（基于 Pydantic）
与 PydanticAI 区别	Instructor 只做结构化输出，不做编排
现状	库小但用户多

Marvin (Prefect 团队)

维度	内容
起源	2023, Prefect
强项	类型驱动 AI 函数
现状	小众但优雅

小结一行：类型安全派的精神是"用 Pydantic 模型作为接口" —— 适合不想被 LangChain 抽象绑架但又想要保护的团队。

§10.5 编译式：DSPy / TextGrad / Trace

DSPy (Stanford)

维度	内容
起源	2023, Stanford NLP
心智	"把 prompt 当代码编译优化"
核心创新	Signature (函数签名) + Module (程序) + Optimizer (优化器)
适合	严肃 NLP 团队 / 研究

TextGrad

第 342 页

Trace（微软）

维度	内容
起源	2024，微软
心智	DSPy 思路推广到任意 AI 系统
现状	学术圈小众

小结一行：编译式派的核心是"prompt 不该手写而该自动优化" —— 学术领先但工业采用慢。

§10.6 各派别在 8 节点上的着力点对比表（Mermaid #29）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph Nodes["8 节点"]
        N1["①"]
        N2["②"]
        N3["③"]
        N4["④"]
        N5["⑤"]
        N6["⑥"]
        N7["⑦"]
        N8["⑧"]
    end

    LC["LangChain"] --> N1
    LC --> N2
    LC --> N4
    LC --> N6

    LG["LangGraph"] --> N3
    LG --> N6
    LG --> N7

    Multi["Multi"] --> N3
    Multi --> N7

    Type["Type"] --> N1
    Type --> N8

    Comp["Comp"] --> N1
    Comp --> N2

    style LC fill:#d9e1f2
    style LG fill:#e9d5ff
    style Multi fill:#fce7f3
    style Type fill:#d1ecf1
    style Comp fill:#f2f2f2
```

小结一行：5 个分支各有节点专长 —— LangChain 全方位、LangGraph 状态/循环、多智能体派分支、类型安全派输入输出、编译派 prompt 优化。

§10.7 Anthropic Building Effective Agents 五大模式

「 2024.12 Anthropic 官方博客发布的智能体设计模式 —— 业界最被引用的智能体设计纲领。
详见附录 I。

五大模式总览

模式	中文	节点	一句话
Prompt Chaining	提示词链	② 串行	把任务拆成几个 prompt 串行
Routing	路由	③	模型先决定走哪条路再处理
Parallelization	并行化	⑦	同任务多模型 / 多角度并行
Orchestrator-Workers	协调-工作者	②③	协调者拆任务、工作者执行
Evaluator-Optimizer	评估-优化	⑥⑦	生成 → 评估 → 改进的循环

各模式与三件套的对应

模式	用 LangChain	用 LangGraph
Prompt Chaining	LCEL: <code>prompt1 \\\ llm \\\ parser \\\ prompt2 \\\ llm</code>	简单链可以，但复杂用图
Routing	<code>RunnableBranch</code>	条件边 + Command
Parallelization	<code>RunnableParallel</code>	Send API
Orchestrator-Workers	难（需手写）	Supervisor 模式
Evaluator-Optimizer	难	Plan-Execute / Reflexion

小结一行：Anthropic 五大模式是智能体设计的"基本词汇表" —— 任何复杂智能体都是这五种模式的组合。详见附录 I。

§10.8 本章小结

#	核心结论
1	通用框架家族 5 分支：第一代（LangChain / Haystack / Semantic Kernel） / 第二代（LangGraph / Burr / Inngest） / 多智能体（CrewAI / AutoGen / OpenAI Agents） / 类型安全（PydanticAI / Instructor / Marvin） / 编译式（DSPy / TextGrad / Trace）
2	各派别有 8 节点专长，互补而非完全替代
3	多智能体专门派起步推荐 CrewAI（PoC） / 严肃生产用 LangGraph Supervisor
4	类型安全派增长最快（PydanticAI），适合 Pydantic 重度用户
5	Anthropic 五大模式是智能体设计的基本词汇表

§10.9 反模式速记 + 术语速查 + 推荐下一章

反模式	解决
多智能体场景直接上 Swarm	先 Supervisor
用 PydanticAI 跑大型多智能体系统	用 LangGraph
用 DSPy 替代生产框架	DSPy 是优化器、不是运行时

术语	含义
Crew	CrewAI 的"团队"
Handoff	OpenAI Agents SDK 的"交接"
Signature	DSPy 的函数签名
Module	DSPy 的程序单元
Optimizer	DSPy 的优化器

下一章：[§11 领域专门派](#) —— RAG 框架、编程类智能体、计算机使用 / 浏览器使用 / 语音 / 多模态。

§11 领域专门派

§11.0 本章定位

「在基础流程中的位置：每个垂直领域专精基础流程的不同节点 —— RAG 重做节点 ① 上下文增强、编程类智能体重做节点 ④ 工具集（含沙箱）、Computer Use 把节点 ④ 扩展到视觉操作。

垂直领域有比通用框架更专门的工具：RAG 有 LlamaIndex（专精节点 ①）、编程有 Claude Code / Cursor（端到端 8 节点垂直特化）、Anthropic Computer Use 重做节点 ④ 把工具调用扩展到屏幕操作。本章覆盖 6 个垂直方向。

§11.1 检索增强生成（RAG）框架

LlamaIndex

维度	内容
起源	2022.11, Jerry Liu
强项	RAG 专精 / 数据连接器 200+
与 LangChain 区别	LlamaIndex 只做 RAG，但做得更深；LangChain 啥都做
适合	重度 RAG 项目

LlamaIndex 与 LangChain 的关系

可以**互补使用**：用 LlamaIndex 做 RAG 部分（数据加载 / 索引 / 检索），用 LangGraph 做编排。

Cognita（TrueFoundry）

维度	内容
起源	2024, TrueFoundry
强项	工程化 RAG 平台（不只是库）
现状	企业级用户

详见用户工程已有的 RAG 文档：[03-rag.md](#)、[15-knowledge-engineering.md](#)、[37-rag-implementation-plan.md](#)。

小结一行：RAG 专门派以 LlamaIndex 为主流，与 LangGraph 互补；详细 RAG 知识参考用户工程已有文档。

§11.2 编程类智能体

共同点：端到端产品（不是框架）

编程类智能体不卖框架，卖产品。每个产品内部都有自己的智能体编排（自研 / 闭源），不暴露给开发者用。

代表产品对比

产品	出品方	形态	强项
Claude Code (Anthropic)	CLI + IDE	终端 + 文件 + git	Anthropic 全栈
Cursor	Anysphere	IDE Fork (VSCode-based)	自然语言改代码
Devin (Cognition)	浏览器 IDE	长程任务 / 沙箱	端到端任务
Cline / Roo Cline	开源 VSCode 插件	编辑器集成	开源、免费
OpenHands (前 OpenDevin)	开源	沙箱 + 浏览器	开源 Devin 替代
SWE-Agent	Princeton	学术	SWE-bench benchmark
Aider	开源 CLI	git + 文件	git 集成
Continue.dev	开源插件	IDE	可配置
Windsurf (前 Codeium)	Codeium	IDE	商业化

编程类智能体的共同 8 节点设计

节点	编程类智能体常见做法
① 上下文	大量包含项目文件 / git diff / 错误日志
② 推理	通常用最强模型（Claude Opus / GPT-4o）
③ 决策	主要工具：read_file / write_file / run_bash
④ 工具	文件操作 / Shell / 浏览器 / 包管理
⑤ 返回	工具返回（含可能很大）做截断
⑥ 状态	跟踪改了哪些文件
⑦ 循环	长循环（往往 30+ 步）+ 测试驱动
⑧ 输出	流式 + 文件 diff

小结一行：编程类智能体是"端到端产品垂直派"，不是框架；其内部 8 节点设计与通用智能体类似但循环更长、工具更专。

§11.3 业界为什么这两个领域要"自立门户"

RAG 自立门户的原因

痛点	通用框架不擅长
200+ 文档加载器	通用框架不会专门维护
索引策略多（HNSW / IVF / DiskANN）	通用框架抽象不到这层
重排序（Reranker）链路	通用框架不深入
多向量库适配	专门做反而更全

编程类智能体自立门户的原因

痛点	通用框架不能给
终端集成（CLI 工具）	通用框架是库
IDE 集成（VSCode 扩展）	框架不做 UI
沙箱（隔离的代码执行）	涉及基础设施
大量产品级优化（提示词 / 模型选）	闭源产品的护城河

小结一行：垂直领域自立门户因为"通用框架的抽象层次不够" —— 要么太底层（LangChain）要么太上层（云平台）。

§11.4 计算机使用 / 浏览器使用：Anthropic Computer Use / Browser Use

Anthropic Computer Use

维度	内容
起源	2024.10, Anthropic Sonnet 3.5 v2
心智	让模型直接操作屏幕（看截图 + 输出鼠标键盘）
工具	screenshot / computer（鼠标点击 / 键盘输入）
适合	自动化没有 API 的任务
风险	很容易出错（截图理解仍弱）

OpenAI Operator / Computer Use

维度	内容
起源	2025.01, OpenAI
心智	与 Anthropic Computer Use 类似

Browser Use（开源）

维度	内容
起源	2024，开源
心智	浏览器 DOM + 截图 + LLM
与 Computer Use 区别	限制在浏览器（更可控）

8 节点设计差异

节点	普通智能体	Computer Use
① 上下文	文本 + 工具	文本 + 屏幕截图
② 推理	文本理解	视觉 + 文本理解
④ 工具	几个 API	鼠标 + 键盘 + 截图
⑦ 循环	中等长度	通常很长（每步一截图）

小结一行：Computer Use 是"视觉智能体"赛道 —— 在 8 节点上加入"看屏幕"和"鼠标键盘"，闭环长但能做 API 没有的任务。

§11.5 语音智能体

OpenAI Realtime API

维度	内容
起源	2024.10, OpenAI
心智	端到端语音（不是 STT + LLM + TTS 串起来）
强项	低延迟（200ms 级）+ 自然

ElevenLabs

维度	内容
起源	2022
强项	TTS 质量极高 + 多语言
现状	配音 / 有声书 / 客服外呼

Vapi

维度	内容
起源	2023
强项	语音智能体平台（电话 / 应用）

小结一行：语音智能体从"STT + LLM + TTS 串"演化到"端到端模型"（OpenAI Realtime），低延迟和自然度大幅提升。

§11.6 多模态智能体

主流多模态模型

模型	输入	输出
GPT-4o / GPT-4V	文本 + 图 + 语音	文本 + 语音
Claude Sonnet 4.6 / Opus 4.7	文本 + 图 + PDF	文本
Gemini 2.5 Pro	文本 + 图 + 视频 + 音频	文本

多模态智能体的应用

场景	模型
截图理解（Computer Use）	Claude / GPT-4o
表单 OCR + 理解	GPT-4V / Gemini
视频问答	Gemini
设计稿转代码	GPT-4o

小结一行：多模态让节点 ① 上下文从"文本"扩展到"图 / 音 / 视频" —— 智能体能处理的任务范围指数级扩大。

§11.7 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	RAG / 编程类智能体是两个最大的垂直领域，有专门工具链
2	RAG 主流是 LlamaIndex（与 LangGraph 互补）
3	编程类智能体是端到端产品（不是框架），各自闭源
4	Computer Use / Browser Use 是新兴"视觉智能体"赛道
5	语音智能体从串行架构演化到端到端模型（OpenAI Realtime）
6	多模态让节点 ① 上下文扩展到图 / 音 / 视频

反模式速记

反模式	解决
用通用框架做 RAG 重活	用 LlamaIndex
用 Computer Use 做有 API 的任务	直接调 API
把 STT + LLM + TTS 串行做语音智能体	用 OpenAI Realtime API

术语速查

术语	含义
Coding Agent	编程类智能体
Computer Use	计算机使用（操作屏幕）
Browser Use	浏览器使用（DOM 操作）
Multimodal	多模态（文本+图+音+视频）
TTS / STT	语音合成 / 语音识别
Realtime API	端到端语音 API

推荐下一章

下一章: [§12 JavaScript / TypeScript 平行宇宙](#) — Python 之外的另一个 AI 生态。

§12 JavaScript / TypeScript 平行宇宙

§12.0 本章定位

「在基础流程中的位置：与 Python 生态平行覆盖节点 ① 到 ⑧，但工具栈完全不同。

后端用 Python 的世界与前端用 TS 的世界几乎不交叉。本章讲 TS 这边的玩法 —— 同样的 8 节点流程，换一套实现。

§12.1 Vercel AI SDK

维度	内容
起源	2023.06, Vercel
心智	前端 AI 应用的"事实标准"
强项	与 Next.js / React 完美集成、流式 UI 组件
关键 hook	<code>useChat</code> / <code>useCompletion</code> / <code>streamText</code>
现状	TS 生态前端必备

CODE

```
// Vercel AI SDK [?][?]  
import { streamText } from "ai";  
import { openai } from "@ai-sdk/openai";  
  
const result = await streamText({  
  model: openai("gpt-4o"),  
  messages,  
  tools: { /* tool defs */ },  
  [?][?][?]  
  
for await (const chunk of result.textStream) {  
  console.log(chunk);  
  [?]
```

§12.2 Mastra

维度	内容
起源	2024, Mastra Inc.
心智	全栈 TS 智能体框架（类似 LangGraph）
强项	工作流编排 + 评估 + 部署一体
现状	2025 增长最快的 TS 智能体框架

CODE

```
// Mastra [?][?]  
import { Agent } from "@mastra/core";  
  
const agent = new Agent({  
  model: { provider: "openai", name: "gpt-4o" },  
  tools: { search, calculator },  
  instructions: "[?][?][?][?][?]  
  [?][?]  
  
const response = await agent.generate("[?][?][?][?][?]);
```

§12.3 Inngest Agent Kit

事件驱动 TS（详见 §10.2）。

§12.4 LangChain.js / LangGraph.js（采用率真相）

维度	内容
起源	LangChain Python 的 TS 平移
现状	存在但热度不及 Vercel AI SDK / Mastra
适合	全栈团队需要 Python 后端 + JS 前端共享逻辑

为什么 TS 用户不爱 LangChain.js：

原因	说明
抽象太重	TS 用户更喜欢轻量
Vercel AI SDK 更"原生"	与 React / Next.js 生态契合
平移版常落后	API 变化滞后于 Python 版

§12.5 Cloudflare Workers AI + Agents

维度	内容
起源	2024, Cloudflare
心智	边缘计算 + 智能体
强项	低延迟（全球 300+ 节点）+ 按调用计费
适合	Latency 敏感场景

§12.6 双宇宙的技术与组织原因

技术原因

原因	说明
Python 是 AI/ML 主导语言	模型训练 / 推理生态在 Python
TS 是 Web 主导语言	前端 / Node.js 后端
两套生态独立演化	互相借鉴但不通用

组织原因

原因	说明
后端 / 前端团队分工	Python 团队和 TS 团队不同人
公司技术栈选择	一个公司往往主选一边
招聘市场	Python AI 工程师 vs TS 前端工程师不重叠

§12.7 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	TS 生态以 Vercel AI SDK 为前端事实标准、Mastra 为全栈后起之秀
2	LangChain.js / LangGraph.js 存在但 TS 用户不爱
3	Cloudflare Workers 是边缘计算选项
4	双宇宙的存在是技术 + 组织双重原因

反模式速记

反模式	解决
Python 团队用 LangChain.js 给前端	前端走 Vercel AI SDK, 后端走 Python LangGraph
全栈 TS 团队硬接 Python LangGraph	用 Mastra 或 LangGraph.js
TS 项目自研所有 LLM 集成	用 Vercel AI SDK 标准化

术语速查

术语	中文	含义
Vercel AI SDK	Vercel AI SDK	前端 AI 应用的事实标准 (React/Next.js 集成)
Mastra	Mastra	TS 全栈智能体框架 (LangGraph 在 TS 的对应物)
Inngest Agent Kit	—	事件驱动 TS 智能体框架
Cloudflare Workers AI	—	边缘计算 + LLM 推理
streamText	—	Vercel AI SDK 的流式输出函数
useChat / useCompletion	—	Vercel AI SDK 的 React hooks

推荐下一章

下一章: [§13 低代码与可视化](#) — 不写代码也能做智能体。

§13 低代码与可视化

§13.0 本章定位

「在基础流程中的位置：覆盖节点 ① 到 ⑧ 全部，但是把代码层面的实现"图形化封装"。读者不写代码、用拖拽搭流程。

低代码工具用拖拽 / 可视化代替代码。**优势**：节点 ①②④⑧ 都封装成可视块，业务人员能搭。**劣势**：节点 ③⑥⑦ 复杂逻辑可视化表达能力弱，生产级仍有局限。

§13.1 国际：Flowise / Langflow / n8n

Flowise

维度	内容
起源	2023.04，开源
心智	LangChain 可视化
现状	中等

Langflow

维度	内容
起源	2023, 开源
收购	2024 被 Datastax 收购
心智	LangChain 可视化（更精美）

n8n + AI 节点

维度	内容
起源	2019（先于 LLM 时代的 workflow 引擎）
心智	通用 workflow + 后加 AI 节点
强项	与 1000+ 集成（Slack / Notion / Airtable / 数据库）
适合	已有 n8n 工作流、想加 AI

§13.2 中国生态

Dify

维度	内容
起源	2023, 中国团队
开源	是
心智	LLM 应用开发平台 (含 RAG / 智能体 / 工作流)
现状	国内最流行 + 全球开源用户多

Coze (字节)

维度	内容
起源	2023, 字节
心智	C 端 bot 平台 (豆包 / 抖音内嵌)
现状	国内 C 端首选

百度 AppBuilder / 千帆

维度	内容
起源	2023, 百度
心智	企业级 AI 应用平台
现状	国内企业向

§13.3 适用边界

低代码 OK 的场景

场景	原因
原型快速验证	1 小时搭出来给业务看
业务人员自助	不需要工程师
简单工作流	节点数 < 20
内部工具	不上严肃生产

低代码不行的场景

场景	原因
复杂状态管理	可视化表达不出 LangGraph 的图
大流量生产	性能 / 监控不够
严格合规	数据流不可控
自定义逻辑多	总要写代码"逃逸"
Code review / 版本管理	可视化 diff 难

§13.4 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	国际低代码以 Flowise / Langflow / n8n 为主
2	中国生态 Dify / Coze / 百度 AppBuilder 各占细分
3	低代码适合原型 / 简单工作流 / 业务人员，不适合复杂生产

反模式速记

反模式	解决
在 Dify 里硬怼复杂逻辑	复杂部分写代码、简单部分用 Dify
用低代码做高流量生产	迁到代码框架
业务人员搭核心生产链路	业务搭原型 / 工程接生产

术语速查

术语	中文	含义
Low-code	低代码	拖拽 + 配置代替编码
Flowise / Langflow	—	LangChain 可视化（Langflow 已被 Datastax 收购）
n8n	—	通用 workflow + AI 节点（含 1000+ 集成）
Dify	—	中国团队开源 LLM 应用平台（全球用户广）
Coze	—	字节 C 端 bot 平台
AppBuilder	—	百度企业 AI 平台

推荐下一章

下一章：[§14 云厂商平台](#) —— 三大云的智能体托管方案。

§14 云厂商平台

§14.0 本章定位

「在基础流程中的位置：云厂商把基础流程节点 ① 到 ⑧ + 部署运维全部托管，开发者只需配置而非编码。

云厂商把"智能体托管"做成自家服务，覆盖完整 8 节点 + 节点 ⑥ 状态持久化 + 部署形态。优势：开箱即用 / 与云生态集成。劣势：厂商锁定 / 节点 ④ 工具受限于平台。

§14.1 AWS Bedrock Agents

维度	内容
起源	2023.07, AWS
心智	Agent + Knowledge Base + Action Group
模型	Bedrock 上的所有模型 (Claude / Llama / Mistral)
集成	Lambda (工具) / S3 / DynamoDB
适合	已用 AWS 的企业

§14.2 Google Vertex AI Agent Builder

维度	内容
起源	2024, Google Cloud
心智	Agent + DataStore + Tool
模型	Gemini 系列
适合	GCP 客户

§14.3 Azure AI Foundry（前 AI Studio）

维度	内容
起源	2023, 微软
心智	Agent + Skill + Search
模型	OpenAI（独家）+ 开源模型
强项	与 Microsoft 365 / Teams / SharePoint 集成

§14.4 Databricks Agent Framework

维度	内容
起源	2024, Databricks
心智	与 Databricks 数据湖深度集成
适合	Databricks 客户

§14.5 厂商锁定 vs 开箱即用

真实账单举例（粗估）

任务：每月 100 万次智能体调用，每次平均 5 步、3000 token。

部署	成本（美元/月）
自部署 LangGraph + 自购模型 API	\$3000（仅 token）+ \$200（基础设施）= \$3200
AWS Bedrock Agents（Claude）	\$4500（含管理费）
LangGraph Platform（Cloud）	\$3500 + \$订阅费
Azure AI Foundry（GPT-4o）	\$5000+

锁定风险

风险	解释
模型锁定	只能用厂商支持的模型
数据锁定	数据存在厂商系统、迁移代价大
技能 / 工具锁定	写出的 Action 只能在该平台跑
价格上涨	没有议价空间

§14.6 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	AWS / GCP / Azure / Databricks 各自有智能体平台
2	优势开箱即用 + 与云生态集成
3	劣势厂商锁定 + 价格高
4	已是该云客户 → 用其智能体平台合理；否则自部署更灵活

反模式速记

反模式	解决
不是某云客户却选用其 Agents 平台	选自部署 LangGraph
把核心业务全锁在一个云平台	留好迁移路径
不算 vendor lock-in 风险直接上	上线前先估算迁移成本

术语速查

术语	中文	含义
Bedrock Agents	—	AWS 智能体托管平台
Vertex AI Agent Builder	—	Google Cloud 智能体平台
AI Foundry	—	Azure 智能体平台（前 AI Studio）
Databricks Agent Framework	—	Databricks 数据湖智能体方案
Vendor Lock-in	厂商锁定	数据/工具/技能绑定单一云的代价
Action Group	—	Bedrock 把工具调用打包的概念

推荐下一章

下一章：[§15 基础设施层](#) —— 横切的 Sandbox / 评估 / 观测层（被低估）。

§15 基础设施层（横切节点 ⑥ + ⑧）

§15.0 本章定位

§15.1 沙箱与运行时：E2B / Daytona / Modal / Replicate

「在基础流程中的位置：节点 ④ 工具运行的隔离环境。

为什么需要 Sandbox

当工具是"运行任意代码"（编程类智能体最常见）时，必须把代码跑在隔离环境里 —— 否则模型可能 `rm -rf /`。

主流 Sandbox 服务

工具	起源	强项
E2B	2023	代码执行 sandbox, 最早最专门
Daytona	2024	dev 环境（含 IDE）
Modal	2022	Serverless GPU + Sandbox
Replicate	2019	模型托管 + Sandbox

E2B 用法

CODE

```
from e2b import Sandbox

sandbox = Sandbox()
result = sandbox.run_code("print(2 + 2)")
print(result.text)  # 4
sandbox.close()
```

何时需要 Sandbox

场景	需要
编程类智能体（执行代码）	必须
数据分析智能体（跑 SQL / Python）	推荐
客服智能体（仅调 API）	不需要
文档分析智能体（仅读文档）	不需要

小结一行：Sandbox 是"代码执行类"智能体的标配 —— 选 E2B（最专精）或 Modal（已用 Modal 时）。

§15.2 评估（Evaluation）

「在基础流程中的位置：横切节点 ⑥⑧，量化智能体输出质量。

主流评估工具

工具	起源	心智	适合
Braintrust	2023	商业领先	严肃团队
Inspect AI	2024	英国 AI 安全研究所 (AISI)	严肃评估 / 政府
Promptfoo	2023	yaml 配置 / 跨框架	开源 / 轻量
DeepEval	2024	pytest 风格	Python 工程团队
Phoenix (Arize)	2023	评估 + 观测一体	中小团队

Promptfoo 示例

CODE

```
prompts:
  "???"question???"
providers:
  - openai:gpt-4o
  - anthropic:claude-sonnet-4-6
tests:
  - vars:
      question: "?????????"
    assert:
      - type: contains
        value: "???"
      - type: cost
        threshold: 0.01
      - type: llm-rubric
        value: "?????????"
```

评估的几种类型

类型	例子
字符串匹配	contains / equals / regex
LLM-as-judge	用 LLM 当评分员
数值约束	cost / latency / token
自定义函数	Python 函数返回 0-1

小结一行：评估是严肃团队上生产的前提 —— 商业用 Braintrust / Inspect AI、开源用 Promptfoo。

§15.3 观测（Observability）

「在基础流程中的位置：横切所有节点的运行追踪。

主流观测工具

工具	部署	强项
LangSmith	云 / 自托管	与 LangChain 系深度集成
Langfuse	自托管 / 云	开源最强、隐私自主
Helicone	云	LLM 代理 + 观测一体
W&B Weave	云	ML 团队复用 W&B

观测能看什么

维度	内容
Trace（链路）	单次智能体调用的完整树
Token 用量	每次调用 / 累计
Latency	每个节点 / 总耗时
Error	异常追踪
Cost	按厂商定价计算费用
评估	集成评估工具的结果

Langfuse 自托管

CODE

```
git clone https://github.com/langfuse/langfuse
cd langfuse
docker compose up -d
🔗🔗🔗🔗http://localhost:3000
```

观测工具选型

团队类型	选择
已用 LangChain 系 + 可上云	LangSmith
数据合规要求高	Langfuse 自托管
已用 W&B 训练模型	W&B Weave
同时要 LLM proxy	Helicone

小结一行：观测推荐 LangSmith（云）或 Langfuse（自托管，开源最强）。

§15.4 提示词管理工具

工具	强项
PromptHub	Git-like 版本管理
PromptLayer	团队协作
Promptfoo （部分功能）	评估 + 管理一体
LangSmith Prompt Hub	与 LangSmith 集成

痛点	解决
Prompt 写在代码里改一次发一次版	抽出来用工具管
多人改 prompt 没历史	工具的 Git-like 版本
A/B 测试 prompt 没法做	工具集成评估

小结一行：大团队（>10 人 prompt 工作者）需要专门工具，小团队代码版控就够。

§15.5 成本观测：FinOps for Agents

工具	成本观测能力
Helicone	仪表盘按用户 / 项目 / 模型聚合
Langfuse	内置 cost calculator
自建	用厂商 usage 字段 + Prometheus

关键指标

指标	含义
Cost per request	单次调用成本
Cost per user / DAU	单用户成本
Cost per session	单会话成本
Token efficiency	输出 token / 输入 token 比
Cache hit rate	Prompt caching 命中率

小结一行：智能体的 FinOps 是"看得见的省钱" —— 没仪表盘就不知道钱花哪了。

§15.6 业界为什么把这层独立出来（Mermaid #30）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph FW["Framework"]
        LC["LangChain"]
        LG["LangGraph"]
        Cr["CrewAI"]
        OAI["OpenAI Agents SDK"]
    end

    subgraph CT["Context"]
        Eval["Evaluation"]
        Obs["Observation"]
        SB["Sandbox"]
        PM["Prompt"]
        Cost["Cost"]
    end

    LC -.-> Obs
    LG -.-> Obs
    Cr -.-> Obs
    OAI -.-> Obs

    LC -.-> Eval
    LG -.-> Eval

    LG -.-> SB
    LC -.-> SB

    style Obs fill:#fed7aa
    style Eval fill:#fed7aa
    style SB fill:#fed7aa
    style PM fill:#fed7aa
    style Cost fill:#fed7aa
```

独立的好处

好处	说明
与框架解耦	换框架不用换观测
跨框架可比	同一仪表盘看 LangChain / LangGraph / CrewAI
专门做更深	评估 / 观测的特性比框架自带丰富
商业模式稳	框架可能被淘汰，观测 / 评估服务长期需要

小结一行：横切层与框架解耦是关键设计 —— "框架可换，观测和评估永驻"。

§15.7 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	基础设施层四大块：Sandbox / 评估 / 观测 / 成本
2	评估是严肃团队上生产的前提
3	观测推荐 LangSmith（云）或 Langfuse（自托管）
4	Sandbox 是代码执行类智能体必备
5	横切层与框架解耦是核心设计原则

反模式速记

反模式	解决
不评估直接上生产	必须先建评估集
不开观测	看不见钱、看不见错
让模型直接 exec 用户代码	用 E2B Sandbox

术语速查

术语	含义
Sandbox	隔离运行环境
LLM-as-judge	用 LLM 评分
Trace	单次调用的完整链路记录
FinOps	财务运营
Cache hit rate	缓存命中率

推荐下一章

下一章：[§16 协议与标准](#) —— MCP 是 2025-2026 最重要的趋势。

§16 协议与标准（重做节点 ④ 工具调用）

§16.0 本章定位

「在基础流程中的位置：协议层重做节点 ④ 工具调用 —— 把"应用进程内调函数"标准化为"跨进程跨厂商发现 + 调用"。

协议层往往比框架层更重要 —— 框架可换，协议是行业地基。**MCP 把基础流程节点 ④ 工具调用从"私有函数"升级为"跨厂商可发现的标准接口"**，是 2024.11 以来最大的协议趋势。

§16.1 模型上下文协议（MCP）—— Anthropic 推出的事实标准

MCP 是什么

模型上下文协议（Model Context Protocol，简称 MCP）—— Anthropic 2024.11 推出的"工具发现 + 调用"跨厂商标准。

解决的问题	旧做法	MCP
工具描述格式	每家 LLM 不一样	统一 JSON Schema
工具发现	应用代码里硬编码	运行时发现
工具复用	写一次只能给一家用	写一次任意 MCP 客户端可用
工具部署	应用进程内	独立 MCP server

MCP 三层架构 (Mermaid #22)

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    subgraph Host ["MCP Host"]
        App["LLM Claude Desktop / Claude Code / Cursor"]
        Client["MCP Client"]
        App --> Client
    end

    subgraph Servers ["MCP Servers"]
        S1["filesystem server"]
        S2["github server"]
        S3["postgres server"]
        S4["custom server"]
    end

    Client <--> S1
    Client <--> S2
    Client <--> S3
    Client <--> S4

    style App fill:#dbeafe
    style Client fill:#fce7f3
    style S1 fill:#d1fae5
    style S2 fill:#d1fae5
    style S3 fill:#d1fae5
    style S4 fill:#d1fae5
```

三层职责

层	职责
Host	用户面对的 LLM 应用（Claude Desktop / Cursor / IDE 插件）
Client	内嵌在 Host 里、与 Servers 通信
Server	独立进程，提供一组工具（与具体 LLM 无关）

通信协议

MCP 用 JSON-RPC 2.0：

传输	用法
stdio	本地进程通信（最常见）
SSE / HTTP	远程 server
WebSocket	双向通信

一个 MCP server 示例

CODE

```
from mcp.server import Server
from mcp.types import TextContent

server = Server("my-tools")

@server.tool()
async def search_docs(query: str) -> str:
    """Search the internal knowledge base for the given query"""
    return f"Found {len(query)} results for query '{query}'"

if __name__ == "__main__":
    server.run_stdio()
```

MCP 与现有方案对比

维度	OpenAI Function Calling	Anthropic MCP
范围	单次 API 调用	工具发现 + 调用 + 部署
跨厂商	否	是
工具部署	应用进程内	独立 server
现状	厂商专属	正在成事实标准

MCP 在 LangGraph 的集成

CODE

```
from langchain_mcp_adapters.client import MultiServerMCPClient

async with MultiServerMCPClient({
    "filesystem": {
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path"],
        "transport": "stdio",
        ???
    }) as client:
    tools = await client.get_tools()
    agent = create_react_agent(llm, tools)
```

小结一行：MCP 是 2024.11 以来最重要的智能体协议 —— 工具与具体 LLM / 框架解耦，"写一次任意客户端可用"。

§16.2 智能体到智能体协议（A2A） / AGNTCY

协议	来源	现状
Agent-to-Agent（A2A）	多家提议 2024-2025	早期，未成标准
AGNTCY	Cisco 推 2025	提案阶段

小结一行：智能体间协议尚在早期，可观察不必跟进。

§16.3 Anthropic Skills 模式

维度	内容
起源	2025, Anthropic
心智	把"能力"打包成"skill" (含 prompt / tools / 示例)
与用户工程的呼应	services/api-server 已有 skills 概念

Skills vs Tools 差异

维度	Tool (工具)	Skill (技能)
粒度	单个函数	一组工具 + prompt + 示例
复用	跨智能体	跨智能体 + 跨用户
配置	代码	配置 / 数据库

小结一行：Anthropic Skills 与 custom_agent 已有 skills 概念相通。

§16.4 MCP 服务器开发实战

写一个 MCP server 给 Claude Code 用

CODE

```
from mcp.server import Server

server = Server("my-company-tools")

@server.tool()
async def search_company_docs(query: str) -> str:
    """Search company documents"""
    return f"Search results for {query}"

@server.tool()
async def check_employee_status(employee_id: str) -> dict:
    """Check employee status"""
    return {"id": employee_id, "status": "active"}

if __name__ == "__main__":
    server.run_stdio()
```

在 Claude Code 配置

CODE

```
{
  "mcpServers": {
    "my-company": {
      "command": "python",
      "args": ["/path/to/my_company_server.py"]
    }
  }
}
```

公开的 MCP server 生态

Server	提供的工具
filesystem	读写文件 / 列目录
github	PR / Issue / Commit
postgres	SQL 查询
memory	长期记忆
sequential-thinking	引导思维链
200+ 社区 server	各种集成

MCP 在 custom_agent 的潜在价值

价值	解释
接入 Claude Code 用户	用户用 Claude Code 时自动有公司知识库工具
跨 LLM 复用工具	同一 server 给 Claude / Cursor / Cline 都用
标准化 Skills 系统	custom_agent 的 skills 可暴露成 MCP server

详见 §19.5。

小结一行：MCP server 开发简单（Python SDK 几行代码）+ 接入广 —— 高 ROI。

§16.5 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	MCP 是 2024.11 以来最重要的协议，工具与 LLM/框架解耦
2	MCP 三层：Host（应用） / Client（嵌入） / Server（独立工具进程）
3	LangGraph 通过 langchain-mcp-adapters 接入 MCP server
4	Anthropic Skills 与 custom_agent 已有 skills 概念相通
5	写一个 MCP server 给 Claude Code 用极易

反模式速记

反模式	解决
工具只为单一 LLM 写	写成 MCP server 跨用
不关注 MCP 进展	这是 2026 最重要的趋势

术语速查

术语	含义
MCP	模型上下文协议
Host / Client / Server	MCP 三层架构
stdio / SSE	MCP 通信传输
A2A / AGNTCY	智能体间协议（早期）
Skill	Anthropic 推的"能力包装"

推荐下一章

下一章: [§17 2025–2026 新兴趋势](#) — — Agent as Code / Ambient Agent / Reasoning Models。

§17 2025–2026 新兴趋势

§17.0 本章定位

§17.1 智能体即代码（Agent as Code）

含义

把智能体定义当源码 —— 走 GitOps（PR 审核 / CI / CD / 版本化）。

维度	老做法	Agent as Code
提示词	散落代码	git 管理
工具	在线注册	代码定义
智能体配置	数据库	YAML / Python
评估	手动	CI 自动跑
部署	点击	git push 触发

工程实践

实践	工具
Prompt 版本管理	Git + LangSmith Hub
Agent CI	GitHub Actions + Promptfoo
Eval 准入	评估指标低于阈值不让合
灰度发布	配合特性开关

小结一行：Agent as Code 把"智能体工程化"提升到与软件工程同等级别 —— 严肃团队 2026 的方向。

§17.2 常驻智能体（Ambient Agent）

含义

不是"用户问 → 智能体答"，而是"智能体后台持续工作"。

模式	触发	例子
用户驱动	用户消息	客服 / 助手
事件驱动	Webhook / 文件上传	自动 PR 审核
时间驱动	Cron	每日报告 / 工单巡检
监控驱动	告警	自动诊断

LangGraph 的支持

能力	API
后台 run	<code>client.runs.create</code>
Cron	<code>langgraph.json</code> <code>crons</code> 字段
长会话	Checkpoint + <code>thread_id</code>
中断恢复	<code>interrupt</code>

小结一行：Ambient Agent 把智能体从"被动应答"升级到"主动工作" —— 是 SaaS 智能体产品的演化方向。

§17.3 MCP 服务器生态爆发

数据

2024.11 MCP 推出至今（约 1.5 年），公开 MCP server 数量增长：

时间	公开 server 数
2024.11	5（官方初始）
2025.06	~80
2026.01	~300+
2026.04	~500+

生态结构

类型	占比
官方 server（Anthropic 维护）	~10
厂商提供（GitHub / Notion / Slack 等）	~50
社区贡献	大头
公司自家私有 server	（不公开）

推荐关注的 MCP server

Server	用途
filesystem	文件操作
github	git / PR / issue
memory	长期记忆
sequential-thinking	思维链工具
postgres / sqlite	数据库
brave-search / google-search	搜索
docker	容器管理
slack / linear	协作工具

小结一行：MCP server 生态正在像 npm 那样爆发 —— 工具不再需要每家自己写。

§17.4 编译式提示词（DSPy 范式扩展到主流）

DSPy（详见 §10.5）的"prompt 当代码编译"思路正逐渐被主流接纳：

工具	DSPy 思路的吸收
LangChain	LangSmith Hub 加自动优化
OpenAI	内部用类似优化（未公开）
Anthropic	Claude 训练时融入

小结一行：编译式提示词从学术圈走向工业 —— 2027 可能成主流。

§17.5 提示词缓存（Prompt Caching）

「节点 ② 省钱神器。」

工作原理

LLM 厂商把 prompt 的"前缀"缓存（比如 system prompt + 大段固定上下文），下次同前缀直接命中缓存，只算"差异部分"的 token。

厂商	支持	价格折扣
Anthropic	是（2024.08）	90% off（缓存命中部分）
OpenAI	是（2024.10）	50% off
Google	是（2024.06）	50% off

用法 (Anthropic)

CODE

```
import anthropic

client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-sonnet-4-6",
    system=[
        """
        "type": "text",
        "text": "<[redacted]>",
        "cache_control": {"type": "ephemeral"},
        """
    ],
    messages=[{"role": "user", "content": "..."}],
)
```

适用场景

场景	缓存效果
多轮对话（系统提示不变）	极好
RAG 智能体（工具描述固定）	好
单次调用	不省（首次还要付全价）
一次性任务	不必开

节点 ② 的成本影响

回到 §7.5 的实测：开 Prompt Caching 后 LangGraph 智能体成本从 \\$.24 降到 \\$.10 —— 省 60%。

小结一行：Prompt Caching 是"白送的钱" —— 2026 任何严肃智能体应用都该开。

§17.6 延展思考模型（Reasoning Models）对智能体设计的冲击

推理模型的特殊

模型	推理能力	节点 ② 时间	节点 ② 成本
GPT-4o	普通	短（1-3 秒）	中
OpenAI o1 / o3	强	长（5-30 秒）	高（5-10x）
Anthropic Extended Thinking	强	长	高
DeepSeek-R1	强	长	中（开源）

对智能体设计的冲击

设计点	普通 LLM	推理模型
节点 ② 时长	几秒	几十秒
节点 ⑦ 循环次数	多（5-10 步）	少（1-3 步，因为单次推理就深）
流式	必要	极必要 （不然用户等 30 秒看不见）
工具调用	多步	推理一次后批量
适合场景	简单 / 多步 / 工具密集	复杂规划 / 数学 / 代码

实际效果

任务	普通 LLM + ReAct 步数	推理模型步数
"查 A 加 B 的天气差"	3 步（查 A / 查 B / 算差）	可能 1 步（思考很深）
"规划一周旅行"	10+ 步迭代	1-2 步给完整计划

设计建议

场景	用什么模型
高频简单查询	快 LLM（Sonnet / GPT-4o-mini）
复杂规划 / 推理	推理模型（o1 / Opus / R1）
智能体路由：简单走快、复杂走推理	混合

小结一行：推理模型让"少而深"的智能体设计成为可能 —— 但需要重新评估循环和流式策略。

§17.7 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	6 大趋势：Agent as Code / Ambient Agent / MCP 生态爆发 / 编译式 prompt / Prompt Caching / Reasoning Models
2	Agent as Code 把工程化提到软件工程级
3	Ambient Agent 是 SaaS 智能体产品的方向
4	MCP server 生态正在像 npm 一样爆发
5	Prompt Caching 是"白送的钱"，2026 必开
6	Reasoning Models 让"少而深"成为可能，需重设计循环和流式

反模式速记

反模式	解决
Prompt 散落代码不走 git	走 Agent as Code
不开 Prompt Caching	多花 50-60% 钱
推理模型不开流式	用户等 30 秒看不见会跑

术语速查

术语	含义
Agent as Code	智能体即代码（GitOps）
Ambient Agent	常驻智能体
Prompt Caching	提示词缓存
Reasoning Model	推理模型
Extended Thinking	延展思考（Anthropic）

推荐下一章

下一章：[§18 选型决策矩阵](#) — 综合所有维度的选型决策树。

§18 选型决策矩阵

§18.0 本章定位

「在基础流程中的位置：横切 8 节点 + 6 层架构 —— 决定每个节点的工具栈选择。

选型不是"哪个框架最强"，是"在你的场景下哪个组合最匹配 8 节点流程的需求"。本章按 4 个维度（语言栈 / 场景 / 部署形态 / 团队规模）拆决策树，最终得到一个综合决策。

§18.1 按团队语言栈

决策树（Mermaid #23）



【Mermaid 图表 — 请在 HTML 中查看交互式渲染】

```
graph TD
    Lang{Lang} --> Python[Python]
    Lang --> TypeScript[TypeScript]
    Lang --> Multi[Multi]

    Python --> PyChoice{PyChoice}
    PyChoice --> Agent[Agent]
    PyChoice --> RAG[RAG]
    PyChoice --> Pyd[Pyd]
    PyChoice --> NLP[NLP]

    TypeScript --> Vercel[Vercel]
    TypeScript --> Mastra[Mastra]

    Multi --> SK[SK]

    style LG_PY fill:#e9d5ff
    style LC_PY fill:#dbeafe
    style Pyd fill:#d1fae5
    style Vercel fill:#fef3c7
    style Mastra fill:#fef3c7
    style SK fill:#fce7f3
```

混合架构是默认答案

场景	推荐组合
Python 后端 + TS 前端	Python LangGraph + Vercel AI SDK 前端流式
Python 全栈	LangGraph + LangChain 组件
TS 全栈	Mastra 或 Vercel AI SDK + Inngest
跨语言（C# / Java / Go）	Semantic Kernel 或自部署 LangGraph Server + REST

小结一行：Python 严肃团队默认 LangGraph + LangChain 组件，TS 团队首选 Vercel AI SDK / Mastra。

§18.2 按场景

场景决策树（Mermaid #24）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Scene1[Scene] --> RAG[RAG]
    RAG --> RAGN[RAGN]
    RAGN --> LlamaIndex[LlamaIndex]
    RAGN --> LangGraph1[LangGraph]
    Scene1 --> Gen[Gen]
    Gen --> LangGraph2[LangGraph]
    Scene1 --> WF[WF]
    WF --> Airflow[Airflow / Temporal]
    WF --> LLM[LLM]
    Scene1 --> CA[CA]
    CA --> ClaudeCode[Claude Code / Cursor]
    Scene1 --> Multi[Multi]
    Multi --> PoC[PoC]
    PoC --> Crew[CrewAI]
    Multi --> LGSup[LangGraph Supervisor]
    Scene1 --> Dify[Dify]
    Scene1 --> Voice[Voice]
    Voice --> OpenAI[OpenAI Realtime]
    Scene1 --> Vis[Vis]
    Vis --> Anthropic[Anthropic Computer Use]
```

```
style RAGN fill:#d1fae5
style Gen fill:#e9d5ff
style WF fill:#fef3c7
style CA fill:#fce7f3
style Crew fill:#dbeeaf
style LGSup fill:#e9d5ff
style Dify fill:#fed7aa
style Voice fill:#ddd6fe
style Vis fill:#fecaca
```

小结一行：场景优先级远高于框架选型 —— 先确定要解决什么场景再选工具。

§18.3 按部署形态

部署	推荐工具
完全自部署	LangGraph + Postgres + Langfuse 自托管
公有云 SaaS	LangGraph Platform / AWS Bedrock / Vertex AI / Azure AI Foundry
边缘计算（低延迟）	Cloudflare Workers AI
私有云 / 国内合规	Dify 自部署 + 自家 LLM
桌面 / 离线	Ollama + LangGraph 嵌入式

小结一行：部署形态约束很大，先看数据合规和成本预算再选。

§18.4 按团队规模

团队规模	推荐
个人 / 1-3 人	直接 OpenAI / Anthropic SDK 或 LangChain v0.3
小队 4-10 人	LangChain + LangGraph + LangSmith
中型 10-50 人	LangGraph + Langfuse + Promptfoo + 评估流水线
大厂 50+	LangGraph + 自研观测 + 自研评估 / 部分自研框架

投入与产出

阶段	学习 + 搭建成本	收益
直接 SDK	1 周	跑起来
LangChain	2-3 周	标准化
LangGraph	4-6 周	生产级
自研框架	6 个月+	完全可控（仅大厂值得）

小结一行：团队规模决定投入上限 —— 1-3 人不要硬上 LangGraph，10+ 人不上反而效率低。

§18.5 总决策树

Mermaid #25

 [Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    Start([Start]) --> Q1{Q1}
    Q1 --> Q2{Q2}
    Q2 --> L1[L1]
    L1 --> Q3{Q3}
    Q3 --> CC[CC]
    CC --> SDK[SDK]
    Q1 --> Q4{Q4}
    Q4 --> Python[Python]
    Python --> Q5{Q5}
    Q5 --> LCNew[LCNew]
    LCNew --> LG[LG]
    LG --> TS[TS]
    TS --> Q6{Q6}
    Q6 --> Mastra[Mastra]
    Mastra --> LG
    LG --> Q7{Q7}
    Q7 --> Sup[Sup]
    Sup --> Single[Single]
    Single --> create_react_agent[create_react_agent]
```

```
style L1 fill:#d1fae5
style CC fill:#fce7f3
style SDK fill:#dbeeaf
style LCNew fill:#f3e5f5
style LG fill:#e9d5ff,stroke-width:3px
style Sup fill:#e9d5ff
style Q6 fill:#fed7aa
```

小结一行：决策树以"是否需要状态循环中断"为第一分水岭、"语言 + 规模"为第二维 ——
LangGraph 是严肃生产 Python 项目的默认。

§18.6 以观测为先（observability-first）选型

严肃团队的真实决策顺序

老套路：选框架 → 写代码 → 上生产 → 才发现没法监控 新套路：先确定能不能监控，再选框架

检查清单（上生产前必须有）

维度	必备能力
Trace（链路）	每次调用的完整树
Cost	按用户 / 项目 聚合
Latency	P50 / P99
Error	异常告警
Eval	自动评估通过率
灰度	A/B 切换能力

上述能力的工具组合

团队偏好	组合
LangChain 生态 + 可上云	LangSmith + LangChain / LangGraph
自托管 / 数据合规	Langfuse + LangGraph
跨框架 + 不绑生态	Helicone + Promptfoo + 自研 dashboard

小结一行：observability-first 是严肃团队的 2026 选型纪律 —— "看不见就不上生产"。

§18.7 本章小结 + 反模式 + 术语 + 推荐下一章

本章小结

#	核心结论
1	选型四个维度：语言栈 / 场景 / 部署 / 团队规模
2	总决策树以"状态循环中断"为分水岭
3	observability-first 是严肃团队纪律
4	LangGraph 是 Python 严肃生产默认，混合架构（LC 组件 + LG 编排）是常态

反模式速记

反模式	解决
看到新工具就换	先看是否解决具体痛点
不评估观测就上	observability-first
跟着 hype 选	跟着场景选

术语速查

术语	含义
Hybrid Architecture	混合架构
Observability-first	观测优先
Vendor Lock-in	厂商锁定
GitOps	Git 驱动运维

推荐下一章

下一章: [§19 全栈对照 custom_agent](#) —— 把全部生态映射到用户工程，给出一年路线图。

§19 全栈对照 custom_agent（选型视角）

§19.0 本章定位

§19.1 项目当前栈映射到生态全景图 + 8 节点流程

现状对照（Mermaid #26）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```

graph TD
    subgraph CA["custom_agent [?][?][?]"]
        FE["apps/web-console  
Next.js"]
        GW["services/gateway"]
        API["services/api-server  
FastAPI"]
        DB["(Postgres)"]
        FE --> GW
        GW --> API
        API --> DB
    end

    subgraph EC["[?][?][?][?][?][?]"]
        E1["[?][?][?][?]Vercel AI SDK [?][?][?]"]
        E2["[?][?][?][?][?][?]LangChain v0.3 / LangGraph"]
        E3["RAG: [?][?][?][?][?][?]"]
        E4["[?][?][?][?][?][?]"]
        E5["[?][?][?][?][?][?]"]
        E6["MCP: [?][?][?]"]
    end

    FE -. "[?][?][?][?][?][?]" E1
    API -. "[?][?][?][?][?][?]" E2
    API -. "[?][?][?][?][?][?]" E3
    API -. "[?][?][?][?][?][?]" E4
    API -. "[?][?][?][?][?][?]" E5
    API -. "[?][?][?][?][?][?]" E6

    style E4 fill:#fecaca,stroke:#ef4444
    style E5 fill:#fecaca,stroke:#ef4444
    style E6 fill:#fecaca,stroke:#ef4444

```

8 节点对照

节点	custom_agent 现状（推测）	对照生态最佳实践
① 上下文	routes/chat.py 拼 messages	LangChain ChatPromptTemplate / LangGraph 节点
② 推理	OpenAI / Anthropic SDK	bind_tools / 推理模型
③ 决策	暂无显式	LangGraph 条件边
④ 工具	Skills 系统	@tool + ToolNode + MCP
⑤ 返回	Skills 调用结果	写回 State
⑥ 状态	Sessions 表	LangGraph Checkpointer + Store
⑦ 循环	暂无	LangGraph recursion_limit + interrupt
⑧ 输出	SSE 流式	LangGraph stream_mode=messages

小结一行：custom_agent 在节点 ①②④⑤⑥⑧ 已有基础，缺节点 ③⑦（决策路由 + 循环控制）。

§19.2 缺什么：评估 / 观测 / MCP 接入是优先级最高的三个空白

三大空白

空白	风险	优先级
观测	生产出问题看不见、调不动	最高
评估	提示词 / 模型变了不知道好坏	最高
MCP 接入	错过 2026 趋势、Skills 系统标准化机会	高
LangGraph 编排	当前直接 SDK 仍能跑	中
Prompt Caching	多花 60% 钱	中（可立即开）
自动化路线 / Agent as Code	提升工程化	中-低

为什么观测和评估是"最高"

观测和评估是其他改进的前提 —— 没有它们就不知道任何改动是否真的"改善了"。

最小可行实现

缺口	最小可行做法
观测	接 Langfuse 自托管，所有 LangChain 调用自动 trace
评估	Promptfoo + 几十个核心用例的 yaml 配置
MCP	把 Skills 系统暴露为一个 MCP server

小结一行：三大空白中观测 / 评估是当下就该补的、MCP 是面向未来的战略投入。

§19.3 建议优先级：不动 / 评估 / 立即引入

三档分级

档	项	时间窗
立即引入	Langfuse 自托管 / Promptfoo 评估 / Prompt Caching	1 个月 内
评估（量化后再决定）	LangGraph / MCP server / Reasoning 模型	2-3 个 月
不动	RAG 实现重写 / 前端框架换 / 数据库换	6+ 个月

立即引入的具体动作

CODE

Week 1: [?][?][?]Langfuse [?][?][?][?]docker compose
Week 2: [?][?]LangSmith trace [?][?][?][?][?]Langfuse
Week 3: [?][?][?][?][?][?][?][?][?]Promptfoo [?][?][?][?]
Week 4: [?][?]Anthropic Prompt Caching ([?][?][?][?][?][?][?][?][?])

评估期的具体动作

CODE

Month 2: [?][?]LangChain v0.3 [?][?][?][?][?][?][?][?][?][?][?][?][?]
Month 3: [?][?][?][?][?]vs [?][?][?][?][?]
[?]
[?][?][?][?][?][?][?][?][?][?][?][?][?][?][?][?]

小结一行：分三档让团队精力集中 —— 立即引入观测和评估，再用量化决定是否上 LangGraph。

§19.4 一年路线图建议

季度路线图（Mermaid #27）



[Mermaid 图表 — 请在 HTML 中查看交互式渲染]

```
graph TD
    title custom_agent
    dateFormat YYYY-MM-DD
    section 
    Langfuse a1, 2026-05-01, 14d
    Promptfoo a2, after a1, 14d
    Prompt Caching a3, after a2, 7d
    section LangChain
    LCEL b1, 2026-07-01, 30d
    bind_tools + Skills b2, after b1, 21d
    section LangGraph
    create_react_agent c1, 2026-10-01, 30d
    Postgres Checkpointer c2, after c1, 14d
    HITL c3, after c2, 21d
    section MCP
    Skills MCP server d1, 2027-01-01, 30d
    Claude Code d2, after d1, 14d
```

各季度核心目标

季度	核心目标	关键产出
2026 Q2	观测和评估打底	Langfuse + Promptfoo 上线 / Prompt Caching 开启
2026 Q3	LangChain 现代化	LCEL 重写关键路由 + bind_tools 适配 Skills
2026 Q4	LangGraph 试点	试点 1-2 个会话场景上 LangGraph + Checkpointer
2027 Q1	MCP 战略接入	Skills 暴露为 MCP server，接入 Claude Code 用户

小结一行：一年路线图按"观测 → LangChain → LangGraph → MCP"四步走。

§19.5 MCP 接入路径：未来对接 Claude Code / Cursor 客户端

接入策略

custom_agent 的 Skills 系统暴露为 MCP server 后，可以让用户在 Claude Code / Cursor / Cline 等客户端里直接调用 —— 这是巨大的获客和黏性机会。

实现步骤

CODE

```

???mcp-server-customagent (Python)
?????Skills API
?????Claude Code ???
?????
    "mcpServers": {
        "customagent": {
            "command": "uvx",
            "args": ["mcp-server-customagent"],
            "env": {"CUSTOM_AGENT_API_KEY": "..."}
        }
    }
?????
?????
?????
?????Claude Code ?????X",
    Claude Code ???customagent server ???

```

价值预估

指标	现状	接 MCP 后
用户接入门槛	Web Console 或 API	IDE / 终端里直接用
获客	营销 + 产品官网	+ Claude Code / Cursor 用户群
黏性	中	高 (IDE 集成)

小结一行： MCP 是 custom_agent 触达 Claude Code / Cursor 用户的最便宜路径 —— 优先级高于 LangGraph 编排。

§19.6 评估与回归测试基础设施缺口：Promptfoo 最小可行实现

最小评估流水线

CODE

```
description: customagent [?]
prompts:
  - file://prompts/system.txt

providers:
  - id: anthropic:claude-sonnet-4-6
    config:
      apiKey: ${ANTHROPIC_API_KEY}

tests:
  - description: [?]
    vars:
      input: "[?]"
    assert:
      - type: contains
        value: "[?]"
      - type: latency
        threshold: 5000
      - type: cost
        threshold: 0.05
      - type: llm-rubric
        value: "[?]"

  - description: [?]
    vars:
      input: "[?]"
    assert:
      - type: contains
        value: "[?]"
```

CI 集成

CODE

```
name: Promptfoo Evaluation
on:
  pull_request:
    paths: ['prompts/**', 'src/agent/**']

jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npx promptfoo eval --config .promptfoo/customagent-eval.yaml --output result.json
      - run: |
          PASS_RATE=$(jq '.results.stats.successes / .results.stats.total' result.json)
          if [ $(echo "$PASS_RATE < 0.9" | bc) -eq 1 ]; then exit 1; fi
```

小结一行： Promptfoo + CI 让 prompt 改动可被自动评估 —— 上线前自动门槛。

§19.7 本章小结 + 反模式 + 术语 + 全书完结语

本章小结

#	核心结论
1	custom_agent 在 8 节点上有 ①②④⑤⑥⑧ 基础，缺 ③⑦
2	三大空白：观测 / 评估 / MCP 接入
3	三档优先级：立即引入（观测评估） / 评估期（LangGraph、推理模型） / 不动（RAG 重写）
4	一年路线：观测打底 → LangChain → LangGraph → MCP
5	MCP 是触达 Claude Code / Cursor 用户的最便宜路径

反模式速记

反模式	解决
不上 Langfuse / Promptfoo 直接做大改动	先打底
跳过 LangChain 直接上 LangGraph	先 LCEL 标准化
一上来重写所有 Skills 为子图	简单 Skill 转 @tool 即可
不开 Prompt Caching	立即开
不接 MCP	错过 2026 最大机会

术语速查

术语	含义
Roadmap	路线图
MCP server	MCP 服务器（可暴露 Skills）
Eval gate	评估门槛（CI 中通过率）
Hybrid Architecture	混合架构

全书完结语

读到这里，你应该形成了完整的智能体心智模型：

- 1. **基础流程 8 节点**是全书脊梁 —— 任何智能体都在解决这 8 个节点上的具体问题
- 2. **三件套定位明确**：ReAct 是范式（精神层）、LangChain 是组件库（货架层）、LangGraph 是编排引擎（蓝图层）
- 3. **业界主流玩法**：LangChain 组件 + LangGraph 编排（混合架构），约 60% 项目采用
- 4. **生态 16 大类各司其职** —— 通用框架管全流程、专门派管特定节点、基础设施横切观察、协议规范化接口
- 5. **2025-2026 趋势**：MCP 成事实标准、Reasoning 模型重塑设计、Prompt Caching 是白送的钱
- 6. **custom_agent 落地**：观测和评估先行 → LangChain → LangGraph → MCP 四步走

接下来可去附录查找：

- [附录 A 接口速查卡](#) — LangChain LCEL / LangGraph StateGraph 一页对照
- [附录 H 全书英文术语总表](#) — 字典级速查
- [附录 I Anthropic 五模式映射 8 节点](#) — 设计模式速查
- [附录 J LangChain → LangGraph 迁移速查表](#)
- [附录 K 三家工具调用协议对照](#)

附录

附录 A — 接口速查卡

LangChain LCEL 速查

```

from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o", temperature=0)

llm_with_tools = llm.bind_tools([tool1, tool2])

structured_llm = llm.with_structured_output(MyPydanticModel)

# Prompt
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "..."),
    ("placeholder", "{chat_history}"),
    ("user", "{input}"),
])

from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()

# LCEL
chain = prompt | llm | parser

result = chain.invoke({"input": "..."})
async for chunk in chain.astream({"input": "..."}):
    print(chunk)

robust = chain.with_retry(stop_after_attempt=3).with_fallbacks([backup_chain])

from langchain_core.tools import tool

@tool
def search(query: str) -> str:
    return ...

```


LangGraph StateGraph 速查

```

from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END, add_messages
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import ToolNode, create_react_agent
from langgraph.types import Command, Send

# State [?]
class State(TypedDict):
    messages: Annotated[list, add_messages]
    counter: int

[?] [?] [?] [?] [?]
def node(state: State) -> dict:
    return {"messages": [AIMessage("...")]}

[?] [?] [?] [?] [?]
def router(state: State) -> str:
    return "next" if state["counter"] > 5 else "stop"

[?] [?] [?] [?]
graph = StateGraph(State)
graph.add_node("a", node_a)
graph.add_node("b", node_b)
graph.add_edge(START, "a")
graph.add_conditional_edges("a", router, {"next": "b", "stop": END})
graph.add_edge("b", "a") # [?]
app = graph.compile(checkpointer=MemorySaver())

[?] [?] [?]
config = {"configurable": {"thread_id": "1"}, "recursion_limit": 25}
result = app.invoke({"messages": [...]}, config)

[?] [?] [?]
async for chunk in app.astream({...}, config, stream_mode="messages"):
    print(chunk)

# Send [?]
def fanout(state):
    return [Send("worker", {"item": x}) for x in state["items"]]

# Command [?] [?] [?] [?] [?]
def smart_node(state):
    return Command(update={"messages": [...]}, goto="next")

[?] [?] [?] [?] [?]
for state in app.get_state_history(config):
    print(state.values)

```

```
# prebuilt
agent = create_react_agent(llm, tools, checkpoint=MemorySaver())
```

附录 B — 参考链接

论文

- ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2022)
- Chain-of-Thought Prompting Elicits Reasoning (Wei et al., 2022)
- Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023)
- Tree of Thoughts (Yao et al., 2023)
- Pregel: A System for Large-Scale Graph Processing (Google, 2010)

官方文档

- LangChain : <https://python.langchain.com>
- LangGraph : <https://langchain-ai.github.io/langgraph/>
- LangSmith : <https://smith.langchain.com>
- Anthropic Building Effective Agents : <https://www.anthropic.com/research/building-effective-agents>
- MCP : <https://modelcontextprotocol.io>

业界博客

- Klarna: AI assistant handles 2/3 of customer service chats
- Anthropic: How we built Claude Code
- Replit Agent technical breakdown

附录 C — 各章反模式汇总速查

章	反模式	解决
§0	把 workflow 叫成 "智能体"	简单场景用 workflow
§0	期望 LLM 自己记得过去对话	用 Memory / State
§1	用 ReAct 字符串解析做新项目	用 Function Calling
§1	给 ReAct 不加 max_iterations	必须设上限
§2	学 2023 教程的 LLMChain 写法	用 LCEL
§2	输出解析手写 PydanticOutputParser	用 with_structured_output
§3	用 initialize_agent 写新代码	create_tool_calling_agent 或迁 LangGraph
§3	工具内异常直接抛	工具内 try/except
§4	不 compile 直接用 StateGraph	必须 graph.compile()
§4	节点函数返回完整 State	只返回需要更新的字段
§5	不传 thread_id 用 checkpointer	必须 configurable.thread_id
§5	MemorySaver 上生产	用 PostgresSaver
§6	State 字段无 Reducer 时 Send 并行写	用 operator.add 或 add_messages
§6	用 Swarm 跑简单任务	先 Supervisor
§7	简单 prompt 任务也上 LangGraph	直接 SDK
§7	不开 Prompt Caching	必开
§8	一上来就用独立 agent service	先嵌入 api-server

章	反模式	解决
§10	多智能体直接上 Swarm	先 Supervisor
§11	用通用框架做 RAG 重活	用 LlamaIndex
§13	在 Dify 里硬怼复杂逻辑	复杂部分写代码
§14	不是云客户却选其 Agents 平台	自部署 LangGraph
§15	不评估直接上生产	必须先建评估集
§16	工具只为单一 LLM 写	写成 MCP server
§17	Prompt 散落代码不走 git	走 Agent as Code
§18	跟着 hype 选	跟着场景选
§19	跳过 LangChain 直接上 LangGraph	先 LCEL 标准化

附录 D — 版本时间线

LangChain

版本	时间	关键变化
v0.0	2022.10	初始发布
v0.0.x	2023	高频重构
v0.1	2024.01	拆包 core / community / partners
v0.2	2024.05	LCEL 成熟 / with_structured_output / 推荐迁 LangGraph
v0.3	2024.09	AgentExecutor 退役

LangGraph

版本	时间	关键变化
0.0.x	2024.01	初始发布
0.1	2024.05	StateGraph + Reducer + Checkpoint
0.2	2024.10	Functional API + Send + Store + 子图
0.2.x	2025-2026	Command + 推理模型 + Platform

生态里程碑

时间	事件
2022.01	思维链（CoT）论文
2022.10	ReAct 论文 / LangChain 创建
2023.03	AutoGPT 走红
2023.04	BabyAGI
2023.06	OpenAI Function Calling
2024.05	Anthropic Tool Use
2024.10	Anthropic Computer Use / OpenAI Swarm
2024.11	Anthropic MCP
2024.12	OpenAI o1 / Anthropic Building Effective Agents
2026.Q1	LangGraph Platform GA

附录 E — 完整生态版本快照表 (2026.04)

工具	当前版本	最后核对
LangChain	0.3.x	2026.04
LangGraph	0.2.x	2026.04
LangSmith	SaaS	2026.04
Langfuse	3.x	2026.04
LlamaIndex	0.12.x	2026.04
Haystack	2.x	2026.04
Semantic Kernel	1.x	2026.04
CrewAI	0.x	2026.04
AutoGen / AG2	0.x	2026.04
OpenAI Agents SDK	1.x	2026.04
PydanticAI	1.x	2026.04
DSPy	2.x	2026.04
Burr	0.x	2026.04
Vercel AI SDK	4.x	2026.04
Mastra	0.x	2026.04
Dify	1.x	2026.04
Promptfoo	0.x	2026.04
Inspect AI	0.x	2026.04

工具	当前版本	最后核对
MCP Python SDK	1.x	2026.04
langchain-mcp-adapters	0.x	2026.04

▮ 版本快照随时间过期，请以官方文档为准。

附录 F — 中外生态对比表

维度	海外主流	中国主流
通用框架	LangChain / LangGraph	自研 + LangChain
多智能体	CrewAI / AutoGen	MetaGPT
低代码	Flowise / Langflow / n8n	Dify / Coze / 百度 AppBuilder
模型	OpenAI / Anthropic / Google / Meta	通义千问 / 智谱 GLM / DeepSeek / Kimi / 文心一言
评估	Braintrust / Inspect AI	多在自研
观测	LangSmith / Langfuse	Langfuse 自托管
编程类 Agent	Claude Code / Cursor / Devin	通义灵码 / 文心快码

附录 G — Mermaid 索引 (31 张)

#	章节	内容
1	§0.7	8 节点基础流程主图
2	§0.10	三件套学习地图
3	§1.1	纯 prompt 输出混杂
4	§1.3	ReAct Thought-Action-Observation 内循环
5	§2.4	LCEL 管道符 vs 旧 LLMChain
6	§3.1	LangChain AgentExecutor 黑盒
7	§3.2	LangChain Tool Calling Agent 流程
8	§4.1	LangChain 痛点 → LangGraph 解法对照
9	§4.5	LangGraph 三要素关系
10	§5.1	LangGraph 手写 ReAct 循环
11	§5.3	检查点写入时序
12	§5.4	HITL 暂停-恢复时序
13	§6.1	Reducer 三种语义
14	§6.2	Send 动态扇出
15	§6.3	Supervisor 拓扑
16	§6.3	Plan-Execute 拓扑
17	§6.3	Swarm 拓扑
18	§6.6	流式时序

#	章节	内容
19	§7.3	三件套选型决策树
20	§8.4	接入 custom_agent 集成架构
21	§9.1	16 大类生态全景
22	§16.1	MCP 三层架构
23	§18.1	按语言栈选型
24	§18.2	按场景选型
25	§18.5	总决策树
26	§19.1	custom_agent 现状对照
27	§19.4	一年路线图 Gantt
28	(预留)	—
29	§10.6	各派别在 8 节点上的着力点
30	§15.6	横切层与框架解耦

附录 H — 全书英文术语总表

「字典级速查。每个英文术语 → 中文释义 → 首次出现章节。」

A

术语	中文	出现章节
Agent	智能体	§0.1
AgentExecutor	智能体执行器	§3.1
Agent Scratchpad	智能体草稿板	§1.4
Action / Action Input	行动 / 行动输入	§1.2
<code>add_messages</code>	消息追加器	§6.1
<code>add_conditional_edges</code>	加条件边	§4.5
Ambient Agent	常驻智能体	§17.2
<code>ainvoke</code> / <code>astream</code> / <code>abatch</code>	异步执行接口	§4.6
Anthropic	Anthropic 公司	§0
API	应用编程接口	—
AutoGPT	自主 GPT	§1.7

B

术语	中文	出现章节
BabyAGI	婴儿通用智能	§1.7
<code>bind_tools</code>	绑定工具	§2.13
<code>BinaryOperatorAggregate</code>	二元运算累加通道	§6.8
<code>BaseStore</code>	存储基类	§6.5
BM25	经典关键字检索算法	§0.3.4

C

术语	中文	出现章节
Chain	链	§2
<code>ChatModel</code> / <code>ChatPromptTemplate</code>	聊天模型 / 提示词模板	§2.3
Chain-of-Thought (CoT)	思维链	§0.6
Channel	通道	§6.8
Checkpoint / Checkpointer	检查点 / 保存器	§5.3
<code>Command</code>	命令对象	§6.1
CompiledGraph	已编译图	§4.6
Computer Use	计算机使用	§11.4
Context Window	上下文窗口	§0.5
Context Engineering	上下文工程	§0.6
Cron	定时调度	§6.13
CrewAI	CrewAI 框架	§10.3

D / E

术语	中文	出现章节
Dataclass	数据类	§6.1
DSPy	编译式提示词框架	§10.5
Edge	边	§4.3
Embedding	向量嵌入	§6.5
END	终点常量	§4.5
<code>entrypoint</code>	入口装饰器	§6.4
Eval / Evaluation	评估	§15.2
Extended Thinking	延展思考	§17.6

F / G / H

术语	中文	出现章节
FFN (Feed-Forward Network)	前馈网络	§0.1
FSA	有限状态自动机	§0.2
Function Calling	函数调用	§0.4
Functional API	函数式接口	§6.4
GQA (Grouped-Query Attention)	分组查询注意力	§0.1
<code>GraphRecursionError</code>	图递归错	§5.6
GRPO	组相对策略优化 (RL 算法)	§0.4
Hallucination	幻觉	§0.3
Handoff	交接	§10.3
Haystack	Haystack 框架	§10.1
HITL (Human-in-the-Loop)	人在回路	§5.4
Hybrid Retrieval	混合检索	§0.3.4

I / J / L

术语	中文	出现章节
Instructor	Instructor 库	§10.4
<code>interrupt</code> / <code>interrupt_before</code>	中断 / 静态断点	§5.4
<code>invoke</code> / <code>stream</code> / <code>batch</code>	同步执行接口	§4.6
LangChain / LangGraph / LangSmith	LangChain 系列	§2 / §4
Langfuse	开源观测平台	§15.3
LCEL (LangChain Expression Language)	LangChain 表达式语言	§2.4
LLM (Large Language Model)	大语言模型	§0.3
LlamaIndex	LlamaIndex 框架	§11.1

M / N / O

术语	中文	出现章节
Map-Reduce	映射-归约	§6.2
Matryoshka Embeddings	俄罗斯套娃嵌入	§0.3.4
Memory	记忆	§2.7
MCP (Model Context Protocol)	模型上下文协议	§16.1
MQA (Multi-Query Attention)	多查询注意力	§0.1
MTEB	嵌入评测基准	§0.3.4
Multi-Head Attention	多头注意力	§0.1
Multimodal	多模态	§11.6
Node	节点	§4.3
<code>NodeInterrupt</code>	动态中断	§5.4
Observation	观察	§1.2
Observability	可观测性	§15.3
OpenAI Agents SDK	OpenAI 智能体 SDK	§10.3
Outlines	约束解码引擎	§0.2
OutputParser	输出解析器	§2.9



术语	中文	出现章节
<code>PostgresSaver</code>	Postgres 检查点	§5.3
PPO	近端策略优化 (RL 算法)	§0.4
Pregel	超步并行模型	§4.4
Prompt Caching	提示词缓存	§17.5
Prompt Engineering	提示词工程	§0.6
<code>PromptTemplate</code>	提示词模板	§2.3
Promptfoo	Promptfoo 评估工具	§15.2
PydanticAI	PydanticAI 框架	§10.4
<code>reasoning_effort</code>	思考预算参数	§0.4
Reasoning Model	推理模型	§17.6
ReAct	推理-行动范式	§1
<code>recursion_limit</code>	递归上限	§5.6
Reducer	归约器	§6.1
Reflexion	反思范式	§1.5
Reranker	重排序器	§0.3.4
Residual Connection	残差连接	§0.1
Retriever	检索器	§2.3
RoPE (Rotary Position Embedding)	旋转位置编码	§0.1

术语	中文	出现章节
Routing	路由	§10.7
RRF (Reciprocal Rank Fusion)	倒数排名融合	§0.3.4
Runnable	可运行对象	§2.6

S

术语	中文	出现章节
Sandbox	沙箱	§15.1
Self-Attention	自注意力	§0.1
Self-Consistency	自我一致性	§0.6
Semantic Kernel	语义内核	§10.1
<code>Send</code>	发送原语	§6.2
Sinusoidal	正弦位置编码	§0.1
Skill	技能	§16.3
<code>SqliteSaver</code>	SQLite 检查点	§5.3
START	起点常量	§4.5
State / StateGraph	状态 / 状态图	§4.3
Store API	存储接口（跨会话）	§6.5
Streaming	流式	§2.11
Structured Decoding	结构化解码	§0.2
Subgraph	子图	§6.2
Supervisor	监督者	§6.3
Swarm	蜂群	§6.3

| T / V / W / Z / 其他

术语	中文	出现章节
Layer Norm	层归一化	§0.1
Verifier	验证器（推理模型 RL 用）	§0.4
xgrammar	高性能约束解码引擎	§0.2
---	---	---
Thought / Action / Observation	思考 / 行动 / 观察	§1.2
Token	词元	§0.5
Tool / <code>@tool</code>	工具 / 工具装饰器	§0.4
Tool Use	工具使用（Anthropic）	§0.4
<code>ToolNode</code>	工具节点	§5.1
<code>thread_id</code>	会话标识	§5.3
Tree-of-Thoughts (ToT)	思维树	§1.5
TypedDict	类型化字典	§6.1
Vector Store	向量存储	§2.3
Vercel AI SDK	Vercel AI SDK	§12.1
<code>with_structured_output</code>	结构化输出	§2.9
<code>with_retry</code> / <code>with_fallbacks</code>	重试 / 回退	§2.12
Workflow	工作流	§0.1
Zero-shot / Few-shot	零样本 / 少样本	§0.6

附录 I — Anthropic Building Effective Agents 五大模式映射 8 节点

「 2024.12 Anthropic 官方博客提出，已成业界设计纲领。

模式 1：提示词链（Prompt Chaining）

维度	内容
节点	② 推理 串行
拓扑	$A \rightarrow B \rightarrow C$ 线性
适用	任务可分解为固定步骤
例子	翻译 → 校对 → 格式化
用 LangChain	LCEL: <code>prompt1 \ llm \ prompt2 \ llm</code>
用 LangGraph	简单链可以，但复杂用图

模式 2：路由（Routing）

维度	内容
节点	③ 决策
拓扑	根据输入分流到不同处理
适用	不同输入类别需要不同处理
例子	客服分流（订单 / 退款 / 投诉）
用 LangChain	<code>RunnableBranch</code>
用 LangGraph	<code>add_conditional_edges</code> + <code>Command</code>

模式 3：并行化（Parallelization）

维度	内容
节点	⑦ 循环 + ④ 并发
拓扑	同任务多角度同时处理后合并
适用	投票 / 多视角验证
例子	三个模型分别评估代码安全性后投票
用 LangChain	<code>RunnableParallel</code>
用 LangGraph	<code>Send</code> API

模式 4：协调-工作者（Orchestrator-Workers）

维度	内容
节点	②③ 中央协调 + ④ 工作执行
拓扑	协调者拆任务 → 工作者执行 → 协调者综合
适用	任务复杂、需要动态拆解
例子	长文档摘要（拆段 → 摘要 → 整合）
用 LangChain	难（需手写）
用 LangGraph	Supervisor 模式

模式 5：评估-优化（Evaluator-Optimizer）

维度	内容
节点	⑥ 状态 + ⑦ 循环
拓扑	生成 → 评估 → 改进的循环
适用	输出质量需要迭代提升
例子	写代码 → 跑测试 → 修代码
用 LangChain	难
用 LangGraph	Plan-Execute / Reflexion

五模式组合

复杂智能体往往是这五种的组合：

组合	例子
Routing + Orchestrator-Workers	客服分流后多智能体协作
Prompt Chaining + Evaluator-Optimizer	文档生成 + 自动评分 + 改进
Parallelization + Evaluator-Optimizer	多模型生成 + 投票 + 最优

附录 J — LangChain → LangGraph 迁移速查表

API 对照

老 (LangChain v0.x)	新 (LangGraph 0.x)
<code>from langchain.agents import create_tool_calling_agent</code>	<code>from langgraph.prebuilt import create_react_agent</code>
<code>agent = create_tool_calling_agent(llm, tools, prompt)</code>	(合并到下一行)
<code>executor = AgentExecutor(agent=agent, tools=tools)</code>	<code>agent = create_react_agent(llm, tools, prompt=prompt)</code>
<code>result = executor.invoke({"input": q})</code>	<code>result = agent.invoke({"messages": [{"role": "user", "content": q}]})</code>
<code>chat_history</code> 字段	<code>messages</code> 字段 (统一为 BaseMessage 列表)
<code>agent_scratchpad</code> 占位	LangGraph State 自动管理
<code>max_iterations=8</code>	<code>config={"recursion_limit": 8}</code>
<code>handle_parsing_errors=True</code>	不需要 (结构化字段无解析错)
<code>return_intermediate_steps=True</code>	默认 messages 都返回
<code>ConversationBufferMemory()</code>	<code>MemorySaver()</code> 或更高级 saver
<code>verbose=True</code>	LangSmith trace 或 <code>astream_events</code>

Memory → Checkpointer 迁移

老	新
<code>ConversationBufferMemory(memory_key="chat_history")</code>	<code>MemorySaver()</code> + <code>thread_id</code>
<code>ConversationSummaryMemory(...)</code>	自定义节点写摘要到 State
<code>ConversationKGMemory(...)</code>	自定义 + Store API
<code>VectorStoreRetrieverMemory(...)</code>	Store API + embedding

完整迁移示例

CODE

```
#####
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, memory=memory, max_iterations=8)
result = executor.invoke({"input": "..."})

#####
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_react_agent(
    llm, tools,
    prompt=prompt,
    checkpointer=MemorySaver(),
)
result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"configurable": {"thread_id": "1"}, "recursion_limit": 8},
)
```


附录 K — OpenAI / Anthropic / Google 工具调用三家协议对照

OpenAI Function Calling (2023.06) / Tools (2023.11)

CODE

```
response = openai.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "??????"},
    tools=[{
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "?????",
            "parameters": {
                "type": "object",
                "properties": {"city": {"type": "string"}},
                "required": ["city"],
            },
        },
    },
    "????????????????",
    "?????????????",
    "?????????",
    "?"
)
# response.choices[0].message.tool_calls = [
#     {"id": "call_abc", "type": "function",
#      "function": {"name": "get_weather", "arguments": '{"city": "???"}}'}
# ]
```

Anthropic Tool Use (2024.05)

CODE

```
response = anthropic.messages.create(
    model="claude-sonnet-4-6",
    messages=[{"role": "user", "content": "👉👉👉👉👉"},
    tools=[{
        "name": "get_weather",
        "description": "👉👉👉👉👉
        "input_schema": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
👉👉👉👉👉👉👉👉
👉👉👉👉👉👉
👉
# response.content = [
#     {"type": "tool_use", "id": "toolu_xyz",
#     "name": "get_weather", "input": {"city": "👉👉👉}}
👉👉👉
```

Google Function Declarations (2024)

CODE

```
response = genai.GenerativeModel("gemini-2.5-pro").generate_content(
👉👉👉👉👉👉👉👉👉👉
    tools=[genai.protos.Tool(
        function_declarations=[genai.protos.FunctionDeclaration(
            name="get_weather",
            description="👉👉👉👉👉
            parameters=genai.protos.Schema(
                type=genai.protos.Type.OBJECT,
                properties={"city": genai.protos.Schema(type=genai.protos.Type.STRING)},
                required=["city"],
👉👉👉👉👉👉👉👉👉👉👉
👉👉👉👉👉👉👉👉👉
👉👉👉👉👉👉👉
👉
# response.candidates[0].content.parts[0].function_call =
#     name="get_weather", args={"city": "👉👉👉}
```

字段对照表

维度	OpenAI	Anthropic	Google
入参字段名	<code>tools</code>	<code>tools</code>	<code>tools</code>
工具描述键	<code>function.parameters</code>	<code>input_schema</code>	<code>parameters</code>
输出标识	<code>tool_calls[i].id</code>	<code>tool_use.id</code>	(无显式 ID)
工具名	<code>function.name</code>	<code>name</code>	<code>name</code>
参数	<code>function.arguments</code> (JSON 字符串)	<code>input</code> (dict)	<code>args</code> (dict)
并行调用	是	是	部分模型支持
结果回填	<code>role: tool, tool_call_id,</code> <code>content</code>	<code>role: user, type:</code> <code>tool_result,</code> <code>tool_use_id</code>	<code>role: function,</code> <code>parts</code>

跨厂商兼容工具（推荐）

工具	作用
LangChain <code>bind_tools</code>	自动适配三家
Anthropic MCP	协议层完全跨家
Pydantic（参数定义）	三家都从 Pydantic 模型生成 schema

「 全书完结

共 §0-§19 二十章 + 附录 A-K 十一项 / 30 张 Mermaid 图 / 约 25 万字。
持续更新：每季度核对一次版本号、每年盘点一次趋势变化。